Покроковий посібник до курсового проєкту

«Основи інтернету речей та аналітика великих даних»

Версія від 09.11.2026

github.com/kostiasSav/kursova-iot

Покроковий посібник до курсового проєкту

Цей посібник проводить вас через увесь курсовий проєкт — від встановлення Python до захисту. Він доповнює методичні вказівки: вказівки кажуть, **що** треба зробити і як це оцінюється, а посібник — **як** саме це зробити, крок за кроком.

Посібник розрахований на те, що ви ніколи раніше не працювали з терміналом, Docker чи MQTT. Якщо щось уже вмієте — просто пропускайте відповідні кроки.

Як читати посібник

Кожен крок побудовано однаково:

1. **Що робимо і навіщо** — одним-двома реченнями.
2. **Команда або код** — копіюйте як є.
3. **Що маєте побачити** — справжній вивід програми. Якщо у вас інакше, зупиніться і подивіться розділ Типові проблеми.

Позначки в тексті:

- **Перевірка.** Контрольна точка: не йдіть далі, доки тут не все гаразд.
- **Увага.** Місце, де найчастіше помиляються.
- **У звіт.** Що з цього кроку треба внести в пояснювальну записку.

Команди наведено окремо для **Windows**, **macOS** і **Linux** там, де вони відрізняються. Якщо окремих варіантів немає — команда однакова для всіх.

Посібник є у двох видах: вебсторінки на GitHub і PDF для друку (`docs/posibnyk-studenta.pdf`). **Команди копіюйте з вебверсії**: у PDF довгі команди переносяться на наступний рядок і після копіювання не спрацюють.

`NNN` у командах — це **ваш** номер варіанта з трьох цифр, наприклад `047`. Приклади виводу в посібнику зроблено на **навчальному наборі** `tuning.parquet` — він однаковий для всіх, тож ви можете отримати точно такі самі числа і переконатися, що все працює, ще до того, як візьметеся за свій варіант.

Зміст

Крок	Що робимо	Орієнтовно
0. Підготовка комп'ютера	Python, редактор, заготовка, ваші дані, перша перевірка	2 год
1. Вузол IoT	ESP32 у Wokwi, MQTT, дослідження захищеності каналу	5 год
2. Конвеєр даних	сховище, вимірювання, обробка, потоковий вхід	14 год
3. Детектори	базовий і призначений детектор, точність і повнота	6 год
4. Реальні дані	набір Intel Lab, хибні спрацювання, очищення	5 год
5. Пошук інцидентів	ваш варіант, `findings.json`, самоперевірка	10 год
6. Дашборд	Grafana, Streamlit або Plotly Dash	6 год
7. Пояснювальна записка	структура, картка результатів, джерела	10–14 год
8. Захист	що буде на захисті і як підготуватися	1–2 год
9. Типові проблеми	помилки і як їх виправити	—

Разом — близько 60 годин. Не залишайте все на останній тиждень: стадії 2 і 5 найбільші, а Docker і бібліотеки іноді доводиться встановлювати не з першого разу.

Орієнтовний план за тижнями

Тиждень	Що зробити
1	Крок 0 і стадія 1
2–3	Стадія 2
4	Стадія 3
5	Стадія 4
6–7	Стадія 5
8	Стадія 6
9–10	Пояснювальна записка, репетиція захисту

Що вам знадобиться

- Один комп'ютер з Windows 10/11, macOS або Linux. Другий пристрій не потрібен.
- Щонайменше 8 ГБ оперативної пам'яті і 10 ГБ вільного місця на диску.
- Інтернет для завантаження програм і даних.
- Ваш номер варіанта і посилання на ваші файли — їх надсилає керівник.

Усе програмне забезпечення в проєкті безкоштовне. Платити за будь-що не потрібно, зокрема за Wokwi.

Що здається наприкінці

1. Пояснювальна записка, 25–30 сторінок.
2. Репозиторій з усім вашим кодом.
3. Файл `findings.json` — знайдені інциденти.

4. Працездатний дашборд і його знімки в додатках.
5. Презентація на 7 хвилин.

Як усе це оцінюється, описано в розділі 8 методичних вказівок.

Чесна гра

Посібник дає майже готовий код для всього, крім одного: пошуку інцидентів у **вашому** наборі даних. Цю частину ви робите самі, і саме її перевіряє захист: там вам дадуть незнайомий набір даних і попросять запустити ваш конвеєр при комісії.

Користуватися системами штучного інтелекту дозволено для написання коду, пошуку помилок і опрацювання літератури — із зазначенням цього у звіті.

Крок 0. Підготовка комп'ютера

Мета: встановити Python, отримати заготовку проєкту і свої дані, переконатися, що все запускається. Займе близько двох годин, більшість із яких — очікування завантажень.

0.1. Термінал за п'ять хвилин

Майже всі дії в проєкті виконуються командами в **терміналі** — вікні, куди вводять текстові команди. Боятися його не треба: ви просто копіюєте команду з посібника, вставляєте у вікно і натискаєте Enter.

Як відкрити термінал:

- **Windows:** натисніть клавішу Windows, наберіть `PowerShell` і відкрийте «Windows PowerShell».
- **macOS:** натисніть Cmd+Пробіл, наберіть `Terminal` і натисніть Enter.
- **Linux:** Ctrl+Alt+T.

Чотири команди, яких достатньо на весь проєкт:

Команда	Що робить
`cd назва`	перейти в папку
`cd ..`	піднятися на рівень вище
`ls`	показати вміст поточної папки (працює і в PowerShell)
`pwd`	показати, в якій папці ви зараз

Порада. Якщо поставите редактор VS Code (див. 0.3), відкривайте термінал прямо в ньому: меню **Terminal** → **New Terminal**. Він одразу відкриється в папці проєкту, і `cd` майже не знадобиться.

Увага. У посібнику команди наведено **без** запрошення `$` чи `PS C:\>` на початку. Копіюйте лише саму команду.

0.2. Встановлення Python

Підходить Python **3.12, 3.13, 3.14 або 3.15**; рекомендуємо **3.14** — на ній перевірено весь посібник. Виняток — Mac з процесором Intel: там потрібна **3.13** (див. нижче).

Якщо у вас уже стоїть 3.15, її можна лишити. Одна заувага для Windows: якщо вам призначено Dask, бібліотеку PyYAML, яку він потребує, pip збиратиме з вихідного коду. Це має пройти саме; якщо з'явиться помилка — див. «Типові проблеми».

Якщо Python потрібної версії у вас уже є, перейдіть одразу до перевірки наприкінці цього кроку.

Windows

1. Відкрийте https://www.python.org/downloads/windows/. У розділі **Stable Releases** знайдіть найвищий номер **Python 3.14.x** (на момент написання — 3.14.8) і натисніть під ним **Download**

Windows installer (64-bit). Ця сама версія працює і на ноутбуках з ARM-процесором (Snapdragon): Windows запускає її сама, а збірки всіх бібліотек курсу є саме для неї.
2. Запустіть завантажений файл. **На першому екрані обов'язково поставте позначку «Add python.exe to PATH»**, потім натисніть **Install Now**.
3. Закрийте і знову відкрийте PowerShell, щоб він «побачив» Python.

Ілюстрована інструкція від розробників Python: https://docs.python.org/3/using/windows.html.

macOS

1. З'ясуйте, який у вас процесор: яблучко в лівому верхньому куті екрана → **Про цей Mac**. Якщо в рядку «Чип» написано Apple M1, M2 чи новіший — вам потрібна версія **3.14**. Якщо в рядку «Процесор» є слово **Intel** — версія **3.13**: для 3.14 на Mac з Intel немає готової збірки бібліотеки, без якої не працює метод Matrix Profile.
2. Відкрийте https://www.python.org/downloads/macos/, знайдіть найвищий номер потрібної версії (наприклад, **Python 3.14.8** або **Python 3.13.16**) і натисніть під ним **Download macOS installer**.
3. Запустіть інсталятор і пройдіть усі кроки з типовими налаштуваннями.
4. Наприкінці відкриється тека з назвою версії, наприклад **Python 3.14**. Двічі клацніть у ній **Install Certificates.command**: без цього Python не зможе завантажувати файли з інтернету (знадобиться на стадії 4).

Якщо у вас уже є Homebrew, можна й так: `brew install python@3.14` (на Mac з Intel — `brew install python@3.13`). Тоді на кроці 0.5 створюйте середовище командою з номером версії: `python3.14 -m venv .venv` (або `python3.13`).

Linux (Ubuntu, Debian, Mint)

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
python3 --version
```

Ubuntu 24.04 і новіші, Debian 13 та Mint 22 ставлять придатну версію. Якщо `python3 --version` показує 3.11 або меншу (Ubuntu 22.04, Debian 12, Mint 21), встановіть новішу з репозиторію deadsnakes (лише Ubuntu і Mint):

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt install python3.12 python3.12-venv
```

і далі в посібнику пишіть `python3.12` замість `python3` (лише на кроці 0.5, де створюється віртуальне середовище).

На Debian 12 та інших системах, де потрібної версії немає в репозиторіях, Python найпростіше поставити інструментом **uv**:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Відкрийте новий термінал і виконайте:

```
uv python install 3.14
```

Тоді на кроці 0.5 замість `python3 -m venv .venv` виконайте `uv venv --seed --python 3.14 .venv`, решта — без змін.

Перевірка

Windows:

```
python --version
```

macOS і Linux:

```
python3 --version
```

Маєте побачити номер версії, наприклад:

```
Python 3.14.8
```

Увага. Надалі в посібнику на Windows пишіть `python`, а на macOS і Linux — `python3` там, де команда запускає Python **поза** віртуальним середовищем (це лише крок 0.5). Усередині віртуального середовища на всіх системах працює просто `python`.

0.3. Редактор коду (рекомендовано)

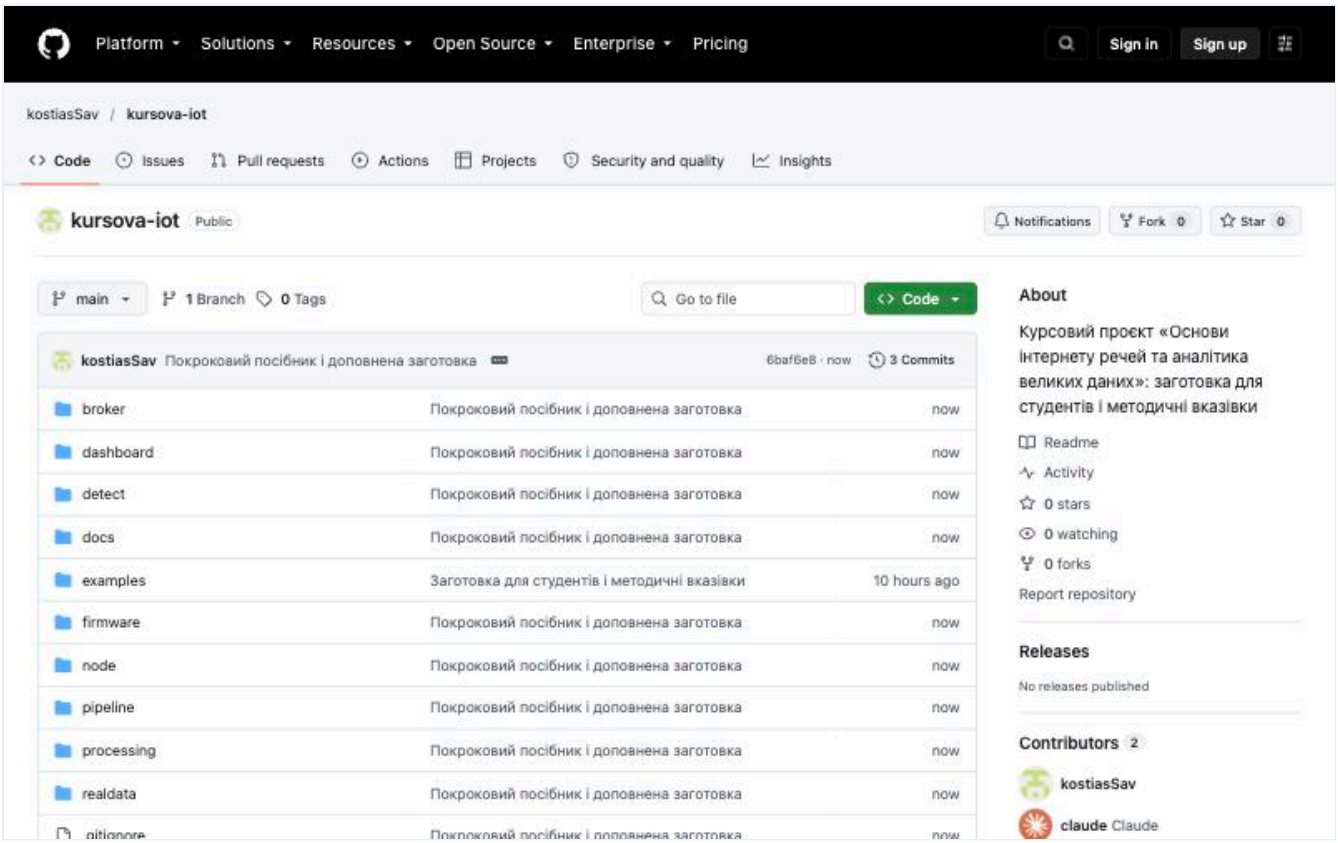
Код можна редагувати будь-де, але найзручніше — у безкоштовному **Visual Studio Code**: https://code.visualstudio.com/.

Після встановлення відкрийте вкладку **Extensions** (Ctrl+Shift+X, на macOS Cmd+Shift+X), знайдіть **Python** від Microsoft і натисніть **Install**.

Коли отримаєте заготовку (крок 0.4), відкрийте її папку у VS Code: **File** → **Open Folder…** і виберіть `kursova-iot`. Відтепер **Terminal** → **New Terminal** відкриває термінал одразу в папці проєкту.

0.4. Отримання заготовки проєкту

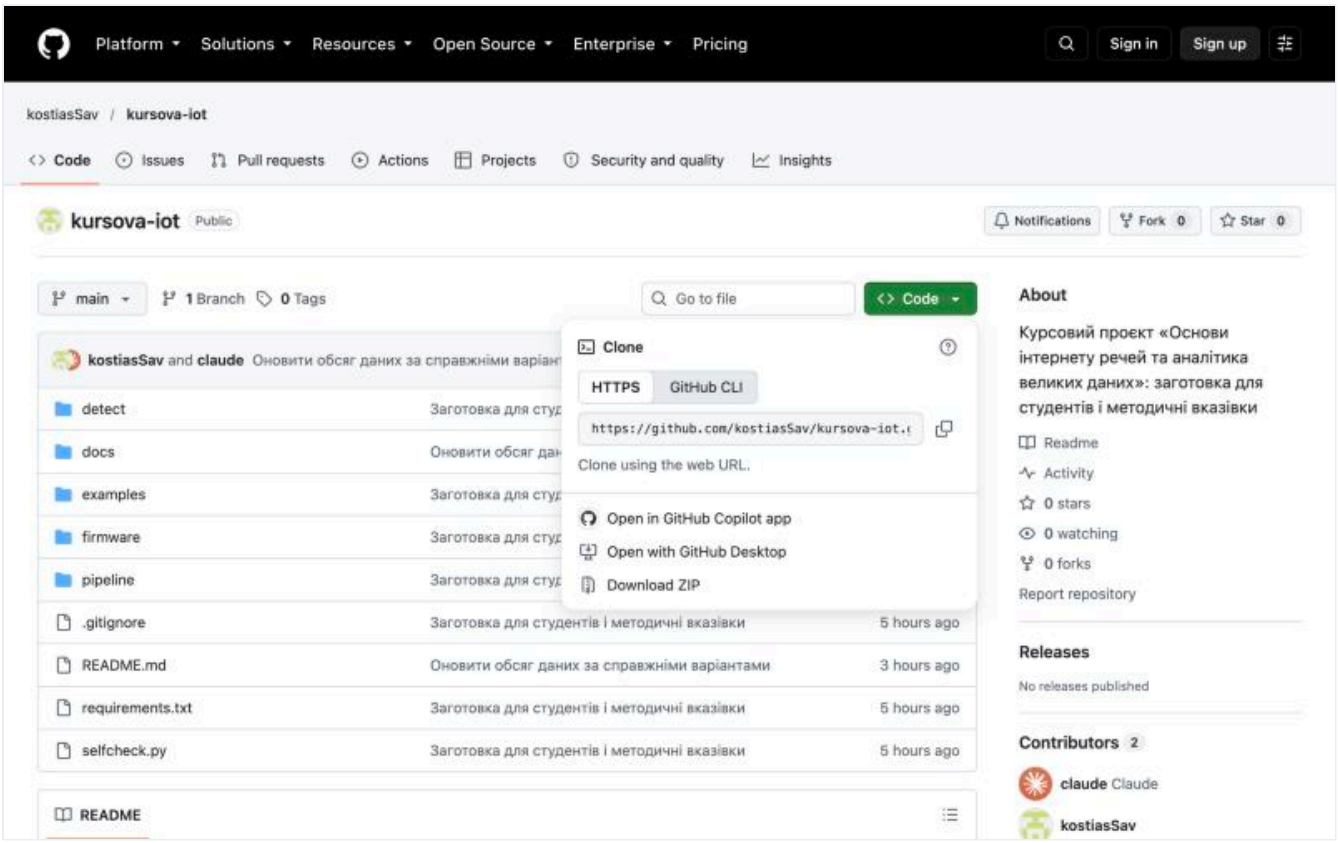
Заготовка лежить у відкритому репозиторії https://github.com/kostiasSav/kursova-iot.



Варіант A — без Git (найпростіше)

1. Відкрийте сторінку репозиторію.
2. Натисніть зелену кнопку **Code** і виберіть **Download ZIP**.
3. Розпакуйте архів (Windows: правою кнопкою → **Видобути все…**; macOS: подвійний клік). Вийде папка `kursova-iot-main`. Перейменуйте її на `kursova-iot` і перенесіть у **просту папку без кирилиці й пробілів у шляху**: - Windows — у корінь диска C:, щоб шлях був `C:\kursova-iot` (не в «Документи» чи на Робочий стіл: кирилиця в шляху і синхронізація OneDrive заважають Docker); - macOS і Linux — у домашню папку, щоб шлях був `~/kursova-iot`.

Windows іноді створює папку в папці: `kursova-iot-main\kursova-iot-main`. Переносьте **внутрішню** — ту, у якій лежить `selfcheck.py`.



Варіант Б — через Git

Якщо ви вже користуєтеся Git (https://git-scm.com/downloads):

```
git clone https://github.com/kostiasSav/kursova-iot.git
```

Клонуйте в ту саму просту папку: на Windows спершу `cd C:\`, на macOS і Linux — у домашній папці.

Свою роботу зберігайте у **приватному** репозиторії на GitHub (створіть новий, а не Fork: форк відкритого репозиторію не може бути приватним) і дайте доступ керівнику — репозиторій з кодом здається разом із запискою. Не публікуйте свої дані й результати (`findings.json`, `my_findings.csv`): передавати їх іншим студентам заборонено (п. 9 вказівок).

Що має вийти

Новий термінал відкривається в домашній папці. Перейдіть у папку проєкту і перегляньте її вміст.

Windows (PowerShell):

```
cd C:\kursova-iot
ls
```

macOS і Linux:

```
cd ~/kursova-iot
ls
```

```
README.md
broker
dashboard
```

```
detect
docs
examples
firmware
make_findings.py
node
pipeline
processing
realdata
requirements.txt
run_all.py
selfcheck.py
```

(PowerShell показує те саме таблицею, з датами й розмірами.)

Перевірка. Ви бачите `selfcheck.py` і `requirements.txt` — отже, ви в правильній папці. Усі подальші команди виконуються **з цієї папки**.

0.5. Віртуальне середовище і бібліотеки

Віртуальне середовище — це окрема «коробка» з бібліотеками для цього проєкту, щоб вони не змішувалися з іншими програмами на вашому комп'ютері.

Створення (один раз)

Windows:

```
python -m venv .venv
```

macOS і Linux:

```
python3 -m venv .venv
```

Активація (щоразу, коли відкриваєте новий термінал)

Windows (PowerShell):

```
.\.venv\Scripts\Activate.ps1
```

macOS і Linux:

```
source .venv/bin/activate
```

Після активації на початку рядка терміналу з'явиться `(.venv)`.

Увага (Windows). Якщо PowerShell відповідає червоним текстом «running scripts is disabled on this system», виконайте один раз:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

підтвердіть літерою `Y` і повторіть активацію.

Встановлення бібліотек

```
pip install -r requirements.txt
```

Займає від пів хвилини до кількох хвилин. Наприкінці не має бути червоних рядків з `ERROR`. Перевірити, що все встановилося:

```
pip list
```

У списку мають бути, серед інших, `cbor2`, `cryptography`, `duckdb`, `matplotlib`, `numpy`, `pandas`, `paho-mqtt`, `psutil`, `pyarrow`, `scipy`.

Увага. Бібліотеки для **вашого** сховища, інструмента обробки, детектора і дашборда в `requirements.txt` закоментовані — ви встановите лише ті, що вам призначено, на відповідних стадіях.

0.6. Ваші дані

Керівник надсилає кожному номер варіанта і посилання на п'ять файлів:

Файл	Що це
`variant_NNN.parquet`	**ваш** набір телеметрії, 4,1–7,3 млн записів
`variant_NNN.json`	маніфест: перелік пристроїв і **призначений вам стек**
`tuning.parquet`	навчальний набір, однаковий для всіх
`tuning.json`	маніфест навчального набору
`tuning_truth.json`	відповіді до навчального набору

Завантажте всі п'ять файлів **у папку проєкту** — поруч із `selfcheck.py`. Найпростіше — просто відкрити кожне посилання в браузері і перенести файл із «Завантажень». Або командою (замініть посилання на свої):

Windows (PowerShell):

```
curl.exe -L0 https://github.com/kostiasSav/kursova-iot/releases/download/v1/tuning.parquet
```

macOS і Linux:

```
curl -L0 https://github.com/kostiasSav/kursova-iot/releases/download/v1/tuning.parquet
```

Увага. Файли `.parquet` не відкриваються в Excel чи в «Блокноті» — це нормально. Вони призначені для програм, а не для перегляду очима.

Ваш призначений стек

Відкрийте `variant_NNN.json` у VS Code і знайдіть блок `assigned_stack` (Ctrl+F, на macOS Cmd+F). Наприклад, для навчального набору він такий:

```
"assigned_stack": {
  "storage": "InfluxDB",
  "processing": "DuckDB SQL",
  "detector": "Matrix Profile",
  "dashboard": "Streamlit"
}
```

Випишіть свої чотири значення — вони визначають, які розділи посібника стосуються саме вас:

Поле	Де знадобиться
`storage` — сховище	стадія 2
`processing` — інструмент обробки	стадія 2
`detector` — призначений детектор	стадія 3
`dashboard` — інструмент візуалізації	стадія 6

0.7. Перша перевірка

Переконаймося, що все працює, на навчальному наборі. Віртуальне середовище має бути активоване (`(.venv)` на початку рядка).

```
python selfcheck.py --findings examples/findings.example.json --data tuning.parquet --
manifest tuning.json
```

Маєте побачити:

```
Картка результатів, перерахована з ваших даних:
  Кількість записів          : 1 157 147
  Пристроїв у маніфесті      : 51
  Пристроїв у даних          : 52
  SHA-256 переліку пристроїв :
1a33058d845b45c2ce0ddff78318cad7cab1f1b8c9a25aed2e3cdbafc84b7098
  (розбіжність на 1 — це не помилка генерації)
  Призначений стек           : storage=InfluxDB, processing=DuckDB SQL, detector=Matrix
Profile, dashboard=Streamlit

Знахідок у файлі: 3

Файл придатний до перевірки.
Це не означає, що знахідки правильні — лише що формат коректний.
```

Тепер те саме для **вашого** варіанта — так ви отримаєте свою картку результатів:

```
python selfcheck.py --findings examples/findings.example.json --data variant_NNN.parquet --
manifest variant_NNN.json
```

Цього разу внизу будуть повідомлення `ПОМИЛКА` про номер варіанта і картку — це очікувано: файл-приклад складено для навчального набору. Важлива верхня частина: **це ваша картка результатів.**

У звіт. Збережіть ці числа: картка розміщується одразу після титульного аркуша пояснювальної записки (п. 6.1 вказівок).

Приклад детектора

```
python detect/example.py
```

```
завантажено записів: 1 157 147; пристроїв: 52

детектор stuck_at: кандидатів — 1
детектор spike_storm: кандидатів — 1

Якість на навчальному наборі (лише реалізовані класи):
  знайдено 2, хибних 0, пропущено 0
  точність 1.00   повнота 1.00   F1 1.00

Усі 12 класів:
  знайдено 2, хибних 0, пропущено 10
  точність 1.00   повнота 0.17   F1 0.29
    ПРОПУЩЕНО  INC-01 rogue_device 2025-09-01T21:09 door-05-mee
    …
```

Приклад знаходить два класи інцидентів із дванадцяти. Решту ви навчитеся шукати на стадіях 3 і 5.

Контрольний список кроку 0

- □ `python --version` (або `python3 --version`) показує 3.12, 3.13, 3.14 або 3.15.
- □ Папка проєкту містить `selfcheck.py`.
- □ Віртуальне середовище створене й активується, `pip list` показує бібліотеки.
- □ П'ять файлів даних лежать у папці проєкту.
- □ `selfcheck.py` друкує вашу картку результатів.
- □ Ви виписали свій призначений стек.

Стадія 1. Вузол інтернету речей

Мета: зібрати в симуляторі Wokwi вузол на ESP32, який надсилає показання давачів через MQTT, і на власному досвіді переконатися, що без шифрування телеметрію може прочитати будь-хто. Складник «Вузол IoT, MQTT, захищеність каналу» — 15 балів; його оцінюють на демонстрації (крок 8.1).

Для вузла потрібні лише браузер і, для збереження проєкту, безкоштовний акаунт Wokwi. Залізо не потрібне.

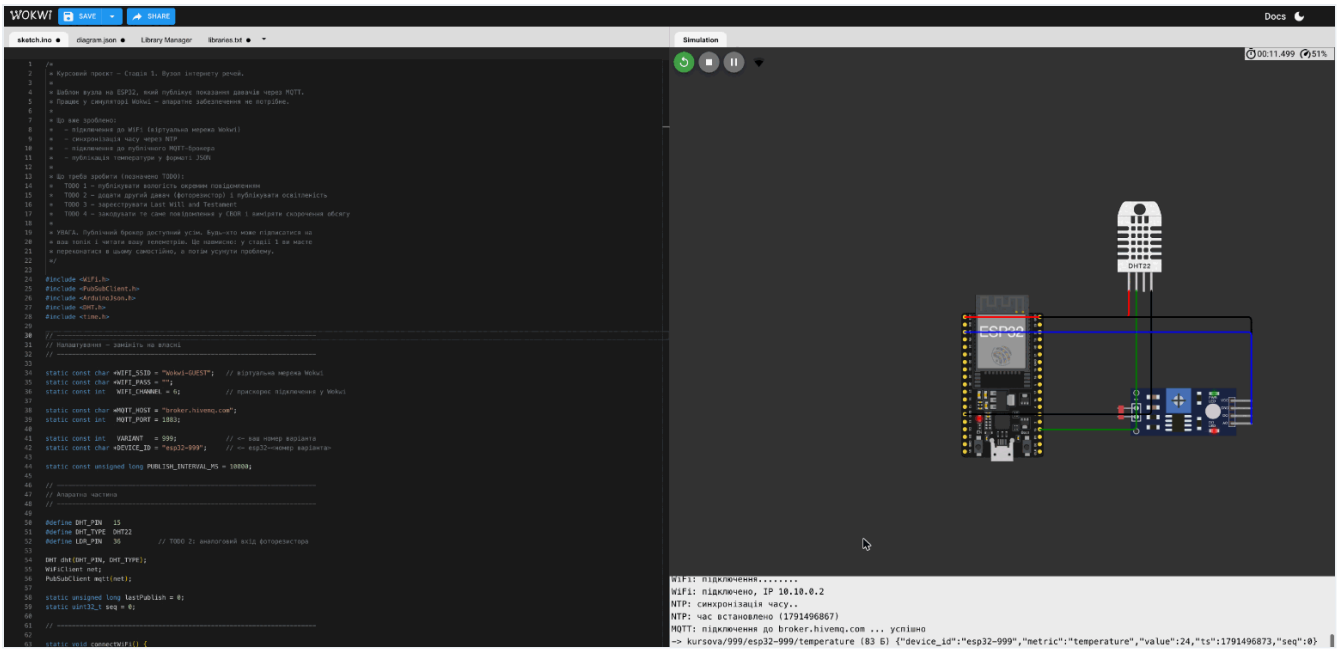
1.1. Проєкт у Wokwi

1. Відкрийте https://wokwi.com/projects/new/esp32. З'явиться редактор з вкладками `sketch.ino` і `diagram.json` ліворуч і симулятором праворуч.
2. Щоб зберігати роботу, натисніть **Sign up** і зареєструйтеся (можна через Google чи GitHub). Це безкоштовно. Без реєстрації симулятор працює, але проєкт зникне, щойно ви закриєте вкладку.
3. Відкрийте у VS Code файл `firmware/sketch.ino` із заготовки, виділіть усе (Ctrl+A, на macOS Cmd+A), скопіюйте і вставте у вкладку `sketch.ino` у Wokwi замість наявного тексту.
4. Те саме зробіть із `firmware/diagram.json` → вкладка `diagram.json`. Праворуч з'являться плата ESP32, давач температури й вологості DHT22 і модуль фоторезистора.
5. Створіть файл бібліотек: натисніть стрілку ▾ праворуч від вкладок, виберіть **New file…**, назвіть файл `libraries.txt` і вставте вміст `firmware/libraries.txt`.
6. Знайдіть на початку `sketch.ino` налаштування і впишіть свій варіант:

```
static const int   VARIANT   = 47;              // <- ваш номер варіанта
static const char *DEVICE_ID = "esp32-047";     // <- esp32-<номер варіанта>
```

7. Натисніть **Save**, потім зелену кнопку ▶ над симулятором.

Перший запуск триває до хвилини: Wokwi компілює прошивку на своєму сервері. Потім унизу з'явиться монітор послідовного порту:



Справжній вивід заготовки (демонстраційний варіант 999; у вас — ваш номер):

```
=== Вузол IoT: курсовий проєкт ===
WiFi: підключення........
WiFi: підключено, IP 10.10.0.2
NTP: синхронізація часу..
NTP: час встановлено (1791496867)
MQTT: підключення до broker.hivemq.com ... успішно
-> kursova/999/esp32-999/temperature (83 Б) {"device_id":"esp32-
999","metric":"temperature","value":24,"ts":1791496873,"seq":0}
```

Перед цими рядками ESP32 друкує кілька службових рядків завантажувача (`ets Jul 29 2019…`, `rst:0x1 (POWERON_RESET)…`) — так і має бути.

Увага. Поки працює симуляція, тримайте вкладку Wokwi **на екрані**. У фоновій вкладці чи під іншим вікном браузер пригальмовує симуляцію (праворуч угорі видно швидкість, наприклад `3%` замість `100%`), і брокер може розірвати з'єднання: у моніторі з'являтимуться повторні `MQTT: підключення…`.

Перевірка. Кожні 10 секунд з'являється новий рядок `->`. Якщо бачите `!!` або MQTT не підключається — розділ Типові проблеми.

1.2. Чотири завдання в прошивці

У `sketch.ino` позначено чотири місця `TODO`. Нижче — код для кожного і пояснення, що він робить. Вставляйте його **замість** відповідного коментаря `TODO`, а потім розберіться в кожному рядку: на захисті про нього спитають.

TODO 1. Вологість окремим повідомленням

Давач DHT22 уже читає вологість у змінну `h`. Опублікуйте її так само, як температуру:

```
  publishMetric("humidity", h);
```

TODO 2. Фоторезистор і освітленість у люксах

Модуль фоторезистора видає напругу, що залежить від освітлення. ESP32 читає її як число від 0 до 4095 (12 біт), що відповідає 0–3,3 В. Формулу переведення в люкси взято з документації Wokwi (https://docs.wokwi.com/parts/wokwi-photoresistor-sensor); там вона наведена для Arduino Uno (5 В, 10 біт), тож ми перерахували її для ESP32:

```
int raw = constrain(analogRead(LDR_PIN), 1, 4094);  // без ділення на нуль
float voltage = raw / 4095.0 * 3.3;
float resistance = 10000.0 * voltage / (3.3 - voltage);
float lux = pow(50 * 1e3 * pow(10, 0.7) / resistance, 1 / 0.7);
publishMetric("illuminance", lux);
```

Під час симуляції клацніть на фоторезистор і посуньте повзунок `lux` — показання в моніторі має змінитися відповідно.

У звіт. Поясніть формулу: дільник напруги з резистором 10 кОм, опір фоторезистора, степенева залежність опору від освітленості (параметри `GAMMA = 0.7`, `RL10 = 50`).

TODO 3. Last Will and Testament

«Останній заповіт» — повідомлення, яке **брокер** публікує замість вузла, якщо вузол зник без попередження (вимкнули живлення, обірвався зв'язок). Його реєструють під час підключення. Замініть рядки

```
    if (mqtt.connect(clientId.c_str())) {
      Serial.println("успішно");
```

на

```
    String willTopic = topicFor("status");
    if (mqtt.connect(clientId.c_str(), willTopic.c_str(), 1, true,
                     "{\"online\": false}")) {
      Serial.println("успішно");
      mqtt.publish(willTopic.c_str(), "{\"online\": true}", true);
```

Аргументи `connect`: ідентифікатор клієнта, топік заповіту, рівень QoS 1, прапорець `retain` (брокер зберігає останнє значення для нових підписників) і текст заповіту. Одразу після підключення вузол сам публікує `{"online": true}` у той самий топік.

Перевірка. Підпишіться на свій топік у браузері (див. 1.3, крок 1) і зупиніть симуляцію кнопкою ■. У топіку `.../status` має з'явитися `{"online": false}` — його надіслав брокер, а не вузол.

TODO 4. CBOR замість JSON — скільки економимо

CBOR (RFC 8949) — двійковий аналог JSON: та сама структура «ключ — значення», але без лапок, дужок і текстових чисел. Повідомлення **публікуються й надалі в JSON** (так вимагає Додаток А); CBOR лише кодується поруч, щоб виміряти різницю в розмірі.

Бібліотеки CBOR для цього не потрібно — нам вистачить трьох коротких функцій. Вставте їх **перед** функцією `publishMetric`:

```
// CBOR (RFC 8949): мінімальний кодувальник рівно для нашого повідомлення.
// Заголовок елемента: 3 біти "основного типу" + довжина або значення.
static size_t cborHead(uint8_t *p, uint8_t major, uint32_t v) {
  if (v < 24)    { p[0] = (major << 5) | v;  return 1; }
  if (v < 256)   { p[0] = (major << 5) | 24; p[1] = v; return 2; }
  if (v < 65536) { p[0] = (major << 5) | 25; p[1] = v >> 8; p[2] = v; return 3; }
  p[0] = (major << 5) | 26;
  p[1] = v >> 24; p[2] = v >> 16; p[3] = v >> 8; p[4] = v;
  return 5;
}

static size_t cborText(uint8_t *p, const char *s) {   // основний тип 3: текст
  size_t len = strlen(s);
  size_t k = cborHead(p, 3, len);
  memcpy(p + k, s, len);
  return k + len;
}

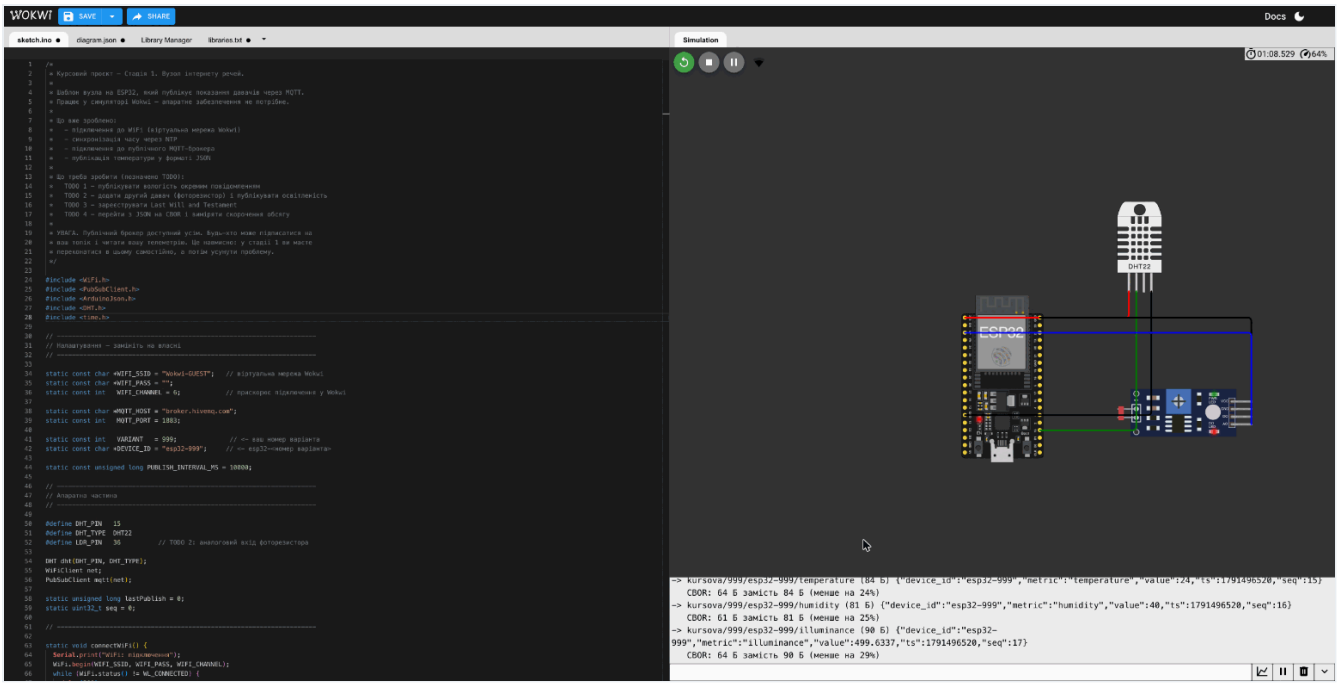
static size_t cborFloat(uint8_t *p, float f) {        // 0xFA: float32
  uint32_t bits;
  memcpy(&bits, &f, sizeof bits);
  p[0] = 0xFA;
  p[1] = bits >> 24; p[2] = bits >> 16; p[3] = bits >> 8; p[4] = bits;
  return 5;
}
```

А на місці коментаря `TODO 4` усередині `publishMetric` — кодування і вивід розмірів:

```
  uint8_t cbor[160];
  size_t m = cborHead(cbor, 5, 5);                    // словник із 5 пар
  m += cborText(cbor + m, "device_id"); m += cborText(cbor + m, DEVICE_ID);
  m += cborText(cbor + m, "metric");    m += cborText(cbor + m, metric);
  m += cborText(cbor + m, "value");     m += cborFloat(cbor + m, value);
  m += cborText(cbor + m, "ts");        m += cborHead(cbor + m, 0, doc["ts"].as<uint32_t>
());
  m += cborText(cbor + m, "seq");       m += cborHead(cbor + m, 0, doc["seq"].as<uint32_t>
());
  Serial.printf("   CBOR: %u Б замість %u Б (менше на %.0f%%)\n",
                (unsigned)m, (unsigned)n, 100.0 * (n - m) / n);
```

Справжній вивід після всіх чотирьох TODO (демонстраційний варіант 999):

```
MQTT: підключення до broker.hivemq.com ... успішно
-> kursova/999/esp32-999/temperature (83 Б) {"device_id":"esp32-
999","metric":"temperature","value":24,"ts":1791496470,"seq":0}
   CBOR: 64 Б замість 83 Б (менше на 23%)
-> kursova/999/esp32-999/humidity (80 Б) {"device_id":"esp32-
999","metric":"humidity","value":40,"ts":1791496470,"seq":1}
   CBOR: 61 Б замість 80 Б (менше на 24%)
-> kursova/999/esp32-999/illuminance (89 Б) {"device_id":"esp32-
999","metric":"illuminance","value":499.6337,"ts":1791496470,"seq":2}
   CBOR: 64 Б замість 89 Б (менше на 28%)
```



Розміри змінюються на байт-два залежно від числа: `"seq":15` на один символ довший за `"seq":3`.

У звіт. Таблиця «показник — розмір JSON — розмір CBOR — економія, %». Потім помножте економію на кількість пристроїв вашого варіанта (з маніфесту), на кількість повідомлень на добу і на 60 діб — скільки трафіку коштує зручність JSON. Подумайте, чому економія лише близько чверті: більшу частину повідомлення займають **назви полів**, а їх CBOR не скорочує.

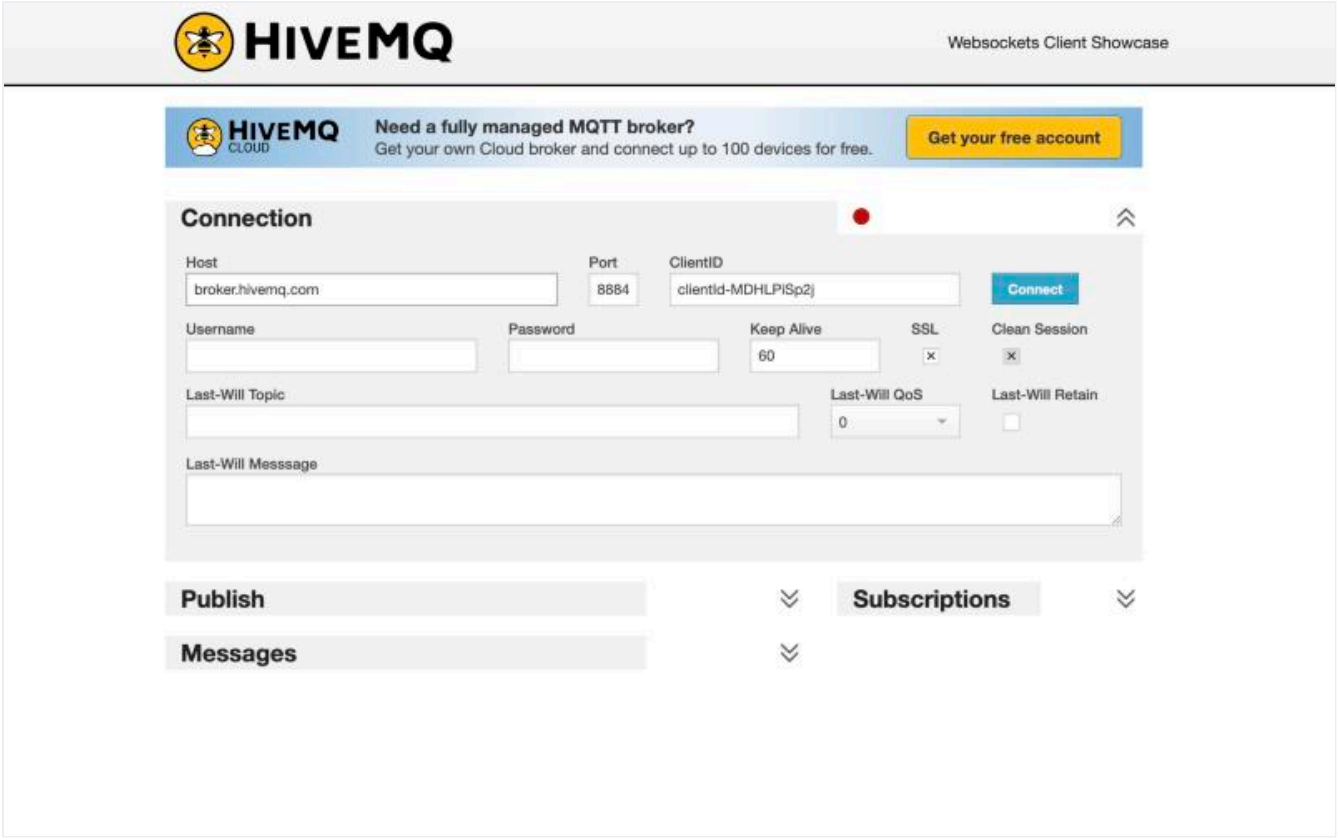
1.3. Дослідження захищеності каналу

Ваш вузол надсилає дані на **публічний** брокер `broker.hivemq.com` без шифрування і без пароля. Три кроки покажуть, що це означає. Усі виконуються на одному комп'ютері.

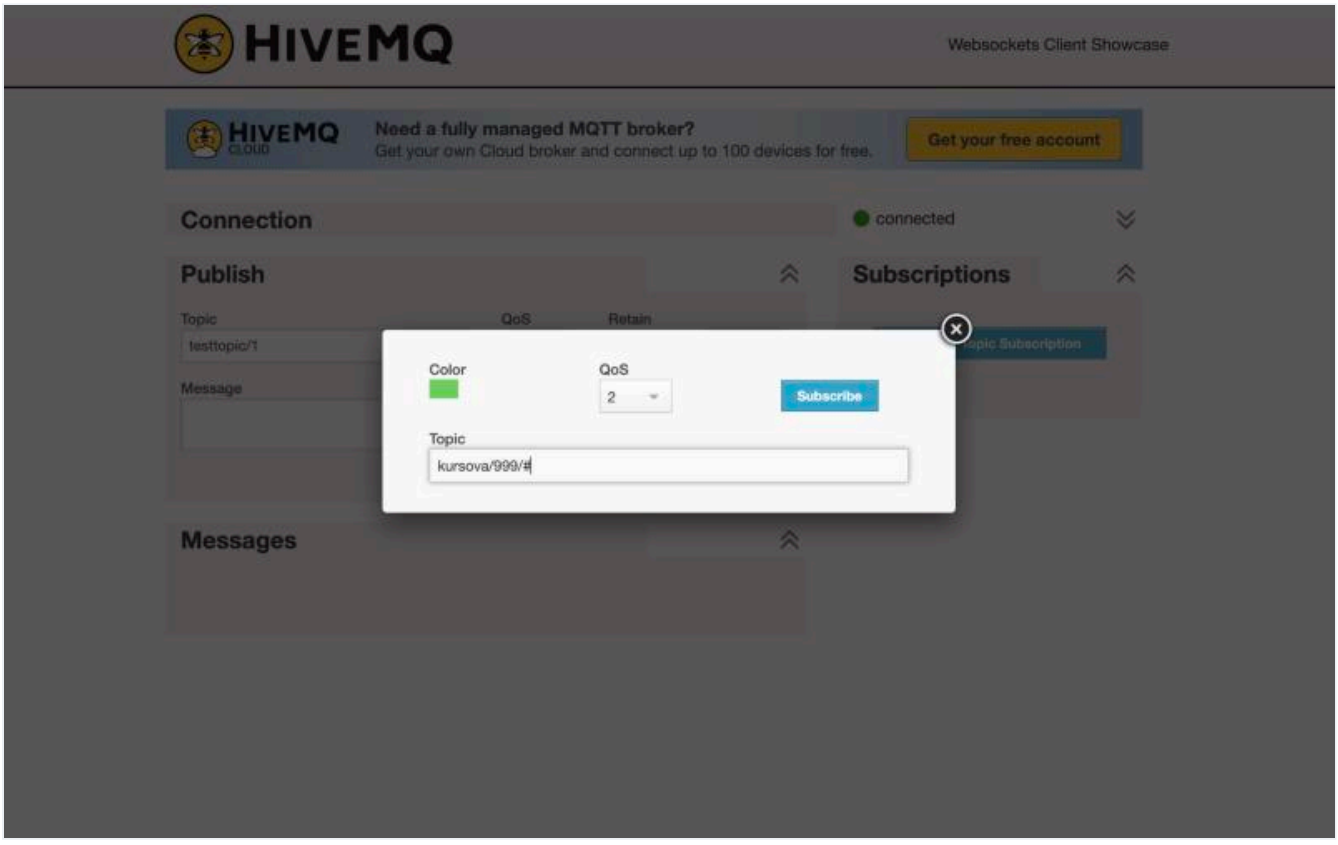
Крок 1. Сторонній клієнт у браузері

Будь-хто в інтернеті може підписатися на ваш топік. Перевірмо це веб-клієнтом MQTT — нічого встановлювати не треба.

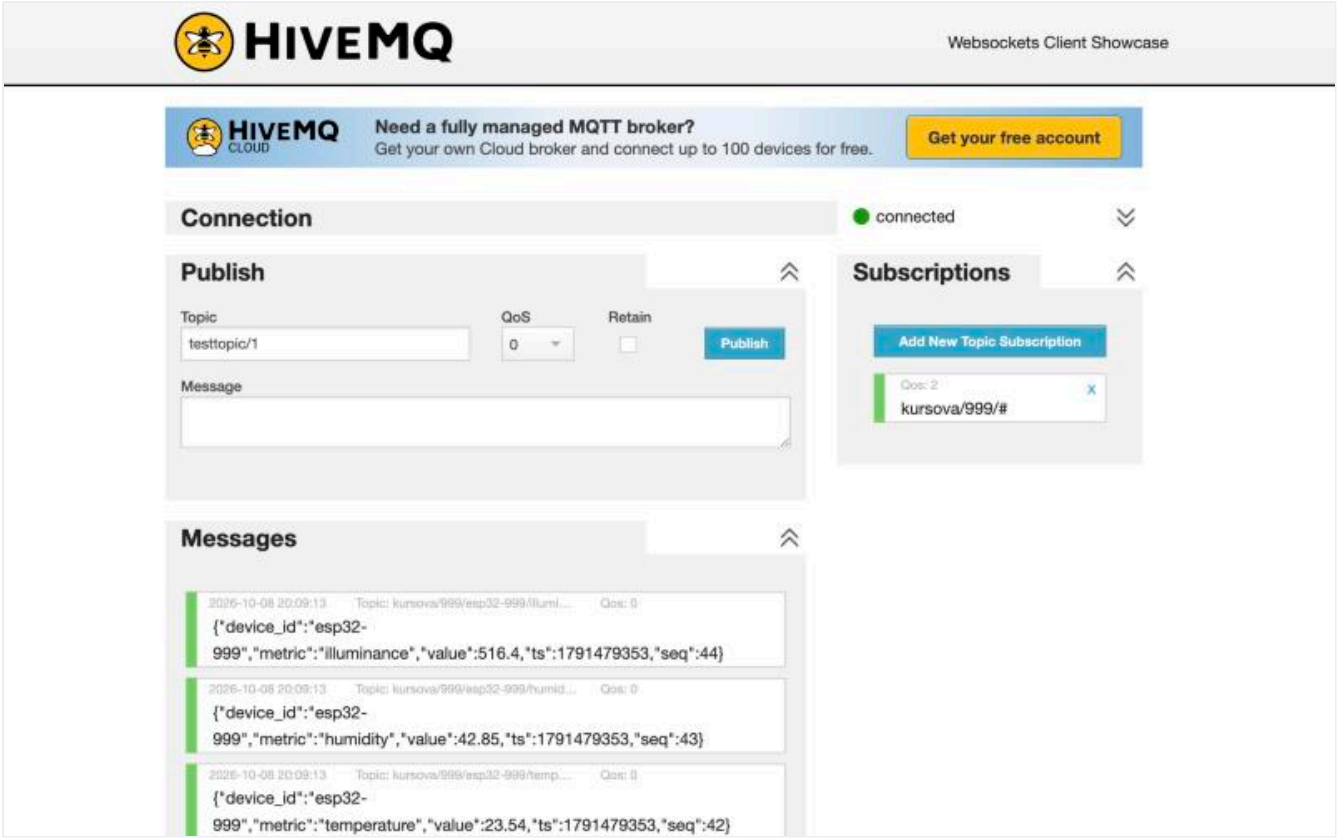
1. Залиште симуляцію в Wokwi запущеною і відкрийте в **іншій вкладці** https://www.hivemq.com/demos/websocket-client/.
2. У полі **Host** впишіть `broker.hivemq.com`, порт залиште `8884`, натисніть **Connect**. Індикатор стане зеленим із написом `connected`.



3. Натисніть **Add New Topic Subscription**, у полі **Topic** впишіть `kursova/<номер>/#` (ваш номер варіанта без нулів попереду, як у прошивці, наприклад `kursova/47/#`) і натисніть **Subscribe**.



4. За кілька секунд у блоці **Messages** з'являться ваші повідомлення — повністю читабельні.



Символ `#` означає «усі підтопіки»: так сторонній бачить усі показники всіх ваших пристроїв однією підпискою.

У звіт. Знімок з повідомленнями і висновок: хто завгодно, знаючи (або вгадавши) назву топіка, отримує ваші дані в реальному часі.

Крок 2. Перехоплення трафіку у Wireshark

Підписка — «чесний» спосіб читати чужі дані: брокер сам їх віддає. Перехоплення — інший: дані дістають безпосередньо з мережевого трафіку, і брокер про це навіть не дізнається. Wokwi вміє записати весь мережевий трафік симульованого ESP32 у файл PCAP, який відкривається програмою Wireshark.

Встановлення Wireshark. Він знадобиться і в кроці 3 — там уже для запису трафіку на вашому комп'ютері, — тому встановлюйте одразу з усім потрібним:

- **Windows.** https://www.wireshark.org/download.html → **Windows x64 Installer** (на ноутбуках з ARM-процесором — **Windows Arm64 Installer**). Під час встановлення погодьтеся встановити **Npcap**: без нього Wireshark не зможе записувати трафік. Додаткових позначок у вікнах Npcap ставити не треба.
- **macOS.** Та сама сторінка → **macOS Universal Disk Image**. Перетягніть Wireshark у теку «Програми», потім у тому самому вікні двічі клацніть **Install ChmodBPF.pkg** — це дозвіл записувати трафік. Якщо Wireshark згодом не показує жодного інтерфейсу, вийдіть із системи й увійдіть знову.
- **Linux.**

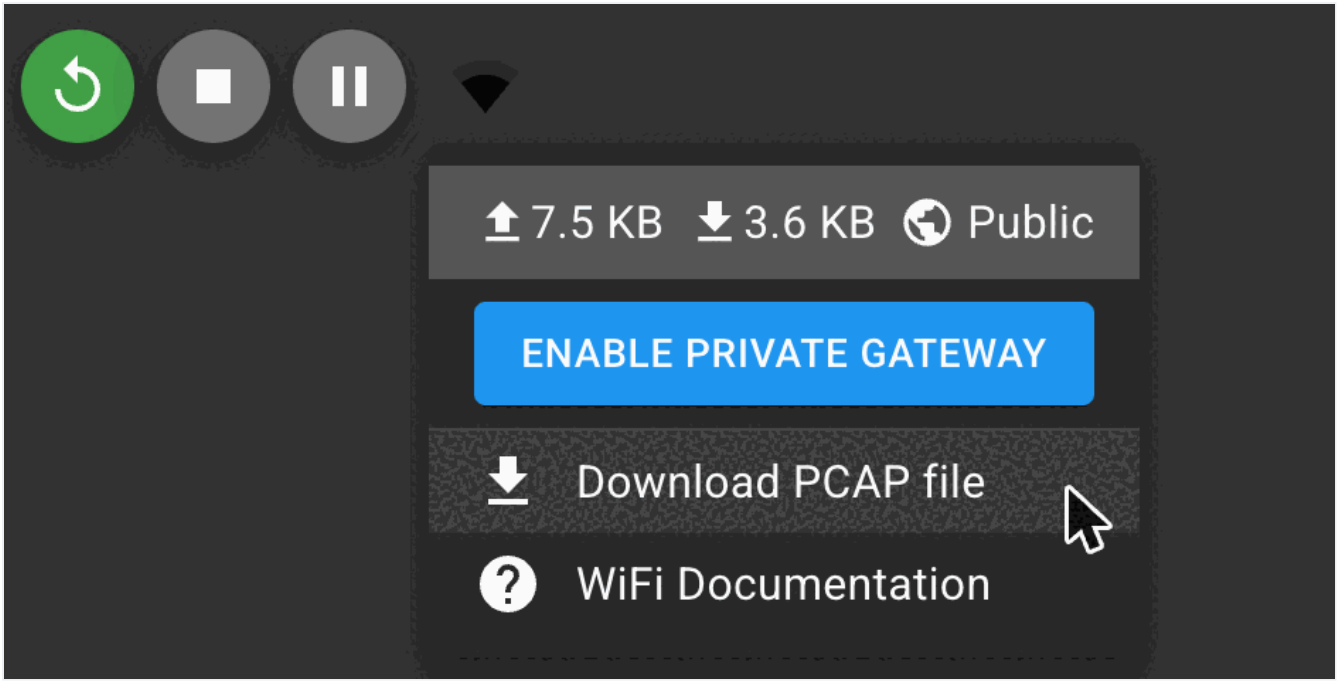
```
sudo apt install wireshark
```

На запитання «Should non-superusers be able to capture packets?» оберіть **Yes**. Потім додайте себе до групи `wireshark` і вийдіть із системи та увійдіть знову:

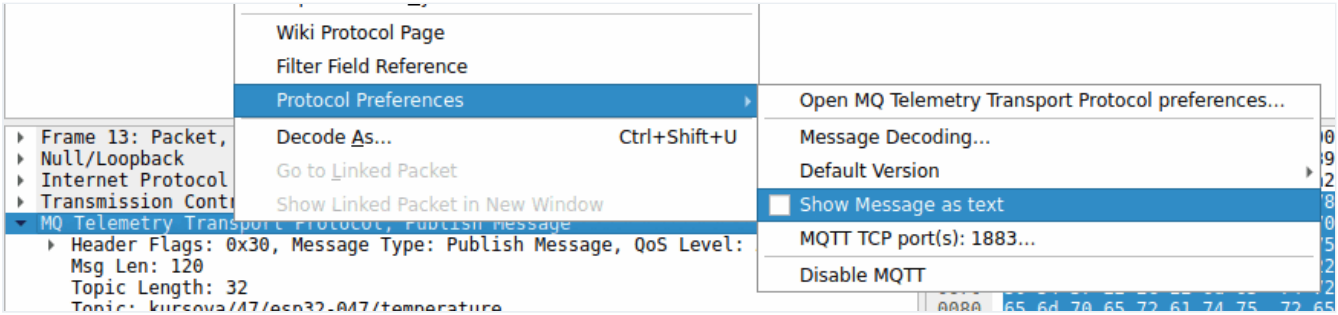
```
sudo usermod -a -G wireshark $USER
```

Перехоплення:

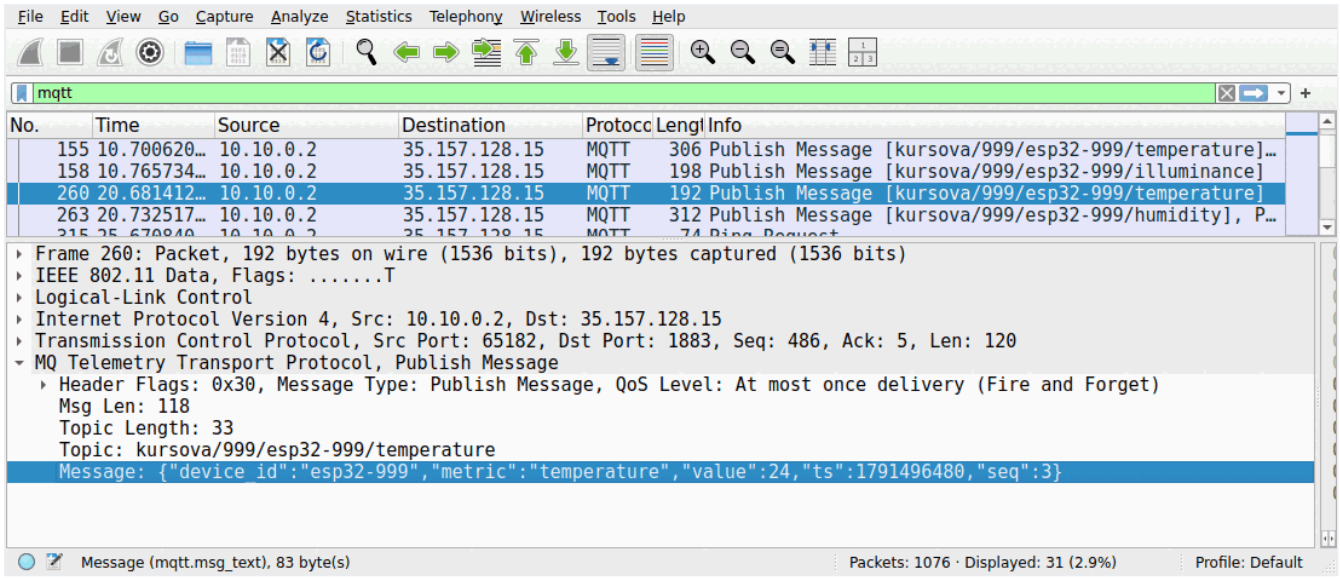
1. Поки симуляція працює хвилину-дві, натисніть у Wokwi значок WiFi — праворуч від кнопок керування симуляцією — і виберіть **Download PCAP file**. Завантажиться файл `wokwi.pcap`. Кнопку **Enable Private Gateway** не натискайте: це платна функція, вона не потрібна.



2. Відкрийте його у Wireshark (**File** → **Open**).
3. У рядку фільтра вгорі (там, де написано *Apply a display filter*) введіть `mqtt` і натисніть Enter. Залишаться лише пакети MQTT.
4. Клацніть будь-який рядок `Publish Message`. У нижній лівій панелі клацніть трикутник ▸ біля **MQ Telemetry Transport Protocol** — розгорнуться поля повідомлення: **Topic** і **Message**.
5. Поле **Message** спершу показано шістнадцятковими цифрами (`7b2264657669...`). Щоб побачити текст, клацніть **правою** кнопкою рядок **MQ Telemetry Transport Protocol** і виберіть **Protocol Preferences** → **Show Message as text**. Wireshark запам'ятає це налаштування.



Тепер у полі **Message** — ваш JSON відкритим текстом:



У стовпцях **Source** і **Destination** — адреса симульованого ESP32 (`10.10.0.2`) і адреса брокера, порт `1883`. Крім MQTT, у записі є службові пакети WiFi, DHCP, DNS і NTP — фільтр `mqtt` їх ховає.

У звіт. Знімок вікна Wireshark з розгорнутим повідомленням і висновок: кожен, хто має доступ до мережі між вузлом і брокером (той самий Wi-Fi, провайдер, будь-який проміжний вузол мережі), бачить дані, навіть не підписуючись на топік.

Крок 3. Власний брокер з TLS і паролем

Тепер зробимо як слід: власний брокер **Mosquitto**, який пускає лише за паролем і шифрує з'єднання протоколом **TLS**. Безкоштовний Wokwi не бачить вашого комп'ютера, тому в цьому кроці вузол замінює програма `node/node.py`: вона надсилає ті самі повідомлення, що й прошивка, — ті самі топіки, той самий JSON, той самий Last Will. Оцінку це не знижує.

Знадобляться **три вікна терміналу** (у VS Code — три вкладки терміналу, кнопка **+** у панелі терміналу):

Вікно	Що в ньому працює
A	брокер Mosquitto
B	вузол `node/node.py`
C	«зловмисник», а потім законний підписник

Кожне нове вікно спершу переведіть у папку проєкту й активуйте віртуальне середовище (як у кроці 0.5). На Windows додайте ще один рядок — про нього нижче, у 3.1.

3.1. Встановлення Mosquitto

Windows

1. Відкрийте https://mosquitto.org/download/ і завантажте інсталятор `mosquitto-2.1.2-install-windows-x64.exe` (або новішу версію 2.x).
2. Запустіть його. На екрані **Choose Components зніміть позначку «Service»**. Інакше інсталятор запустить «системний» брокер, який займе порт 1883, і ваш власний брокер не запуститься.
3. Mosquitto встановлюється у `C:\Program Files\Mosquitto`, і PowerShell про цю папку не знає. **У кожному новому вікні**, де працюватимете з Mosquitto (A і C), спершу виконайте:

```
$env:Path += ";C:\Program Files\Mosquitto"
```

macOS

Mosquitto для macOS встановлюється через **Homebrew** — безкоштовний «магазин» програм для терміналу.

1. Перевірте, чи є Homebrew: `brew --version`. Якщо бачите `command not found`, відкрийте https://brew.sh, скопіюйте команду з розділу **Install Homebrew**, вставте її в Термінал і натисніть Enter. Він попросить пароль від Mac (символи під час введення не показуються — так і має бути) і запропонує встановити Command Line Tools — погодьтеся. Триває 5–15 хвилин. Наприкінці Homebrew надрукує розділ **Next steps** з командами — виконайте їх по черзі.
2. Встановіть Mosquitto:

```
brew install mosquitto
```

3. **Не** виконуйте `brew services start mosquitto`: так запуститься «системний» брокер, який займе порт 1883.

Якщо згодом на команду `mosquitto` Термінал відповість `command not found` (буває на Mac з процесором Intel), виконайте один раз і відкрийте Термінал заново:

```
echo 'export PATH="/usr/local/sbin:$PATH"' >> ~/.zprofile
```

Linux (Ubuntu, Debian, Mint)

```
sudo apt install mosquitto mosquitto-clients
sudo systemctl stop mosquitto
sudo systemctl disable mosquitto
```

Пакет сам запускає «системний» брокер на порту 1883; дві останні команди зупиняють його і забороняють запускатися разом із системою. Свій брокер ви запускатимете як звичайний користувач, **без** `sudo`.

Перевірка (усі системи).

```
mosquitto -h
```

Серед рядків має бути `mosquitto version 2.1.2` (на Linux — `2.0.x`; для проєкту годиться будь-яка версія 2.x).

3.2. Сертифікати TLS

TLS потребує сертифікатів: брокер показує свій сертифікат, а клієнт перевіряє, що його підписав відомий клієнтові центр сертифікації (ЦС). Ми створимо власний маленький ЦС — одна команда (вікно A, папка проєкту, віртуальне середовище активне):

```
python broker/make_certs.py
```

```
Створено у broker/certs:
  ca.crt      сертифікат ЦС — його дають клієнтам (--cafile)
  ca.key      закритий ключ ЦС — секрет
  server.crt  сертифікат брокера для 127.0.0.1, ::1, localhost; дійсний до 2027-10-08
  server.key  закритий ключ брокера — секрет

Далі, з кореня репозиторію:
  1) користувач node і його пароль:  mosquitto_passwd -c broker/passwd node
  2) запуск брокера:                 mosquitto -c broker/mosquitto.conf -v
```

На Windows першим буде ще рядок `0)` — той самий `$env:Path`, що й у 3.1.

`ca.crt` отримують клієнти (вузол і підписник): за ним вони переконуються, що говорять саме з вашим брокером. `ca.key` і `server.key` — секрети. Файл `broker/.gitignore` не дає їм потрапити у ваш репозиторій; не обходьте його.

3.3. Користувачі і паролі

Створіть двох користувачів брокера: `node` — для вузла, `reader` — для законного підписника:

```
mosquitto_passwd -c broker/passwd node
mosquitto_passwd broker/passwd reader
```

Кожна команда двічі попросить пароль (символи не показуються). Придумайте **два різні** паролі й запишіть їх.

Увага. Не беріть пароль від жодного свого справжнього акаунта: у досліді «до» ви побачите його у Wireshark відкритим текстом, а знімок піде у звіт.

Увага. Ключ `-c` («create») — **лише для першого користувача**: він створює файл заново. Mosquitto 2.1 (Windows і macOS) у разі повторного `-c` відмовиться (`Error: Unable to open file broker/passwd for writing. File exists.`) — просто повторіть команду без `-c`. Mosquitto 2.0 (Linux) мовчки створить файл заново, і попередні користувачі зникнуть.

3.4. Запуск брокера (вікно A)

```
mosquitto -c broker/mosquitto.conf -v
```

Справжній вивід (перший рядок — з ім'ям вашого користувача — і кілька однотипних пропущено):

```
17:37:33: mosquitto version 2.1.2 starting
17:37:33: Config loaded from broker/mosquitto.conf.
…
17:37:33: Opening ipv4 listen socket on port 1883.
17:37:33: Opening ipv4 listen socket on port 8883.
17:37:33: mosquitto version 2.1.2 running
```

Брокер слухає два порти: **1883** — без шифрування (лише для порівняння «до»), **8883** — з TLS. Обидва вимагають пароль і приймають з'єднання лише з вашого комп'ютера. Чому саме так, пояснюють коментарі у `broker/mosquitto.conf` — прочитайте їх.

Залиште вікно A відкритим: тут видно кожне підключення й кожну відмову. Зупинити брокер — Ctrl+C.

Перевірка. Останній рядок — `running`. Якщо бачите `Error: Address already in use` (або подібне на Windows), порт 1883 займає інший брокер — див. Типові проблеми.

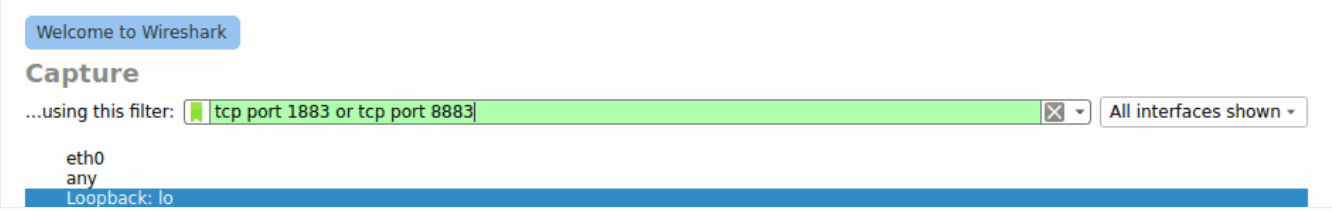
3.5. Запис трафіку у Wireshark

Вузол і брокер працюють на одному комп'ютері, тож їхній трафік іде через особливий «внутрішній» мережевий інтерфейс — **loopback**.

1. Відкрийте Wireshark. На стартовому екрані в полі **…using this filter** введіть фільтр запису:

```
tcp port 1883 or tcp port 8883
```

2. У переліку інтерфейсів двічі клацніть loopback: - Windows — **Adapter for loopback traffic capture**; - macOS — **Loopback: lo0**; - Linux — **Loopback: lo**.



Запис почався. Поки що список порожній — трафіку ще немає.

3.6. «До»: вузол без шифрування (вікно B)

```
python node/node.py --variant 47 --username node --count 3 --interval 2
```

(замість `47` — ваш номер варіанта). Вузол попросить пароль користувача `node`, надішле три цикли вимірювань і завершиться:

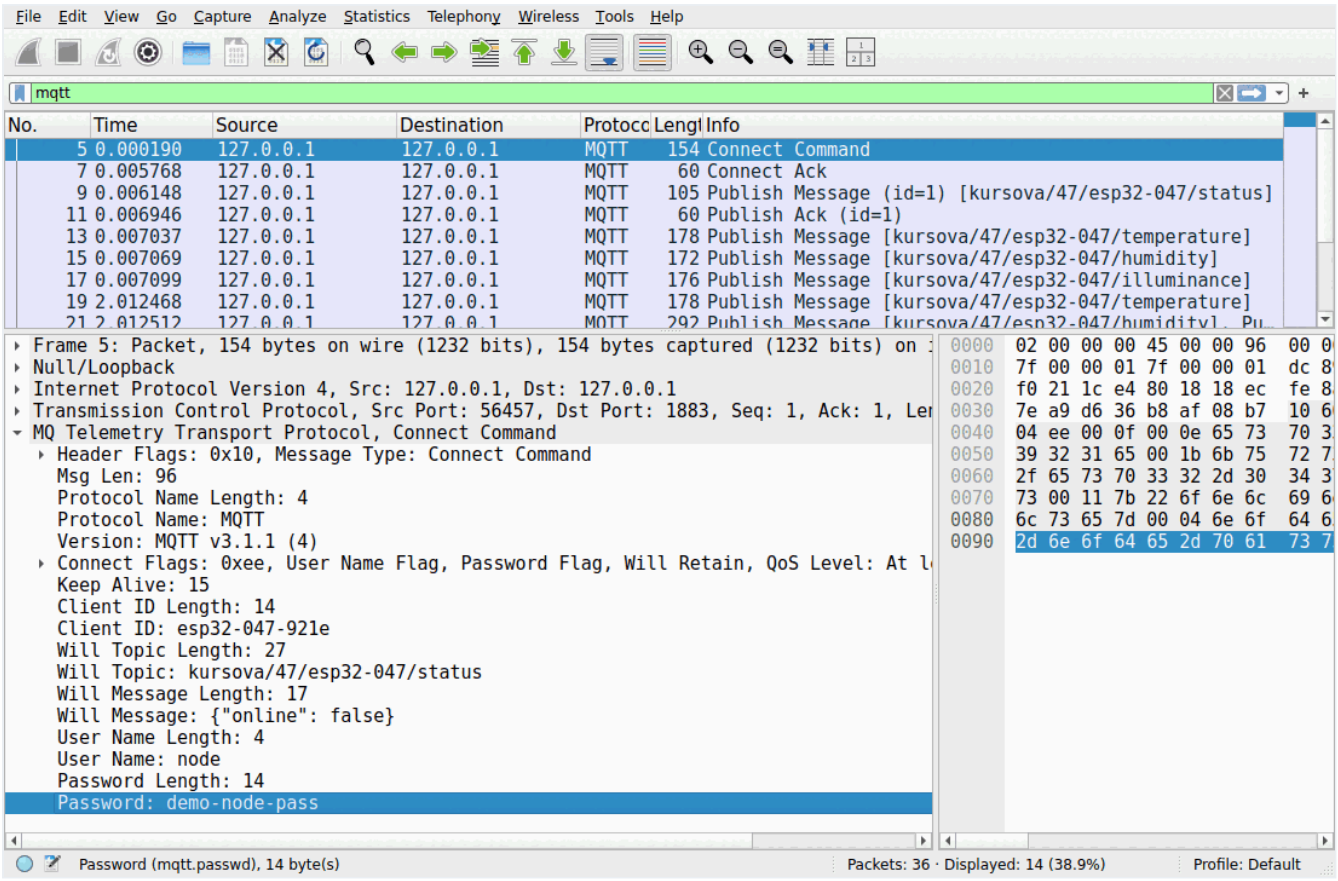
```
Вузол esp32-047, варіант 47 -> 127.0.0.1:1883, TLS: ні, користувач: node
Пароль користувача node:
Підключено до 127.0.0.1:1883. Шифрування: немає.
Last Will зареєстровано: kursova/47/esp32-047/status = {"online": false}
-> kursova/47/esp32-047/status  {"online": true}  (retain)
-> kursova/47/esp32-047/temperature  JSON 86 Б, CBOR 68 Б  {"device_id":"esp32-
047","metric":"temperature","value":21.75,"ts":1791482957,"seq":0}
-> kursova/47/esp32-047/humidity  JSON 83 Б, CBOR 65 Б  {"device_id":"esp32-
047","metric":"humidity","value":46.38,"ts":1791482957,"seq":1}
-> kursova/47/esp32-047/illuminance  JSON 84 Б, CBOR 68 Б  {"device_id":"esp32-
047","metric":"illuminance","value":1.5,"ts":1791482957,"seq":2}
…
Штатне завершення: kursova/47/esp32-047/status = {"online": false}, DISCONNECT надіслано.

Надіслано повідомлень: 9. Середній розмір: JSON 84,3 Б, CBOR 67,0 Б — CBOR коротший на 17,3
Б (20,6 %).
```

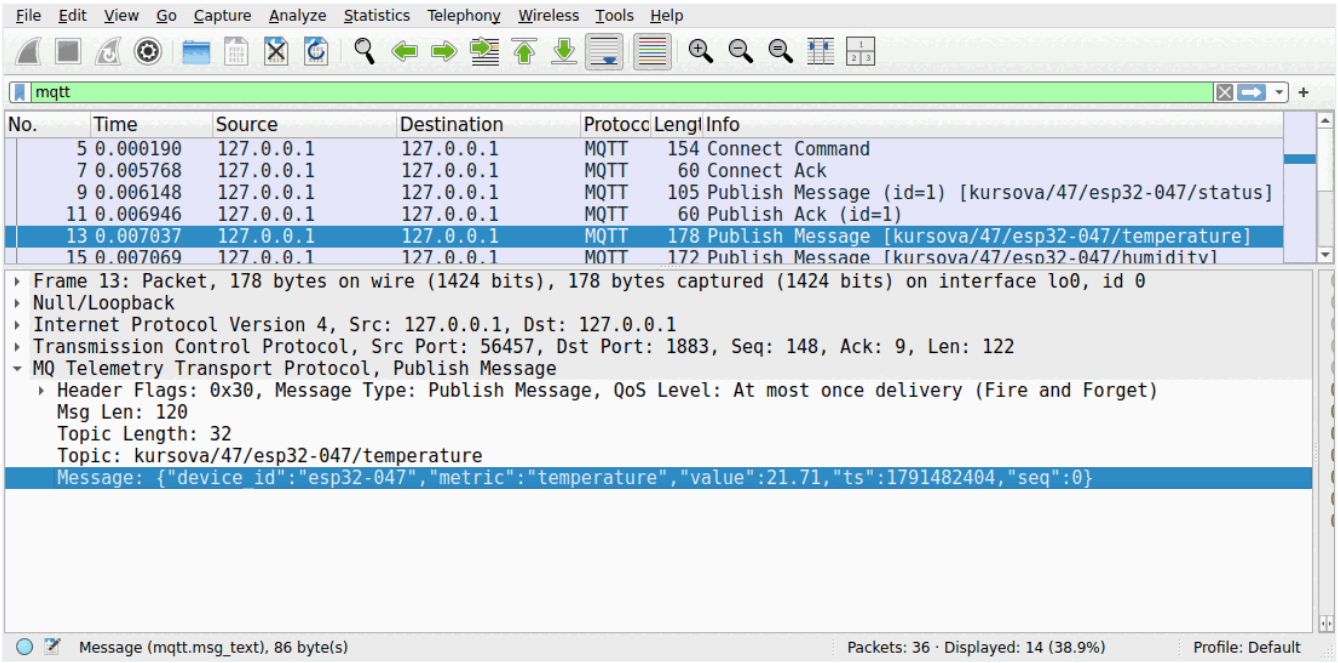
`node.py` наприкінці друкує середню економію CBOR (тут 20,6 %). Вона менша, ніж у прошивці (23–28 %), бо Python записує дробове число у CBOR вісьмома байтами, а ESP32 — чотирма. У звіт наводьте числа з прошивки (TODO 4), а різницю можна пояснити саме цим.

У вікні A з'являться рядки на кшталт `New client connected from 127.0.0.1:… as esp32-047-… (p4, c1, k15, u'node')`.

Тепер у Wireshark введіть фільтр відображення `mqtt` і клацніть рядок **Connect Command** — перше повідомлення вузла. Розгорніть **MQ Telemetry Transport Protocol**:



Ім'я користувача **і пароль** видно відкритим текстом — так само, як і заповіт Last Will. Клацніть далі будь-який **Publish Message**:



Пароль захищає брокер від чужих **підключень**, але нічого не приховує в **мережі**: хто записує трафік, той отримує і дані, і сам пароль.

3.7. «Після»: вузол з TLS (вікно B)

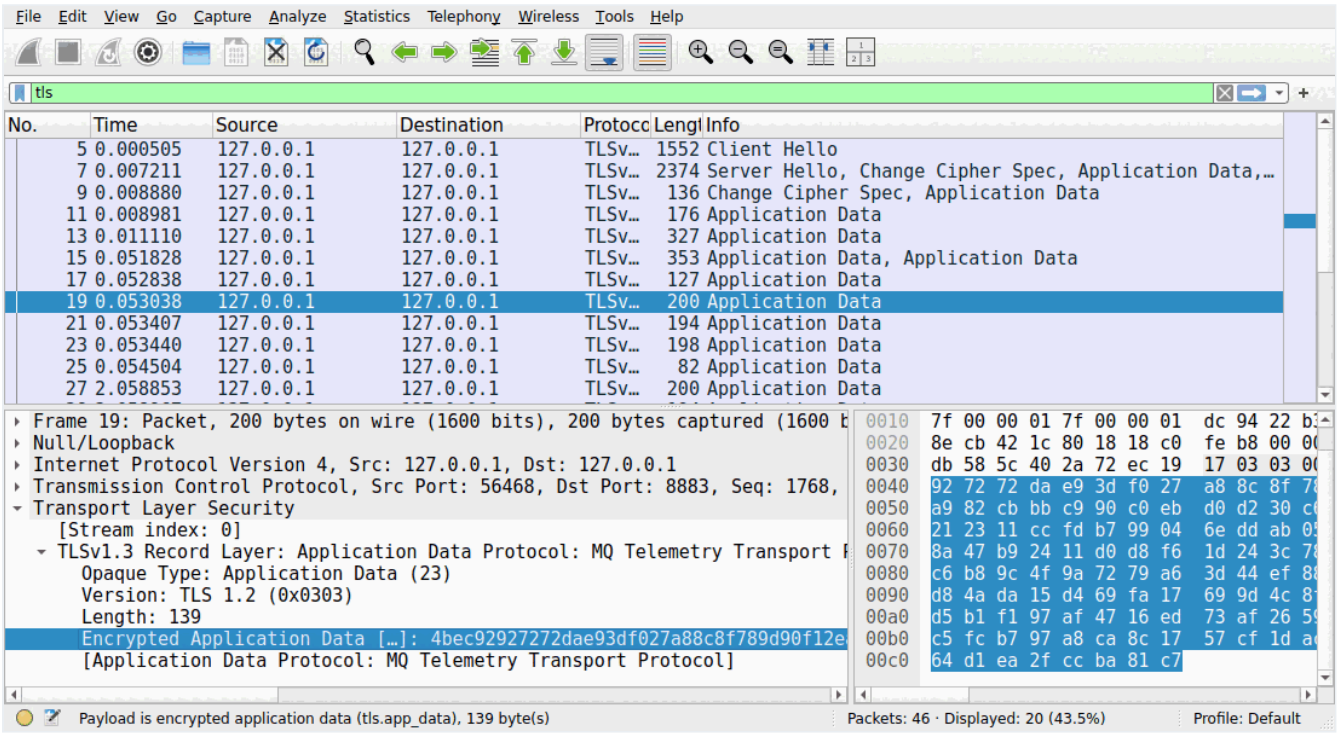
Той самий вузол, але через порт 8883 і з перевіркою сертифіката брокера:

```
python node/node.py --variant 47 --username node --cafile broker/certs/ca.crt --count 3 --
interval 2
```

```
Вузол esp32-047, варіант 47 -> 127.0.0.1:8883, TLS: так, користувач: node
Пароль користувача node:
Підключено до 127.0.0.1:8883. Шифрування: TLSv1.3, шифр TLS_AES_256_GCM_SHA384.
…
```

Порт 8883 вузол вибрав сам: `--cafile` вмикає TLS. У вікні A тепер є ще рядок `negotiated TLSv1.3 cipher TLS_AES_256_GCM_SHA384`.

У Wireshark з фільтром `mqtt` нових рядків **немає**: Wireshark не може навіть розпізнати MQTT. Змініть фільтр на `tls`:



Після «рукостискання» (`Client Hello`, `Server Hello`) іде лише `Application Data` — зашифровані байти. Підпис *Application Data Protocol: MQ Telemetry Transport Protocol* — це лише здогадка Wireshark за номером порту 8883: вмісту він не бачить.

Що TLS **не** приховує: адреси й порти, час кожного повідомлення і його довжину. Порівняйте довжини `Application Data` (194–200 байтів) з розмірами JSON — ритм надсилання вузла видно і крізь шифрування.

Зупиніть запис у Wireshark (червоний квадрат ■) і збережіть його (**File** → **Save As**) — знімки для звіту можна буде зробити пізніше.

3.8. Спроба перехоплення: підписка без пароля (вікно C)

Спробуйте прочитати дані так, як у кроці 1, — підпискою без пароля, спершу на порт без шифрування, потім на TLS-порт:

```
mosquitto_sub -h 127.0.0.1 -p 1883 -t "kursova/#" -v
```

```
mosquitto_sub -h 127.0.0.1 -p 8883 --cafile broker/certs/ca.crt -t "kursova/#" -v
```

Обидві спроби закінчуються однаково:

```
Connection error: Connection Refused: not authorised
```

а у вікні A — `disconnected: not authorised.` Пишіть топік у **подвійних** лапках, як вище: так команда однаково працює на всіх системах.

Тепер законний підписник — `consumer.py` зі стадії 2 від імені `reader`. Залиште його працювати і в сусідньому вікні B ще раз запустіть вузол з TLS (команда з 3.7):

```
python pipeline/consumer.py --variant 47 --cafile broker/certs/ca.crt --username reader --
minutes 1
```

```
Пароль користувача reader:
підключено до 127.0.0.1:8883 (TLS)
підписка на kursova/47/+/#
  [status] esp32-047: ОФЛАЙН (збережений брокером статус)
  [status] esp32-047: онлайн
  [status] esp32-047: ОФЛАЙН

завершено 2026-10-08T18:00:53+00:00
  прийнято записів : 9
  нерозібраних     : 0
  подій status     : 3
  файл             : stream.parquet
```

З `--cafile` адреса `127.0.0.1` і порт 8883 підставляються самі. Дев'ять записів — три цикли по три показники.

3.9. Last Will на власному брокері (за бажанням)

Запустіть у вікні C `consumer.py`, як у 3.8, а у вікні B — вузол з TLS **без** `--count`. Через кілька повідомлень натисніть у вікні B **Ctrl+C**: вузол обірве з'єднання без попередження, і в C з'явиться `[status] esp32-047: ОФЛАЙН` — це брокер опублікував заповіт замість вузла.

У звіт (підрозділ 2.2).

- Таблиця трьох кроків: що робив «зловмисник» → що отримав.

Спроба	Публічний брокер, без TLS	Власний брокер, порт 1883	Власний брокер, порт 8883 (TLS)
Підписка без пароля	…	…	…
Запис трафіку (Wireshark)	…	…	…

- Знімки: Connect з паролем (порт 1883), Publish з JSON, `Application Data` (порт 8883), відмова `not authorised`.

- Висновки: чому пароль без шифрування не захищає; від чого захищає TLS і що залишається видимим (адреси, час, розміри); чим перевірка сертифіката за `ca.crt` захищає від підставного брокера.
- Що лишилося незахищеним у вашій схемі: будь-хто з чинним паролем читає всі топіки (на захисті спитають, як це обмежити — підказка: `acl_file` у документації Mosquitto); порт 1883 без шифрування існує лише для досліду.

1.4. Що внести у звіт (підрозділ 2.2)

- Схема підключення (знімок симулятора Wokwi).
- Формат повідомлення і приклад з монітора.
- Як працює Last Will і знімок із `{"online": false}`.
- Таблиця JSON проти CBOR і розрахунок трафіку за 60 діб.
- Три кроки дослідження захищеності зі знімками і висновок.

Контрольний список стадії 1

- □ Вузол у Wokwi надсилає температуру, вологість і освітленість.
- □ Після зупинки симуляції в топіку `status` з'являється `{"online": false}`.
- □ Ви виміряли розміри JSON і CBOR.
- □ Веб-клієнт у браузері читає ваші повідомлення.
- □ У Wireshark видно JSON у записаному трафіку.
- □ Власний брокер з TLS не віддає дані без пароля, а у Wireshark видно лише зашифрований трафік.

Стадія 2. Конвеєр обробки даних

Мета: завантажити свій набір даних у призначене сховище, виміряти, як воно працює, обробити дані призначеним інструментом і прийняти потік повідомлень від свого вузла. Складник «Конвеєр і вимірювання продуктивності» — 15 балів.

2.1. Що вам призначено

Подивіться у свій маніфест (крок 0.6) на два поля:

Поле	Можливі значення	Що потрібно встановити
`storage`	DuckDB + Parquet, SQLite + Parquet	нічого, вже є
`storage`	InfluxDB	Docker (2.2) і `pip install influxdb-client`
`storage`	TimescaleDB	Docker (2.2) і `pip install "psycopg[binary]"`
`processing`	pandas (порційна), DuckDB SQL	нічого, вже є
`processing`	Polars	`pip install "polars>=1.25"`
`processing`	Dask	`pip install "dask[dataframe]>=2024.8"`

Якщо ваше сховище — DuckDB чи SQLite, а дашборд — не Grafana, розділ 2.2 можна пропустити.

2.2. Docker (для InfluxDB, TimescaleDB і Grafana)

InfluxDB і TimescaleDB — це серверні бази даних. Щоб не встановлювати їх вручну, ми запускаємо їх у Docker: готові «контейнери», які піднімаються однією командою. Docker Desktop безкоштовний для навчання.

Windows 10/11

1. **Віртуалізація.** Відкрийте Диспетчер завдань → **Продуктивність** → **ЦП**. Внизу має бути «Віртуалізація: Увімкнено». Якщо «Вимкнено» — її вмикають у BIOS/UEFI; інструкція Microsoft: https://support.microsoft.com/en-us/windows/enable-virtualization-on-windows-c5578302-6e43-4b4b-a449-8ced115f58e1.
2. **WSL 2.** Відкрийте PowerShell **від імені адміністратора** (правою кнопкою на PowerShell → «Запуск від імені адміністратора») і виконайте:

```
wsl --install
```

Перезавантажте комп'ютер. 3. **Docker Desktop.** Завантажте інсталятор з https://docs.docker.com/desktop/setup/install/windows-install/. Під час встановлення залиште позначку **Use WSL 2 instead of Hyper-V**. 4. Запустіть Docker Desktop і дочекайтеся внизу ліворуч статусу «Engine running».

Увага. Docker Desktop працює на Windows 10/11 через WSL 2, зокрема й на Windows Home; документація Docker вимагає 8 ГБ оперативної пам'яті. Якщо у вас він не запускається — одразу

повідомте керівника, не витрачайте на це вечір.

macOS

Завантажте `Docker.dmg` з https://docs.docker.com/desktop/setup/install/mac-install/ (окремо для Apple Silicon і Intel), перетягніть у «Програми», запустіть і прийміть угоду. Входити в акаунт не обов'язково.

Linux (Ubuntu)

Найлегший варіант — Docker Engine: https://docs.docker.com/engine/install/ubuntu/. Після встановлення виконайте кроки з https://docs.docker.com/engine/install/linux-postinstall/, щоб запускати Docker без `sudo`, і вийдіть із системи та зайдіть знову.

Запуск баз даних

Docker Desktop має бути запущений. З папки проєкту:

```
docker compose -f pipeline/docker-compose.yml up -d
```

Перший запуск завантажує образи (близько 2 ГБ) — це займе кілька хвилин. Перевірка:

```
docker compose -f pipeline/docker-compose.yml ps
```

```
NAME                  IMAGE                                  COMMAND                  SERVICE
CREATED          STATUS                     PORTS
kursova-influxdb      influxdb:2.7                           "/entrypoint.sh infl…"   influxdb
32 minutes ago   Up 32 minutes (healthy)   127.0.0.1:8086->8086/tcp
kursova-timescaledb   timescale/timescaledb:latest-pg16   "docker-entrypoint.s…"
timescaledb   32 minutes ago   Up 21 seconds (healthy)   127.0.0.1:5432->5432/tcp
```

Перевірка. У колонці STATUS — `(healthy)`. Для InfluxDB можна ще відкрити в браузері http://localhost:8086 (логін `student`, пароль `kursova2025`).

Зупинити, коли не працюєте (дані зберігаються):

```
docker compose -f pipeline/docker-compose.yml stop
```

Увага. Ніколи не додавайте `-v` до `docker compose down` — це видалить усі завантажені дані.

2.3. Завантаження і вимірювання

Усі обов'язкові вимірювання стадії (п. 4.2 вказівок) робить один скрипт `pipeline/measure.py`. Він завантажує файл у ваше сховище і вимірює: розмір у CSV і Parquet, швидкість завантаження, розмір сховища, час повного сканування, час вибірки за пристроєм, пам'ять і межу, за якою дані перестануть уміщатися в пам'ять.

Спочатку — на навчальному наборі

Вкажіть **своє** сховище в `--storage` (як у маніфесті, в лапках):

```
python pipeline/measure.py --storage "DuckDB + Parquet" --data tuning.parquet --scale-test
--out measure_tuning.csv
```

Справжній вивід (комп'ютер Apple M1, 16 ГБ):

```
### Вимірювання стадії 2: DuckDB + Parquet, tuning.parquet

Тестова машина: Apple M1 (8 ядер / 8 потоків), RAM 16,0 ГБ, macOS 26.7.1 (arm64), Python
3.12.14.

| Показник | Значення |
|---|---|
| Записів у наборі | 1 157 147 |
| Обсяг у CSV | 44,7 МБ |
| Обсяг у Parquet | 5,3 МБ |
| Коефіцієнт стиснення (CSV / Parquet) | 8,5 |
| Пропускна здатність завантаження | 5 851 988 записів/с (1 157 147 за 0,20 с) |
| Розмір сховища на диску | 8,4 МБ (файл бази duckdb) |
| Повне сканування (кількість і середнє за кожним показником) | 0,004 с |
| Вибірка за одним пристроєм (co2-01-wor, записів: 9 692) | 0,003 с |
| Пікова пам'ять процесу Python (завантаження і запити) | 293,4 МБ (до завантаження 186,1
МБ) |
| Весь файл у pandas: пам'ять на один запис | 14,0 Б у DataFrame, до 66,8 Б під час читання
|
| Межа: вміститься у 16,0 ГБ RAM | ≈ 257 млн записів ≈ 222× цей набір |
```

Таблиця не лише друкується, а й зберігається у файл `measure_tuning.md` (і `measure_tuning.csv`, бо ми вказали `--out`) — його можна вставити у звіт. Без `--out` файл називається за сховищем (`measure_duckdb.md`), і наступний запуск з тим самим сховищем його перезапише.

Потім — на своєму варіанті

```
python pipeline/measure.py --data variant_NNN.parquet --scale-test
```

Без `--storage` скрипт сам бере сховище з вашого маніфесту. На наборі з 7 млн записів вимірювання триває від 15 секунд (DuckDB) до хвилини (InfluxDB).

InfluxDB. Одразу після завантаження InfluxDB тримає дані в робочому журналі, і розмір на диску завищений. Приблизно через 10 хвилин переміряйте лише розмір:

```
python pipeline/measure.py --storage InfluxDB --size-only
```

Порада. Закрийте важкі програми (браузер з десятками вкладок, ігри) і повторіть вимірювання двічі: числа можуть відрізнятися на 30–50 %.

Що означає кожен рядок

Рядок	Як розуміти
Обсяг у CSV / Parquet	Ті самі дані у двох форматах. CSV — текст, Parquet — стовпцевий двійковий формат зі стисненням
Коефіцієнт стиснення	У скільки разів Parquet менший. Поясніть у звіті, звідки така різниця
Пропускна здатність	Скільки записів за секунду сховище приймає
Розмір на диску	Скільки місця займають дані всередині сховища (разом з індексами)
Повне сканування	Час запиту, який мусить прочитати **всі** записи
Вибірка за пристроєм	Час, щоб дістати дані одного пристрою, — те, що постійно роблять детектори
Пікова пам'ять	Найбільше пам'яті, яку займав скрипт
Межа пам'яті	Скільки записів поміститься в пам'ять вашого комп'ютера, якщо читати файл цілком

Межа пам'яті

Вказівки вимагають оцінити, за якого обсягу дані вже не вмістяться в оперативну пам'ять. Ваш варіант ще вміщується (кілька сотень мегабайтів), але реальні системи накопичують у сотні разів більше. Параметр `--scale-test` читає файл, складений з 1, 2 і 4 копій ваших даних, і показує, що пам'ять росте **лінійно**:

```
| Копій | Записів | Файл Parquet | DataFrame у пам'яті | Пік під час читання | Пік на запис
|
|---|---|---|---|---|---|
| 1× | 1 157 147 | 5,3 МБ | 16,2 МБ | 77,3 МБ | 66,8 Б |
| 2× | 2 314 294 | 10,8 МБ | 32,4 МБ | 138,7 МБ | 59,9 Б |
| 4× | 4 628 588 | 21,6 МБ | 64,8 МБ | 265,3 МБ | 57,3 Б |

Пам'ять росте лінійно: ≈ 14,0 МБ + 54,2 Б на кожен запис. За цією прямою 16,0 ГБ RAM
закінчаться на ≈ 316 млн записів.
```

Межа тут (≈ 316 млн) більша, ніж у першій таблиці (≈ 257 млн): там пам'ять на запис рахується з одного читання, де стала частина (≈ 14 МБ) припадає на порівняно мало записів; пряма за трьома розмірами відокремлює цю сталу частину. У звіт наводьте межу за прямою і поясніть різницю.

У звіт (підрозділ 2.3). Ця таблиця, пряма «стільки-то байтів на запис», межа для вашого комп'ютера і як ви її обходите: читання порціями (2.4), стовпцеве зберігання, фільтрація на рівні сховища (запит повертає лише потрібний пристрій).

Для порівняння: усі чотири сховища

Ось що показали ті самі вимірювання на файлі з 6,9 млн записів (розмір типового варіанта) на тестовому комп'ютері. Ваші числа будуть іншими, але співвідношення — схожими.

Сховище	Завантаження, записів/с	На диску	Повне сканування	Один пристрій
DuckDB	9,2 млн	51 МБ	0,014 с	0,015 с
SQLite	0,46 млн	542 МБ	4,27 с	0,123 с
InfluxDB	0,15 млн	38 МБ	0,62 с	0,215 с
TimescaleDB	0,21 млн	1006 МБ	0,62 с	0,093 с

Подумайте для звіту, чому стовпцевий DuckDB і на порядки швидший, і менший за рядковий SQLite, і чому TimescaleDB займає найбільше місця.

2.4. Обробка призначеним інструментом

Для кожного з чотирьох інструментів є приклад у папці `processing/`. Усі чотири приклади рахують однакові дві таблиці, тож результат можна порівнювати:

- `results/daily_stats.parquet` — для кожного пристрою, показника і доби: кількість, середнє, мінімум, максимум;
- `results/hourly_messages.parquet` — скільки повідомлень надіслав кожен пристрій за кожну годину. Ця таблиця дуже знадобиться на стадії 5.

Запустіть **свій**:

```
python processing/pandas_chunked.py --data variant_NNN.parquet
python processing/polars_example.py --data variant_NNN.parquet
python processing/duckdb_sql.py     --data variant_NNN.parquet
python processing/dask_example.py   --data variant_NNN.parquet
```

Приклад виводу (pandas, навчальний набір):

```
pandas 3.0.6, порції по 500 000 рядків; файл tuning.parquet: 1 157 147 записів
Обчислено за 1.70 с (679 018 записів/с); пікова пам'ять процесу 247 МБ

daily_stats: 969 рядків -> results/daily_stats.parquet
 device_id metric        day  count       mean   min   max
co2-01-wor    co2 2025-09-01    716 536.660615 386.0 743.0
co2-01-wor    co2 2025-09-02    716 532.047486 372.0 744.0
co2-01-wor    co2 2025-09-03    715 532.082517 384.0 720.0
co2-01-wor    co2 2025-09-04    715 537.800000 393.0 754.0
co2-01-wor    co2 2025-09-05    716 533.384078 364.0 738.0

hourly_messages: 17149 рядків -> results/hourly_messages.parquet
 device_id                hour  messages
co2-01-wor 2025-09-01 00:00:00        31
co2-01-wor 2025-09-01 01:00:00        30
…
```

Відкрийте свій скрипт і прочитайте коментарі: там пояснено, чому інструмент обробляє файл частинами і не тримає його в пам'яті цілком.

Увага. Усі часи в проєкті — в UTC. Не переводьте їх у київський час, інакше доби і години «з'їдуть» на 2–3 години.

У звіт (підрозділи 2.1 і 3.1). Чому для вашого обсягу цей інструмент підходить; час обробки і пікова пам'ять — обидва числа скрипт друкує в рядку «Обчислено за…».

2.5. Потоковий вхід

Другий вхід конвеєра — повідомлення вашого вузла з Wokwi у реальному часі. Запустіть симуляцію (стадія 1), а в терміналі:

```
python pipeline/consumer.py --variant 47 --device esp32-047 --minutes 10
```

(номер варіанта і `DEVICE_ID` — як у вашій прошивці). Справжній вивід хвилинного запуску для демонстраційного вузла (`--variant 999 --device esp32-999 --minutes 1`):

```
підключено до broker.hivemq.com:1883
підписка на kursova/999/esp32-999/#
  [status] esp32-999: ОФЛАЙН (збережений брокером статус)
  [status] esp32-999: онлайн

завершено 2026-10-08T17:57:13+00:00
  прийнято записів : 33
  нерозібраних     : 0
  подій status     : 2
  файл             : stream.parquet
```

Повідомлення записуються у `stream.parquet` у тому самому форматі, що й ваш варіант (плюс номер `seq` і час отримання `received_at`). Перший рядок `ОФЛАЙН` — це збережене брокером повідомлення з попереднього запуску вузла (так працює прапорець `retain`); другий — свіжий статус після старту симуляції.

Увага. Не завантажуйте `stream.parquet` у те саме сховище, що й ваш варіант: `load_parquet` щоразу **замінює** дані. Для потоку відкрийте окрему базу, наприклад `open_storage("duckdb", database="stream.duckdb")`.

У звіт. Скільки повідомлень прийнято, чи були пропуски номерів `seq` (втрачені повідомлення) і яка затримка між `ts` і `received_at`. Усе це порахує одна команда:

```
python -c "import pandas as pd; d = pd.read_parquet('stream.parquet').sort_values('seq');
lag = d['received_at'] - d['ts']; print('повідомлень:', len(d)); print('пропущено номерів
seq:', int(d['seq'].max() - d['seq'].min() + 1 - d['seq'].nunique())); print('затримка, с:
медіана', lag.median(), ', найбільша', lag.max())"
```

Якщо у файлі кілька пристроїв або ви перезапускали вузол (тоді `seq` починається з нуля знову), рахуйте пропуски для кожного запуску окремо.

Контрольний список стадії 2

- □ Ваше сховище працює (для InfluxDB і TimescaleDB — контейнери `(healthy)`).
- □ `measure.py` пройшов на навчальному наборі і на вашому варіанті, таблиці збережено.
- □ Ви запустили `--scale-test` і знаєте межу пам'яті свого комп'ютера.

- □ Ваш інструмент обробки створив `results/daily_stats.parquet` і `results/hourly_messages.parquet`.
- □ `consumer.py` прийняв повідомлення вашого вузла.

Стадія 3. Детектори

Мета: налаштувати два детектори аномалій — **базовий** статистичний і **призначений** вам у маніфесті — на навчальному наборі, де відповіді відомі, і виміряти їхню точність і повноту. Окремих балів стадія не має: результати йдуть у підрозділи 2.4 і 3.2 записки, а самі детектори — ваші інструменти на стадіях 4 і 5.

3.1. Як влаштовані скрипти детекторів

У заготовці є шість готових детекторів:

Скрипт	Метод	Додаткова бібліотека
`detect/baseline.py`	базовий: рухома робастна z-оцінка	—
`detect/methods/ewma.py`	EWMA control chart	—
`detect/methods/stl_esd.py`	STL + ESD	`statsmodels`
`detect/methods/matrix_profile.py`	Matrix Profile	`stumpy`
`detect/methods/isolation_forest.py`	Isolation Forest	`scikit-learn`
`detect/methods/lof.py`	Local Outlier Factor	`scikit-learn`

Базовий запускають усі. З п'яти інших потрібен лише той, що записаний у вашому маніфесті (`assigned_stack` → `detector`, див. крок 0).

Усі скрипти працюють однаково:

- аналізують **один ряд** — один показник одного пристрою, який обираєте **ви** (`--device`, `--metric`). Вирішувати, які ряди перевіряти, — ваша частина роботи;
- друкують таблицю **вікон-кандидатів** — відрізків часу, де детектор щось помітив;
- зберігають **графік** (PNG) — на ньому видно, що саме помічено;
- з ключем `--evaluate tuning_truth.json` порівнюють вікна з відповідями навчального набору за тими самими правилами, що й автоматична перевірка ваших знахідок (розділ 5.2).

Спільні параметри:

Параметр	Типово	Що означає
`--data`	`tuning.parquet`	файл з даними
`--device`	—	пристрій; без нього скрипт надрукує перелік пристроїв і їхніх показників
`--metric`	—	показник; якщо помилитеся в назві, скрипт підкаже правильні
`--signal`	`value`	що аналізувати: `value` — самі значення; `rate` — швидкість зростання лічильника (для `pkt_sent`, `pkt_lost`); `interval` — секунди між повідомленнями
`--step`	`auto`	крок рівномірної сітки часу (не стосується базового детектора)
`--gap`	`1h`	позначки, ближчі за цей проміжок, зливаються в одне вікно
`--min-run`	`3`	вікна з меншою кількістю позначок відкидаються як шум
`--png`	назва за методом і рядом	куди зберегти графік
`--evaluate`	—	файл відповідей, лише `tuning_truth.json`
`--json`, `--class`	—	записати вікна як кандидатів у знахідки (див. 3.5)

Повний перелік з поясненнями: `python detect/baseline.py --help` (так само для кожного скрипта).

Увага. Перший запуск після встановлення бібліотек повільний — до 30–40 секунд: Python будує кеш шрифтів і компілює бібліотеки. Наступні запуски тривають 1–4 секунди (Matrix Profile — 15–20 секунд щоразу, див. 3.3).

Як відкрити графік. Скрипт друкує шлях до PNG у рядку `Графік:`. Відкрийте файл у VS Code (панель файлів ліворуч) або командою:

- Windows: `Invoke-Item baseline_power-05-mee_power_w.png`
- macOS: `open baseline_power-05-mee_power_w.png`
- Linux: `xdg-open baseline_power-05-mee_power_w.png`

3.2. Базовий детектор: рухома робастна z-оцінка

Ідея. Кожне значення порівнюємо з медіаною значень за попередню добу (`--window 24h`) і ділимо відхилення на MAD — медіанне абсолютне відхилення того самого вікна:

```
z = (x − медіана вікна) / (1,4826 · MAD вікна)
```

Точку позначаємо, якщо `|z| > 10` (`--threshold`). Медіана й MAD, на відміну від середнього і стандартного відхилення, майже не зважають на викиди: один шалений сплеск не «розмиває» норму для решти точок. Поріг 10 замість класичних 3 — бо реальні дані IoT мають «важкі хвости», і з порогом 3 тривоги лунали б щогодини. Обґрунтування — у коментарях на початку `detect/baseline.py`; прочитайте їх.

Запуск на пристрої, де навчальний набір містить сплески потужності:

```
python detect/baseline.py --device power-05-mee --metric power_w --evaluate
tuning_truth.json
```

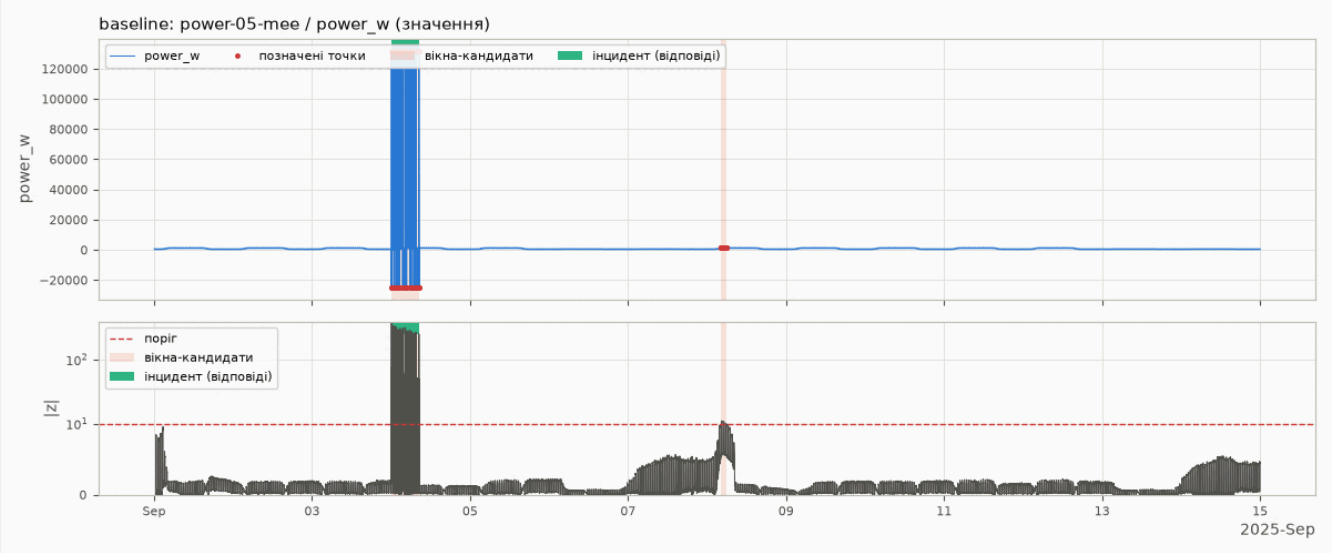
Справжній вивід:

```
power-05-mee / power_w: 39871 повідомлень, 2025-09-01 — 2025-09-14; сигнал: значення; 39871
точок без сітки

Позначено точок: 147 з 39811. Вікон-кандидатів: 2
   №  початок (UTC)     кінець (UTC)      тривалість  точок  пік оцінки
   1  2025-09-04 00:03  2025-09-04 08:31     8.5 год     73         377
   2  2025-09-08 04:18  2025-09-08 05:48     1.5 год     74        11.1

Графік: …/baseline_power-05-mee_power_w.png

Порівняння з відповідями для power-05-mee: інцидентів за участю пристрою — 1
  ЗНАЙДЕНО   вікно 1   = INC-02 spike_storm        IoU 0.98
  ХИБНЕ      вікно 2   2025-09-08 04:18 – 2025-09-08 05:48  (найбільше перекриття з
інцидентом 0.00)
  Точність 0.50 (1 з 2 вікон справжні)   Повнота 1.00 (1 з 1 інцидентів знайдено)
  Нагадування: у findings.json кожне хибне вікно знімає 5 % балів (сумарно до 30 %).
```



Як читати:

- **вікно 1** збіглося з інцидентом `INC-02` (сплески потужності) з перекриттям за часом **IoU 0,98** — майже ідеально (зараховується від 0,25);
- **вікно 2** — хибне: інциденту там немає. На нижній панелі графіка видно, що оцінка ледь перевищила поріг (пік 11,1 при порозі 10);
- **точність** — частка справжніх серед знайдених вікон (1 з 2), **повнота** — частка знайдених серед справжніх інцидентів (1 з 1).

Що ловить базовий детектор: різкі сплески, серії неможливих значень, перші години після різкого стрибка рівня. Чого **не** ловить: повільний дрейф (вікно «пливе» разом із ним) і застигле значення (відхилень немає взагалі). На двійкових сигналах (двері відчинені/зачинені) він марний: там будь-яка зміна — «аномалія».

Спробуйте самі.

1. Запустіть на «здоровому» пристрої `power-01-mee` — має бути 0 вікон.

2. Лічильники шлюзів (`pkt_sent`, `pkt_lost`) лише зростають, тож їхні значення аналізувати безглуздо — аналізуйте їхню швидкість, `--signal rate` (для `pkt_sent` скрипт підкаже це сам):

```
python detect/baseline.py --device gw-01 --metric pkt_lost --signal rate --evaluate
tuning_truth.json
```

З типовим `--gap 1h` один інцидент розпадеться на три шматки: один зарахується, два стануть хибними. Повторіть з `--gap 3h` і порівняйте. Це важливий урок: **один інцидент, розрізаний на кілька вікон, — це одна правильна знахідка і кілька хибних**. 3. Змінюйте `--threshold` (5, 10, 20) і `--window` (6h, 24h, 3D) на кількох пристроях і записуйте, як змінюються точність і повнота.

3.3. Призначений детектор

Встановіть бібліотеку свого методу (у активованому віртуальному середовищі):

Метод	Команда
EWMA control chart	нічого не треба
STL + ESD	`pip install statsmodels`
Matrix Profile	`pip install stumpy`
Isolation Forest, Local Outlier Factor	`pip install scikit-learn`

Далі — розділ про **ваш** метод. Інші прочитайте побіжно: на захисті можуть спитати, чому для певного класу інцидентів ваш метод гірший чи кращий за інший.

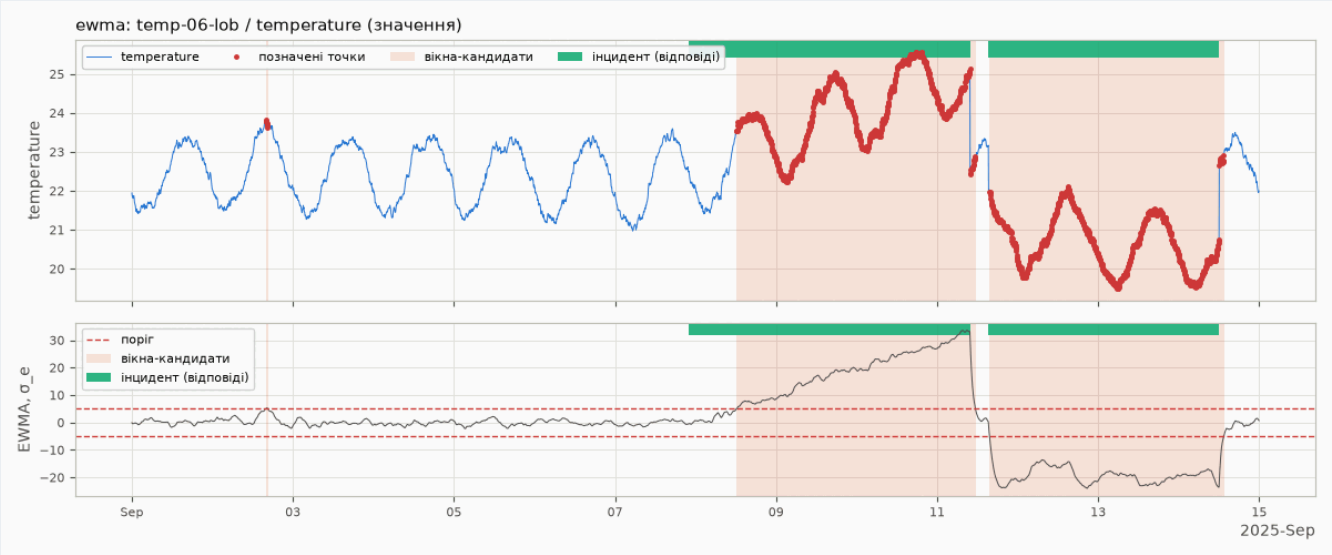
EWMA control chart

Ідея. Окремий відлік шумний, і невеликий, але **сталий** зсув у ньому не видно. EWMA накопичує відхилення від норми, поступово «забуваючи» старі: `e_t = λ·r_t + (1 − λ)·e_(t−1)`, де `r_t` — відхилення від типового значення для цього часу доби. Тривога — коли `e_t` виходить за межі `±L·σ_e`. Типовий профіль доби рахується за перші 7 діб (`--ref-days`), окремо для будніх і вихідних: розпорядок будівлі в них різний.

```
python detect/methods/ewma.py --device temp-06-lob --metric temperature --evaluate
tuning_truth.json
```

```
temp-06-lob / temperature: 19962 повідомлень, 2025-09-01 — 2025-09-14; сигнал: значення;
сітка 5min: 4032 точок
Еталонний період: 2025-09-01 00:00 — 2025-09-08 00:00 UTC

Позначено точок: 1700 з 4032. Вікон-кандидатів: 3
   №  початок (UTC)     кінець (UTC)      тривалість  точок  пік оцінки
   1  2025-09-02 16:00  2025-09-02 16:35     0.6 год      7        5.29
   2  2025-09-08 12:10  2025-09-11 11:25    71.2 год    855        33.6
   3  2025-09-11 15:40  2025-09-14 13:30    69.8 год    838         -24
…
  ЗНАЙДЕНО   вікно 2   = INC-04 drift              IoU 0.81
  ЗНАЙДЕНО   вікно 3   = INC-10 sensor_swap        IoU 0.98
  ХИБНЕ      вікно 1   2025-09-02 16:00 – 2025-09-02 16:35  (найбільше перекриття з
інцидентом 0.00)
  Точність 0.67 (2 з 3 вікон справжні)   Повнота 1.00 (2 з 2 інцидентів знайдено)
```



Під час дрейфу EWMA повільно повзе вгору за межу +5, а після обміну давачів різко падає нижче –5: знак піку оцінки показує напрямок зсуву.

Що ловить: повільний дрейф, тривалий зсув рівня (зокрема швидкості лічильника, `--signal rate`), незвичне споживання. Чого не ловить: поодинокі короткі сплески — згладжування їх «розмазує».

Параметри: `--lam` (0,1; менше — довша пам'ять), `--L` (5; ширина меж), `--ref-days` (7; перші дні мають виглядати нормальними — перевірте це на графіку!), `--profile workdays|day|none`. Спробуйте `--profile day` на `temp-09-lob` — і побачите, навіщо розділяти будні й вихідні.

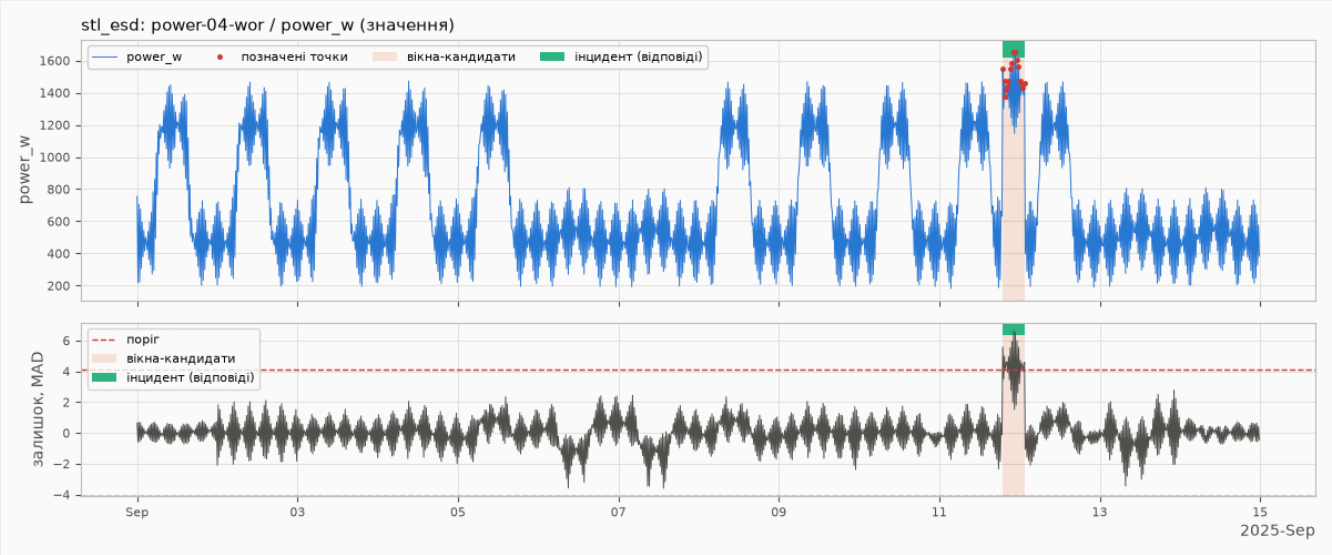
STL + ESD

Ідея. STL розкладає ряд на тренд, добову сезонність і залишок; у залишку узагальнений тест ESD (Rosner, 1983) шукає викиди. Типова сітка — 15 хв: на 5-хвилинній сітці шум перемикань компресора ховає аномалію.

```
python detect/methods/stl_esd.py --device power-04-wor --metric power_w --evaluate
tuning_truth.json
```

```
power-04-wor / power_w: 39950 повідомлень, 2025-09-01 – 2025-09-14; сигнал: значення; сітка
15min: 1344 точок
STL: період 1D = 96 кроків; ESD знайшов викидів: 16 (шукав не більше 67)

Позначено точок: 16 з 1344. Вікон-кандидатів: 1
   №  початок (UTC)     кінець (UTC)      тривалість  точок  пік оцінки
   1  2025-09-11 19:00  2025-09-12 01:45     6.8 год     16        6.66
…
  ЗНАЙДЕНО   вікно 1   = INC-11 energy_anomaly     IoU 0.99
```



Що ловить: сплески і незвичне для цього часу доби значення. Чого не ловить: дрейф (STL відносить його до тренду). Слабке місце: період 1 доба не знає про вихідні — на `temp-09-lob` суботу й неділю буде позначено хибно. Рядок «ESD знайшов викидів: 67 (шукав не більше 67)» означає, що спрацювала стеля `--max-share`, — ознака, що метод «стріляє» забагато.

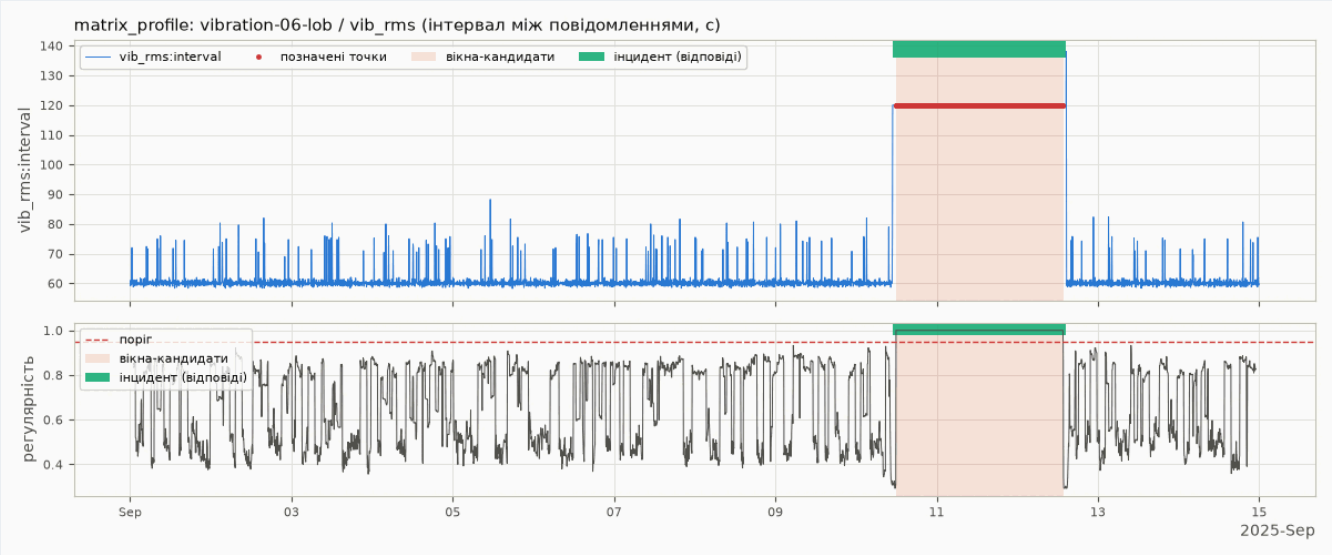
Matrix Profile

Ідея. Для кожного фрагмента ряду довжиною `--window` (2 год) шукаємо найсхожіший фрагмент деінде. Відстань до цього «двійника» — матричний профіль. `--find discords` шукає фрагменти, не схожі ні на що (сплеск, обрив); `--find repeats` — фрагменти, що **надто точно** повторюються (застигле значення або надто регулярні інтервали між повідомленнями).

```
python detect/methods/matrix_profile.py --device vibration-06-lob --metric vib_rms --signal
interval --find repeats --evaluate tuning_truth.json
```

```
vibration-06-lob / vib_rms: 18471 повідомлень, 2025-09-01 — 2025-09-14; сигнал: інтервал
між повідомленнями, с; сітка 5min: 4032 точок
Рахую матричний профіль: фрагменти завдовжки 2h (точок у кожному: 24)...

Позначено точок: 597 з 4009. Вікон-кандидатів: 1
   №  початок (UTC)     кінець (UTC)      тривалість  точок  пік оцінки
   1  2025-09-10 11:55  2025-09-12 13:40    49.8 год    597           1
…
  ЗНАЙДЕНО   вікно 1   = INC-08 beaconing          IoU 0.96
  Точність 1.00 (1 з 1 вікон справжні)   Повнота 1.00 (1 з 1 інцидентів знайдено)
```



Зазвичай інтервал між повідомленнями — близько 60 с з розкидом, а два дні поспіль — рівно 120 с: так поводиться не давач, а програма, що «стукає» назовні за розкладом.

Увага. Кожен запуск триває 15–20 секунд (перший — до 40): бібліотека компілює свій код. Скрипт не завис. На **Mac з процесором Intel** stumpy встановлюється лише на Python 3.12 або 3.13 (крок 0.2).

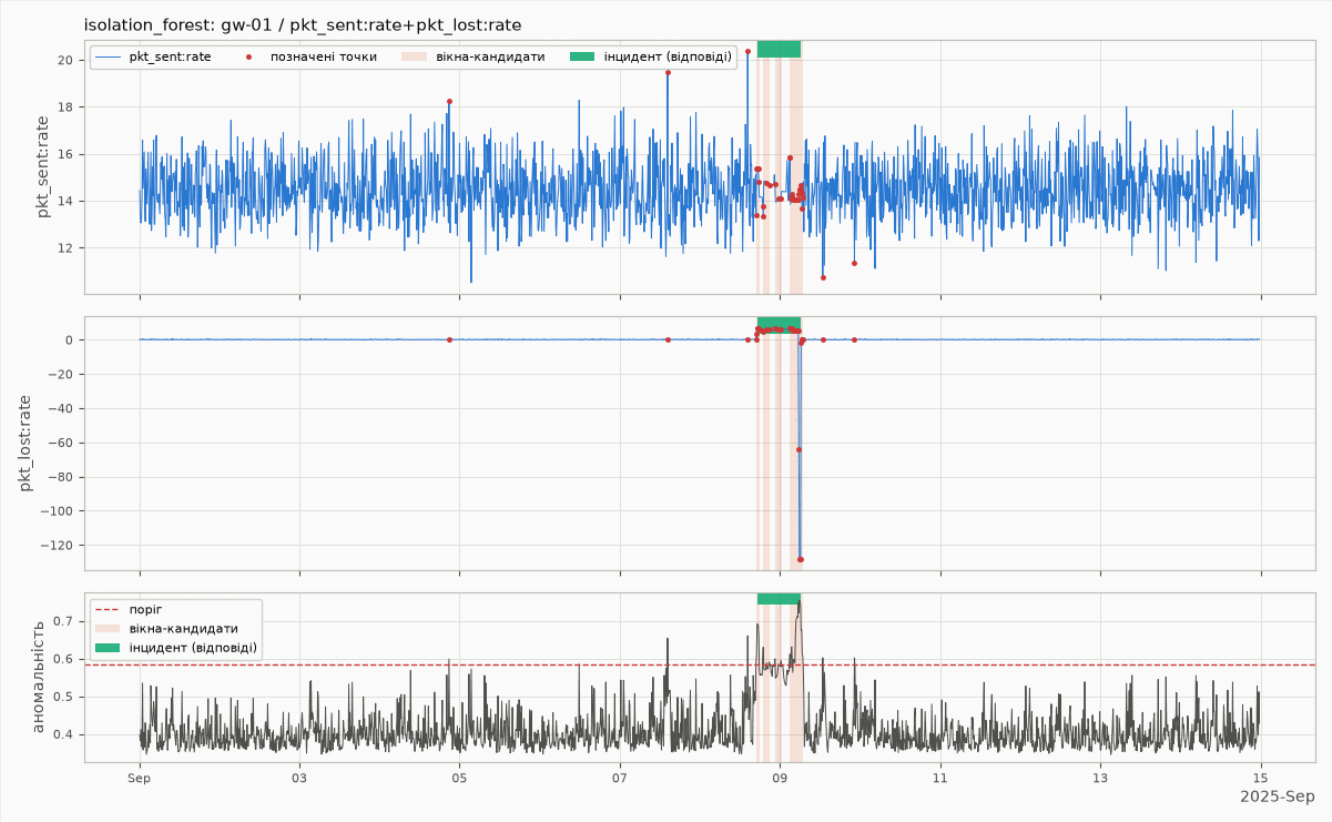
Чого не ловить: повільний дрейф і незвичне споживання. Discords на сигналах «увімкнено/вимкнено» (світло, двері) дають багато хибних вікон: там кожен фрагмент трохи унікальний.

Isolation Forest

Ідея. Багато випадкових дерев ділять точки випадковими розрізами, доки кожна не опиниться окремо. Незвичну точку ізолювати легко — їй вистачає кількох розрізів. Ознаки для кожної точки: значення, відхилення від рухомої медіани і рухомий розкид. Можна аналізувати кілька показників одного пристрою разом (`--metrics a,b`) — тоді аномальними стають і незвичні **поєднання** значень.

```
python detect/methods/isolation_forest.py --device gw-01 --metrics pkt_sent,pkt_lost --
signal rate --contamination 0.05 --evaluate tuning_truth.json
```

```
…
Позначено точок: 101 з 2014. Вікон-кандидатів: 2
   №  початок (UTC)     кінець (UTC)      тривалість  точок  пік оцінки
   1  2025-09-07 13:20  2025-09-07 14:30     1.2 год      3       0.654
   2  2025-09-08 16:00  2025-09-09 07:10    15.2 год     81       0.754
…
  ЗНАЙДЕНО   вікно 2   = INC-06 gateway_group_loss IoU 0.87
  ХИБНЕ      вікно 1   2025-09-07 13:20 – 2025-09-07 14:30  (найбільше перекриття з
інцидентом 0.00)
```



Головна пастка. `--contamination` — частка точок, яку метод **обов'язково** позначить, навіть на здоровому пристрої. З типовим 0,02 13-годинний інцидент вище розпадається на чотири вікна (одне зараховане, три хибні); з 0,05 — ціле вікно. На здоровому шлюзі `gw-03` той самий запуск дасть 11 хибних вікон — дивіться на **величину** оцінки: близько 0,6 на звичайних даних і 0,75–0,85 на справжніх інцидентах.

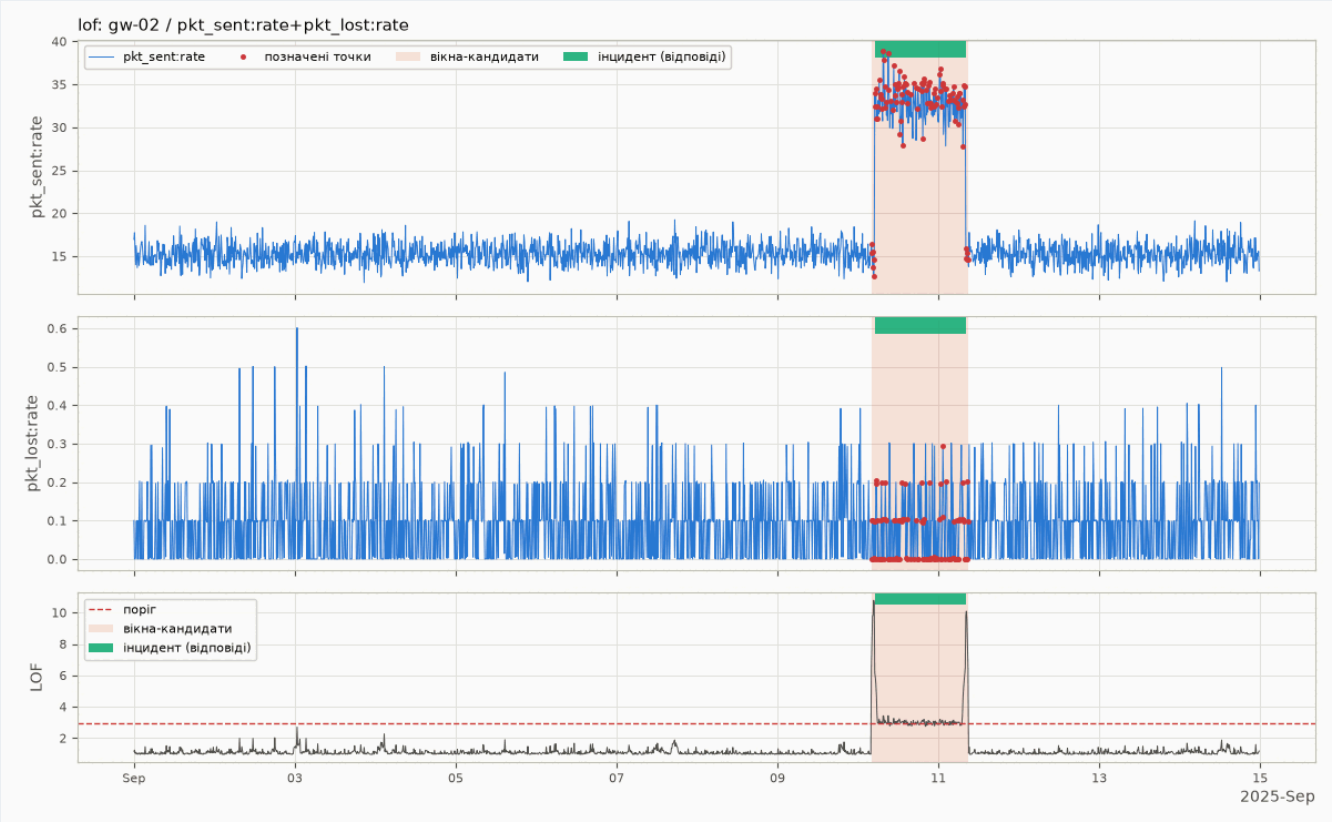
Local Outlier Factor

Ідея. Точка аномальна, якщо щільність її околу набагато менша, ніж у сусідів. Ознаки й режими — як в Isolation Forest.

```
python detect/methods/lof.py --device gw-02 --metrics pkt_sent,pkt_lost --signal rate --
contamination 0.05 --evaluate tuning_truth.json
```

```
gw-02 / pkt_sent:rate+pkt_lost:rate: 4028, 4028 повідомлень; сітка 10min: 2016 точок
Ознак: 6 (pkt_sent:rate, pkt_sent:rate − медіана, pkt_sent:rate розкид, pkt_lost:rate,
pkt_lost:rate − медіана, pkt_lost:rate розкид); рядків без пропусків: 2015

Позначено точок: 101 з 2015. Вікон-кандидатів: 1
   №  початок (UTC)     кінець (UTC)      тривалість  точок  пік оцінки
   1  2025-09-10 04:00  2025-09-11 09:00    29.0 год    101        10.8
…
  ЗНАЙДЕНО   вікно 1   = INC-07 exfil_pattern      IoU 0.94
  Точність 1.00 (1 з 1 вікон справжні)   Повнота 1.00 (1 з 1 інцидентів знайдено)
```



Шлюз надсилає вдвічі більше пакетів, ніж зазвичай, а частка втрат не змінилася — кожен показник окремо виглядає майже правдоподібно, а їхнє поєднання — ні.

Пастка. Якщо `--neighbors` (300) менше за довжину інциденту в точках сітки, довгий інцидент утворює власне «щільне скупчення» і зникає. Спробуйте той самий запуск з `--neighbors 20`: інцидент буде пропущено. Про `--contamination` — те саме, що для Isolation Forest.

3.4. Точність і повнота на всьому навчальному наборі

`--evaluate` рахує точність і повноту для **одного** ряду. Вказівки вимагають навести їх для детектора загалом, тож зведіть кілька запусків у таблицю:

1. Відкрийте `tuning_truth.json` (у VS Code) і випишіть інциденти, які ваш детектор **має** ловити за своєю ідеєю: пристрої, показники, час.
2. Додайте стільки ж «здорових» рядів подібних пристроїв — без них ви не побачите хибних спрацювань.
3. Запустіть детектор на кожному ряді з `--evaluate` і запишіть кількість рядків `ЗНАЙДЕНО`, `ХИБНЕ` і `ПРОПУЩЕНО`.

Ряд	Вікон	Знайдено (TP)	Хибних (FP)	Пропущено (FN)
power-05-mee / power_w	2	1	1	0
power-01-mee / power_w	0	0	0	0
…				
Разом		**ΣTP**	**ΣFP**	**ΣFN**

```
точність = ΣTP / (ΣTP + ΣFP)        повнота = ΣTP / (ΣTP + ΣFN)
```

Зробіть таку таблицю для обох детекторів — з типовими параметрами і після налаштування. «Прийнятні» значення — це ті, для яких ви можете пояснити кожне хибне спрацювання й кожен пропуск (і що зробили, щоб їх стало менше).

Увага. Налаштовуйте параметри на **навчальному** наборі, а не під відповіді конкретного ряду: детектор, що ідеально знаходить один інцидент, але сипле хибними вікнами на всіх інших пристроях, у вашому варіанті нашкодить більше, ніж допоможе.

3.5. Від вікон до знахідок

На стадії 5 вікна-кандидати з вашого варіанта стануть знахідками. Скрипти вміють записати їх у JSON:

```
python detect/methods/ewma.py --device temp-06-lob --metric temperature --json windows.json
--class drift
```

```
Кандидатів записано у windows.json: 3. Це ще НЕ findings.json: перегляньте кожне вікно на
графіку, виправте confidence та evidence і лише тоді переносьте у свій findings.json.
```

Усім вікнам скрипт ставить один клас (той, що ви вказали) і упевненість 0,5. Тут третє вікно насправді — `sensor_swap`, а не `drift`; з `--evaluate` скрипт скаже про це прямо:

```
ЗНАЙДЕНО   вікно 3   = INC-10 sensor_swap        IoU 0.98  (ваш клас drift: зарахується з
коефіцієнтом 0,7)
```

Тому кожне вікно треба переглянути на графіку, виправити клас, упевненість і пояснення — це і є робота стадії 5.

У звіт

- **Підрозділ 2.4.** Обидва детектори: ідея, формула, параметри і чому саме такі значення; на які класи інцидентів розраховано кожен і чому.
- **Підрозділ 3.2.** Таблиці з 3.4 для обох детекторів (до і після налаштування), 2–3 графіки, пояснення хибних спрацювань і пропусків, порівняння детекторів між собою.
- Посилання на джерела методів (статті авторів і документацію бібліотек) — у список джерел.

Контрольний список стадії 3

- □ Базовий детектор запускається, і ви розумієте кожен рядок його виводу.
- □ Бібліотеку призначеного методу встановлено, детектор запускається.
- □ Для обох детекторів є таблиця TP/FP/FN щонайменше на 8 рядах і обчислені точність і повнота.
- □ Ви можете пояснити, які класи інцидентів ваш призначений детектор ловить, а які — ні, і чому.
- □ Збережено графіки для записки.

Стадія 4. Реальні дані

Мета: перевірити детектори на справжніх даних давачів, з'ясувати, які їхні тривоги хибні, і доопрацювати попередню обробку так, щоб хибних стало менше. Окремих балів стадія не має, але її результати — обов'язкова частина підрозділу 3.2 записки, а вміння відрізнити аномалію від особливості даних знадобиться на стадії 5.

4.1. Набір даних Intel Lab

У 2004 році в лабораторії Intel Research у Берклі (Каліфорнія) 54 бездротові вузли Mica2Dot понад місяць, з 28 лютого по 5 квітня, кожні 31 секунду надсилали чотири показники: температуру, вологість, освітленість і напругу батареї. Разом — близько 2,3 млн рядків.

Це **справжні** дані зі справжніми вадами: розрядженими батареями, перервами зв'язку, дублікатами і пристроями, яких немає в переліку. На відміну від вашого варіанта, **еталонних відповідей тут немає**: які тривоги хибні, вирішуєте й обґрунтовуєте ви.

Джерело: https://db.csail.mit.edu/labdata/labdata.html. Автори дозволяють вільно використовувати дані й просять посилатися на них, тож набір **обов'язково** має бути у списку джерел записки (приклад — у розділі про звіт).

4.2. Завантаження і перетворення

Одна команда з папки проєкту (віртуальне середовище активне):

```
python realdata/intel_lab.py
```

Скрипт:

1. завантажує у папку проєкту `data.txt.gz` (34 МБ) і `mote_locs.txt` (координати вузлів), якщо їх там ще немає;
2. читає журнал, переводить місцевий час лабораторії в UTC і друкує звіт про якість даних;
3. записує `intel_lab.parquet` — у **тому самому форматі, що й ваш варіант**, тож на ньому працює весь ваш код, — а також маніфест `intel_lab.json` і таблицю по вузлах `intel_lab_motes.csv`.

Триває 10–20 секунд і потребує близько 1 ГБ пам'яті: на ноутбуці з 4–8 ГБ спершу закрийте браузер.

Справжній вивід:

```
Завантажую https://db.csail.mit.edu/labdata/data.txt.gz
  data.txt.gz: готово, 34.4 МБ
Завантажую https://db.csail.mit.edu/labdata/mote_locs.txt
  mote_locs.txt: готово, 0.6 КБ
Читаю data.txt.gz (2,3 млн рядків)…

=== Звіт про якість даних ===
Рядків у файлі          : 2 313 682
```

```
  не розібрано          : 0
  без номера вузла      : 526 (відкинуто)
  з нерозпізнаним часом : 0 (відкинуто)
Придатних рядків        : 2 313 156
Період, UTC             : 2004-02-28 08:58 — 2004-04-05 18:02 (37.4 доби)
  у поясі America/Los_Angeles: 2004-02-28 00:58 — 2004-04-05 11:02

Вузлів у mote_locs.txt  : 54; з даними: 54
  записів на вузол      : мін 35, медіана 43 443, макс 65 694
  дуже мало записів     : mote-05 (35), mote-15 (2 337)
Вузли, яких НЕМАЄ в mote_locs.txt: mote-55 (2 851), mote-56 (2 507), mote-57 (3), mote-58
(4 502), mote-6485 (1), mote-33117 (1), mote-65407 (1)

Порожні поля (показання відсутнє):
  temperature : 375
  humidity    : 376
  light       : 93 352
  voltage     : 0
  освітленості немає взагалі у: mote-05, mote-28, mote-57, mote-6485, mote-33117, mote-
65407

Інтервал між записами вузла: медіана 33 с (за документацією 31 с)
  перерв > 10 хв      : 6 166 (на вузол: медіана 67, макс 337)
  найдовша перерва    : 27.3 год (mote-34)
  мовчала вся мережа  : з 2004-03-10 17:06 UTC, 24.5 год
  мовчала вся мережа  : з 2004-03-16 00:02 UTC, 19.0 год
  мовчала вся мережа  : з 2004-04-02 23:32 UTC, 11.7 год
  мовчала вся мережа  : з 2004-04-03 15:48 UTC, 6.1 год
  мовчала вся мережа  : з 2004-04-04 00:51 UTC, 8.8 год

Розряд батарей: показань з напругою < 2.3 В: 403 751 (17.5 %)
  серед них температура > 50 °C: 91.7 %; серед решти: 2.0 %
  значення 122.153 °C (стеля давача): 370 196

Поза фізичними межами:
  temperature  [0, 50]: нижче 539, вище 407 597   (мін −38.4, макс 385.568)
  humidity     [0, 100]: нижче 295 183, вище 3 901   (мін −8983.13, макс 137.512)
  light        [0, 100000]: нижче 0, вище 0   (мін 0, макс 1847.36)
  voltage      [2, 3.3]: нижче 2 113, вище 1   (мін 0.00910083, макс 18.56)

Дублікати: той самий вузол і та сама секунда: 323 (з них повністю однакових: 31)
  повторів номера epoch у того самого вузла: 113 610 — після
  перезапуску вузла лічильник іде спочатку, тож epoch не є ні часом, ні ключем

Записано intel_lab.parquet: рядків 9 158 521, 21.9 МБ
  за показниками: temperature 2 312 781, humidity 2 312 780, light 2 219 804, voltage 2 313
156
Записано intel_lab.json і intel_lab_motes.csv
Готово за 10 с.
```

Перевірка. У папці проєкту з'явилися `intel_lab.parquet`, `intel_lab.json` та `intel_lab_motes.csv`, а останні рядки виводу такі самі, як вище.

Якщо завантаження не вдалося (немає інтернету, університетський проксі), скрипт надрукує обидві адреси. Завантажте файли браузером, покладіть у папку проєкту і запустіть скрипт ще раз. Розпаковувати `data.txt.gz` не треба; якщо браузер (Safari) розпакував його сам у `data.txt`, скрипт прийме і такий файл.

Увага. Не відкривайте `data.txt` в Excel: у ньому 2,3 млн рядків, а аркуш Excel вміщує трохи більше мільйона, тож ви побачите менше половини даних і навіть не помітите цього. Малі таблиці

(`intel_lab_motes.csv`, `alarms.csv`) в Excel відкривати можна: **Дані → З тексту/CSV**, роздільник — кома.

Час: чому UTC

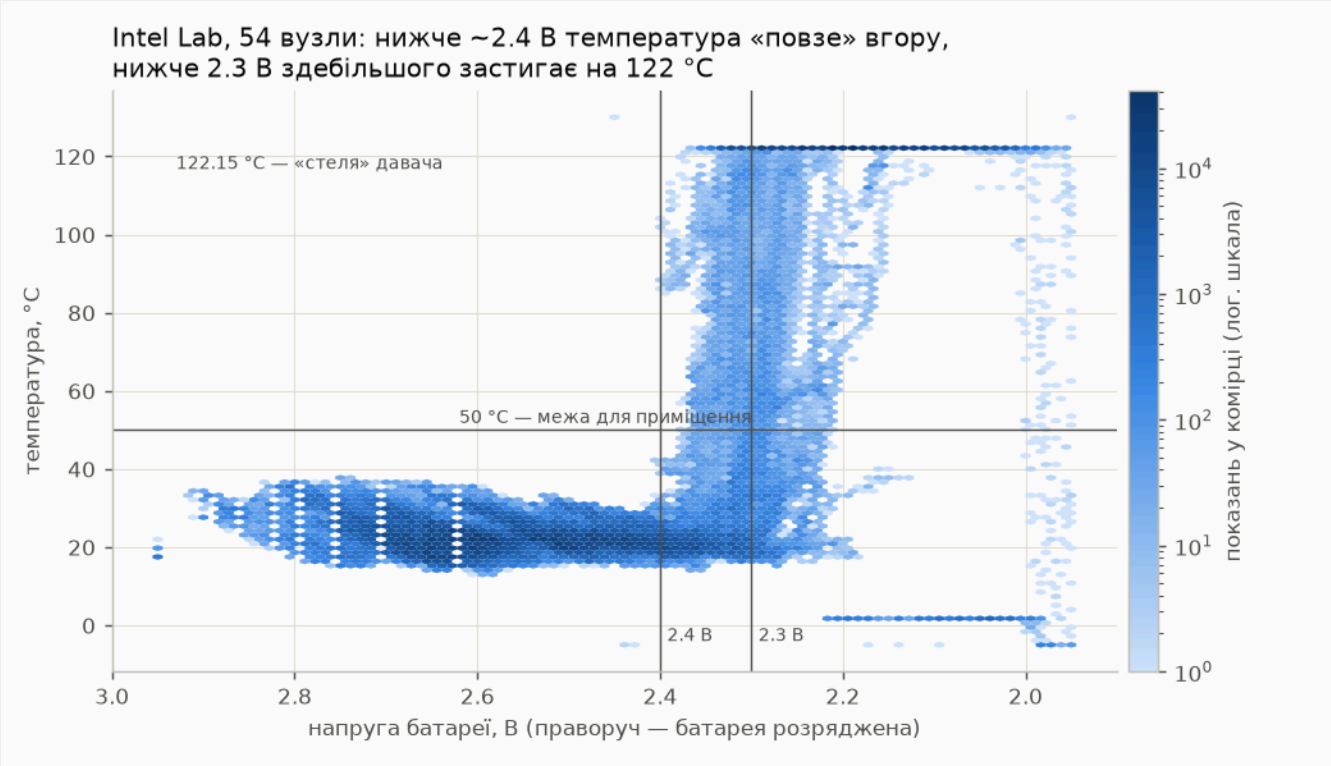
У файлі записано **місцевий** час лабораторії (Каліфорнія, UTC−8, а з 4 квітня — UTC−7). У проєкті весь час — в UTC, тож скрипт переводить його. Наслідок, про який варто пам'ятати: ранок у лабораторії, 07:00, — це 15:00 за UTC і 17:00 за київським часом. Коли бачите на графіку подію «о 15:00», згадайте, котра година була в лабораторії. У записці вкажіть, у якому поясі наводите час.

4.3. Проблеми даних: читаємо звіт

Звіт з 4.2 — це вже половина розділу «перелік виявлених проблем». Розберіть кожен блок:

Проблема	Що у звіті	Чим загрожує детекторам
Розряд батарей	17,5 % показань при напрузі < 2,3 В; 370 196 разів рівно 122,153 °C	сотні тисяч неможливих значень, «застиглих» на стелі давача
Неможливі значення	температура до 385 °C, вологість до −8983 %, напруга до 18,56 В	тривоги, яких не було в приміщенні
Перерви	6 166 перерв понад 10 хв; уся мережа мовчала до доби	вікно детектора «перестрибує» перерву і порівнює непорівнянне
Чужі пристрої	`mote-55` … `mote-58` і три зіпсовані номери, яких немає в `mote_locs.txt`	реальний аналог класу `rogue_device` з вашого варіанта
Неповні ряди	`mote-05` — 35 показань; `mote-28` ніколи не надсилав освітленості	ряд, на якому детектор нічого не може «вивчити»
Дублікати	323 збіги «вузол + секунда»; `epoch` повторюється 113 610 разів	подвоєні точки, а видалення дублікатів за `epoch` знищило б 113 тис. справжніх

Найцікавіша проблема — батареї. Ось усі 54 вузли разом: температура проти напруги батареї (що темніша комірка, то більше там показань):



Поки напруга вища за 2,4 В, температура звичайна для приміщення (15–38 °C). Між 2,4 і 2,3 В вона «повзе» вгору, нижче 2,3 В здебільшого застигає на 122 °C. Давач не нагрівся — він бреше, бо йому бракує живлення.

Ваше завдання тут. Відкрийте `intel_lab_motes.csv` і знайдіть три вузли з найбільшою часткою показань при низькій напрузі (`low_voltage_share`) і три — з найбільшою кількістю перерв (`gaps_over_10min`). Їх ви розглядатимете далі.

4.4. До і після: як обробка змінює кількість тривог

```
python realdata/clean.py --plot mote-07
```

Скрипт запускає простий детектор (ковзна робастна z-оцінка) на температурі кожного вузла, а потім по одному додає кроки обробки й після **кожного** кроку рахує тривоги знову:

Крок	Що робить	Навіщо
1	прибирає дублікати (вузол + показник + секунда)	подвоєні точки
2	лишає лише вузли з `mote_locs.txt`	чужі й зіпсовані номери
3	відкидає всі показники вузла в ті секунди, коли напруга < 2,3 В	розряд батареї
4	відкидає значення поза фізичними межами (0–50 °C тощо)	неможливі значення
5	не дає вікну детектора перетинати перерви понад 15 хв	перерви зв'язку
6	рахує 5-хвилинні медіани	дрібний шум і нерівний інтервал
7	прибирає тривоги, що спрацювали одночасно на чверті й більше вузлів	події всієї будівлі

Справжній вивід (рядки поступу пропущено):

```
Детектор: ковзна робастна z-оцінка, вікно 120 точок, поріг 5; показник: temperature
…
Крок                                        Показань  Неможл.     Точок  На смітті
Епізодів  Вузлів
0. сирі дані                               2 312 781  408 136    55 060        6 %     1
556      55
1. без дублікатів (вузол+показник+секунда)    2 312 463  408 081    55 080        6 %     1
556      55
2. лише вузли з mote_locs.txt                2 302 602  408 078    54 913        6 %     1
546      53
3. без показань при напрузі < 2.3 В          1 899 221   37 489    54 204        2 %     1
482      53
4. фізичні межі (temperature 0…50)           1 861 732        0    53 131        0 %     1
442      53
5. вікно не перетинає перерв > 15 хв         1 861 732        0    49 422        0 %     1
254      53
6. + 5-хвилинні медіани                        304 942        0     5 357        0 %     1
222      53
7. + без подій усієї будівлі                   304 942        0       843        0 %
336      51

Епізодів тривог: 1 556 -> 336 (-78 %)
Неможл. — значення поза фізичними межами, що лишилися в даних. Детектор
їх майже не помічає: ковзна медіана швидко звикає до «застиглих»
неможливих значень (122 °C, -4 %) і далі вважає їх нормою.
Показань у рядках 6–7 — це 5-хвилинні медіани, тож «Точок» там
не порівнюйте з рядками 0–5; порівнюйте «Епізодів».
На смітті — частка «Точок» на показаннях, які прибирають кроки 2–4
(вузол не з mote_locs.txt, розряд батареї, поза межами); з кроку 4 — 0.

Коли починаються епізоди (місцевий час лабораторії):
  до обробки   : 07:00–07:59 — 46 %, 08:00–08:59 — 9 %, 18:00–18:59 — 6 %, 13:00–13:59 — 5
%
  після обробки: 07:00–07:59 — 13 %, 16:00–16:59 — 12 %, 18:00–18:59 — 12 %, 10:00–10:59 —
8 %
  найбільше після обробки: mote-09 (12), mote-32 (12), mote-21 (11), mote-22 (11), mote-46
(11)

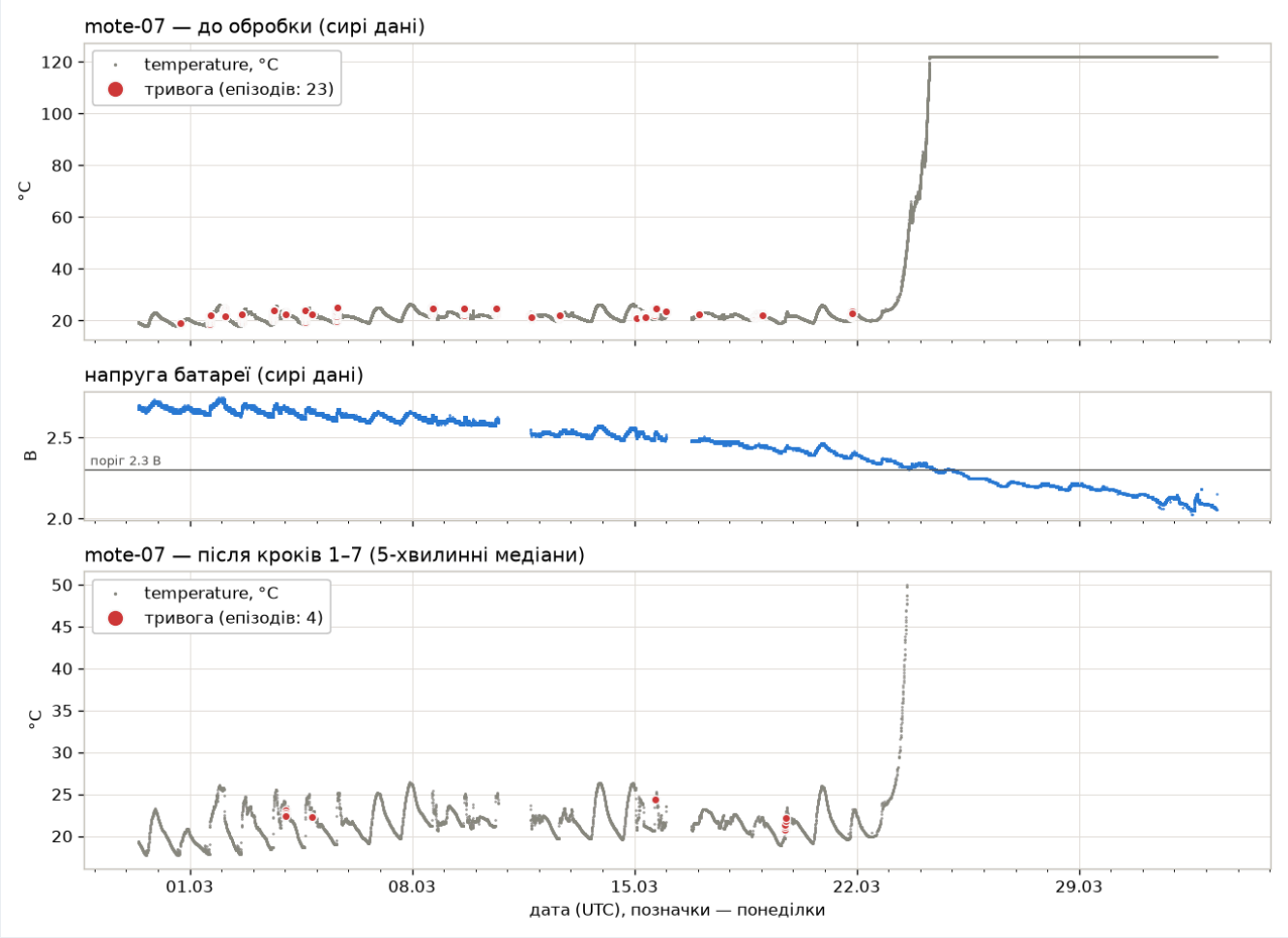
Записано intel_lab_clean.parquet (рядків: 7 477 479) і alarms.csv (епізодів до: 1 556,
після: 336)
Рисунок: mote-07_temperature.png
Готово за 14 с.
```

Що означають стовпці:

- **Показань** — скільки значень температури лишилося після кроку;
- **Неможл.** — скільки з них поза фізичними межами;
- **Точок** — скільки значень детектор позначив;
- **На смітті** — яка частка цих позначок припала на «сміття»: показання вузлів поза переліком, при розрядженій батареї або поза фізичними межами (саме їх прибирають кроки 2–4, тож з кроку 4 тут 0 %);
- **Епізодів** — скільки окремих тривог: позначки одного вузла, між якими менше 30 хв, — це один епізод. Саме епізоди порівнюйте між рядками;
- **Вузлів** — на скількох вузлах була хоч одна тривога.

Скрипт також записує:

- `intel_lab_clean.parquet` — дані після кроків 1–4, у тому самому форматі;
- `alarms.csv` — усі епізоди до (`before`) і після (`after`) обробки, з часом початку в UTC і за місцевим часом лабораторії;
- `mote-07_temperature.png` — рисунок для записки:



Вгорі — сирі дані з 23 епізодами тривог і злетом до 122 °C, посередині — напруга батареї, що перетинає 2,3 В саме тоді, коли температура «злітає», унизу — ряд після обробки з 4 епізодами.

На що звернути увагу (і пояснити у записці своїми словами):

1. Кроки 3–4 прибрали понад 400 тис. неможливих значень, а епізодів поменшало лише на сотню. Чому? Підказка — в останніх рядках виводу і на верхній панелі рисунка: що бачить ковзна медіана через годину після «злету» до 122 °C?
2. Найбільше скоротив тривоги крок 7. Подивіться, о котрій годині починалася майже половина епізодів **до** обробки. Що відбувається в офісній лабораторії о 07:00 на багатьох вузлах одночасно?
3. Деякі тривоги **після** обробки все одно хибні — див. 4.6.

4.5. Ваш детектор на реальних даних

Вказівки вимагають застосувати до реальних даних **ваші** детектори зі стадії 3. `clean.py` запускає будь-який із них замість вбудованого — так само по черзі на кожному вузлі й після кожного кроку обробки:

```
python realdata/clean.py --detector baseline
python realdata/clean.py --detector ewma
```

Для призначеного методу пишіть його назву: `ewma`, `stl_esd`, `matrix_profile`, `isolation_forest` або `lof` (бібліотеку методу має бути встановлено, як на стадії 3). Метод працює з тими самими типовими параметрами, що й на стадії 3, на 5-хвилинній сітці (STL+ESD — на 15-хвилинній). Змінити параметри можна ключем `--method-args` — у лапках, **з пробілом** між назвою і значенням, як у командному рядку стадії 3:

```
python realdata/clean.py --detector matrix_profile --method-args "--window 6h"
```

Увага. Деякі методи працюють довго, бо проходять 54 вузли вісім разів: вбудований, базовий і EWMA — до пів хвилини, Matrix Profile — близько хвилини, LOF — 2–3 хвилини, Isolation Forest — близько 3, STL+ESD — 5–15 хвилин. Запускайте і займіться чимось іншим.

Два типи детекторів

Порогові (вбудований, базовий, EWMA) позначають усе, що виходить за поріг. Чим чистіші дані, тим менше тривог. Справжній вивід для EWMA:

```
Детектор: detect/methods/ewma.py (EWMA: λ=0.1, L=5, еталон — перші 7 діб, профіль workdays;
сітка 5min); показник: temperature
…
Крок                                        Показань  Неможл.     Точок  На смітті
Епізодів  Вузлів
0. сирі дані                               2 312 781  408 136   576 412       75 %     1
205      54
1. без дублікатів (вузол+показник+секунда)    2 312 463  408 081   576 271       75 %     1
206      54
2. лише вузли з mote_locs.txt                2 302 602  408 078   576 173       75 %     1
204      53
3. без показань при напрузі < 2.3 В          1 899 221   37 489   183 504       20 %
601      53
4. фізичні межі (temperature 0…50)           1 861 732        0   145 962        0 %
506      52
5. вікно не перетинає перерв > 15 хв         1 861 732        0    64 419        0 %
266      52
6. + 5-хвилинні медіани                        304 942        0    11 206        0 %
276      52
7. + без подій усієї будівлі                   304 942        0     8 336        0 %
257      51

Епізодів тривог: 1 205 -> 257 (-79 %)
Неможл. — значення поза фізичними межами, що лишилися в даних. EWMA
порівнює кожну точку з «нормою» першого тижня ряду, тож неможливі
значення позначає майже всі — звідси великі «Точок» і «На смітті».
Показань у рядках 6–7 — це 5-хвилинні медіани, тож «Точок» там
не порівнюйте з рядками 0–5; порівнюйте «Епізодів».
На смітті — частка «Точок» на показаннях, які прибирають кроки 2–4
(вузол не з mote_locs.txt, розряд батареї, поза межами); з кроку 4 — 0.
```

«Частка ряду» (STL+ESD, Matrix Profile, Isolation Forest, LOF) щоразу позначають приблизно сталу частку найнезвичніших точок кожного ряду. Тому після очищення кількість тривог майже не падає — змінюється те, **що** позначено. Справжній вивід для LOF:

```
Детектор: detect/methods/lof.py (LOF: сусідів 300, contamination 0.02, вікно ознак 2h;
сітка 5min); показник: temperature
…
Крок                                          Показань  Неможл.     Точок  На смітті
Епізодів  Вузлів
0. сирі дані                                 2 312 781  408 136    32 848       70 %
858      55
1. без дублікатів (вузол+показник+секунда)   2 312 463  408 081    32 321       70 %
833      55
2. лише вузли з mote_locs.txt                2 302 602  408 078    32 202       70 %
818      52
3. без показань при напрузі < 2.3 В          1 899 221   37 489    30 607       25 %     1
155      53
4. фізичні межі (temperature 0…50)           1 861 732        0    34 207        0 %     1
459      53
5. вікно не перетинає перерв > 15 хв         1 861 732        0    35 144        0 %     1
685      52
6. + 5-хвилинні медіани                        304 942        0     5 936        0 %     1
733      52
7. + без подій усієї будівлі                   304 942        0     2 252        0 %
780      52

Епізодів тривог: 858 -> 780 (-9 %)
Неможл. — значення поза фізичними межами, що лишилися в даних.
Цей метод щоразу позначає приблизно сталу частку кожного ряду, тож після
очищення тривоги не зникають, а переходять на інші точки: дивіться, ЩО
позначено («На смітті»), а не лише скільки. Після кроку 5 кожен шматок
ряду дістає власну частку — тому епізодів там може стати більше.
Показань у рядках 6–7 — це 5-хвилинні медіани, тож «Точок» там
не порівнюйте з рядками 0–5; порівнюйте «Епізодів».
На смітті — частка «Точок» на показаннях, які прибирають кроки 2–4
(вузол не з mote_locs.txt, розряд батареї, поза межами); з кроку 4 — 0.
```

До очищення 70 % позначок LOF припадали на сміття (розряджені батареї, неможливі значення), після — 0 %: тепер метод показує найнезвичнішу поведінку справжніх даних. Ріст епізодів після кроків 3–5 — не помилка, а властивість методу: менше сміття → «незвичними» стають інші точки, а розрізаний перервами ряд дає кожному шматку власну частку.

У звіт. Таблиця «до/після» для вашого призначеного детектора і пояснення: які кроки обробки найбільше вплинули саме на нього і чому. Для методів «частки ряду» порівнюйте стовпець «На смітті», а не лише кількість епізодів.

4.6. Які тривоги хибні — ваш аналіз

Це та частина, яку за вас не зробить жоден скрипт.

1. Відкрийте `alarms.csv`. Рядки `after` — тривоги, що лишилися після обробки. Стовпець `start_local` — час початку за годинником лабораторії.
2. Візьміть 3–5 вузлів з найбільшою кількістю тривог (скрипт друкує їх у рядку «найбільше після обробки») і для кожного побудуйте рисунок:

```
python realdata/clean.py --plot mote-09
```

Розглядаєте тривоги свого призначеного детектора — додайте той самий ключ, з яким їх отримали (наприклад, `--detector ewma`): без нього скрипт намалює тривоги вбудованого детектора і

перезапише ними `alarms.csv`.

3. Для кожної тривоги вирішіть: це справжня аномалія (несправність, незвичайна подія) чи нормальна поведінка, яку детектор помилково вважає аномалією? Аргументуйте: час доби, день тижня, сусідні вузли (координати — у `intel_lab.json`), напруга батареї, освітленість у той самий момент (`--metric light`).
4. Спробуйте змінити обробку і подивіться, що зміниться: - `--min-voltage 2.4` — суворіший поріг батареї: скільки показань ви втратите і скільки хибних тривог приберете? Чи варто? - `--metric humidity`, `--metric light` — ті самі кроки для інших показників. - `--window 240 --z 6` — інші налаштування детектора.

Правильної відповіді немає — є переконливе чи непереконливе обґрунтування. Його й оцінюють.

У звіт (підрозділ 3.2)

- Перелік проблем реальних даних з числами зі звіту `intel_lab.py` (таблиця з 4.3, але з вашими формулюваннями).
- Вжиті заходи: кроки обробки і чому кожен потрібен.
- Таблиця «до/після» для базового і вашого призначеного детектора (епізоди тривог після кожного кроку).
- Рисунки: `mote-07_temperature.png` і ваші рисунки з 4.6.
- Обґрунтування, які тривоги хибні і чому; що з цього ви перенесли у попередню обробку своїх детекторів.
- У якому часовому поясі наведено час.
- Посилання на набір Intel Lab у списку джерел.

Контрольний список стадії 4

- □ `intel_lab.py` створив `intel_lab.parquet`, `intel_lab.json` і `intel_lab_motes.csv`.
- □ Ви можете пояснити кожен блок звіту про якість даних.
- □ `clean.py` надрукував таблицю «до/після» для базового детектора і для вашого призначеного.
- □ Ви розглянули щонайменше три вузли з тривогами після обробки і вирішили, які тривоги хибні.
- □ Набір Intel Lab — у списку джерел.

Стадія 5. Пошук інцидентів у власному варіанті

Мета: знайти приховані інциденти у **вашому** наборі даних і оформити їх у `findings.json`. Це 30 балів зі 100 — найбільший складник оцінки.

Це єдина частина проєкту, для якої посібник **не дає** готового коду. Тут описано, що шукати і як оформити результат, а сам пошук — ваша робота. Саме її комісія перевірить на захисті, коли вам дадуть незнайомий набір даних.

5.1. Що відомо заздалегідь

- У вашому варіанті приховано **від 5 до 8 інцидентів**.
- Усі вони належать до **12 класів** з розділу 7 вказівок. Інших не буває.
- Кількість, класи, час і пристрої у кожного студента свої.
- Частину інцидентів не видно в жодному окремому показнику: їх знаходять, лише зіставляючи кілька пристроїв (класи `zone_incoherence`, `gateway_group_loss`, `exfil_pattern`, `sensor_swap`, а також `drift`).

5.2. Як рахуються бали

Повністю правила — у п. 6.2 вказівок. Головне:

Правило	Що це означає для вас
Оцінюються не більше 15 знахідок з найвищою `confidence`	Не надсилайте «про всяк випадок» 50 знахідок — зайві просто відкинуть
Щонайменше п'ята частина пристроїв, указаних у знахідці, має належати до інциденту	Не перелічуйте всі пристрої будівлі в одній знахідці
Час має перекриватися не менше ніж на 25 %	Вікно не повинно бути ні набагато ширшим, ні набагато вужчим за інцидент
Повторна знахідка того самого інциденту — хибне спрацювання	Одна подія — одна знахідка
Кожне хибне спрацювання — мінус 5 % (не більше 30 %)	Сумнівні знахідки краще не подавати: штраф не залежить від `confidence`, вона лише визначає, які 15 знахідок оцінюються
Правильний час, але не той клас — 70 % балів	Назву класу пишіть точно як у розділі 7

Складні інциденти важать 2,5, середні — 1,5, прості — 1,0.

5.3. Порядок роботи

1. **Огляд.** Порахуйте, скільки повідомлень надсилає кожен пристрій щогодини (скрипт обробки зі стадії 2 уже це вміє), і порівняйте перелік пристроїв у даних і в маніфесті.

2. **Від простого до складного.** Спочатку прості класи (`stuck_at`, `outage`, `spike_storm`, `rogue_device`) — вони дають упевненість і знайомство з даними. Потім середні, наприкінці складні.
3. **Кожного кандидата — на графік.** Детектор каже «тут щось є», а графік показує, що саме і де межі. Скрипти стадії 3 зберігають такі графіки.
4. **Записуйте знахідки одразу** в таблицю `my_findings.csv` (див. 5.5), не відкладайте на кінець.
5. **Упевненість.** 0,9 — бачите на графіку однозначно; 0,6 — схоже, але є сумніви; нижче 0,5 — краще не подавати. Пам'ятайте: низька упевненість не зменшує штрафу за хибну знахідку.
6. **Самоперевірка.** Зберіть `findings.json` і проженіть `selfcheck.py`.

Порада. Перш ніж шукати у своєму варіанті, спробуйте свій підхід на навчальному наборі, де відповіді відомі (`tuning_truth.json`). Якщо там клас не знаходиться, у вашому варіанті він теж не знайдеться.

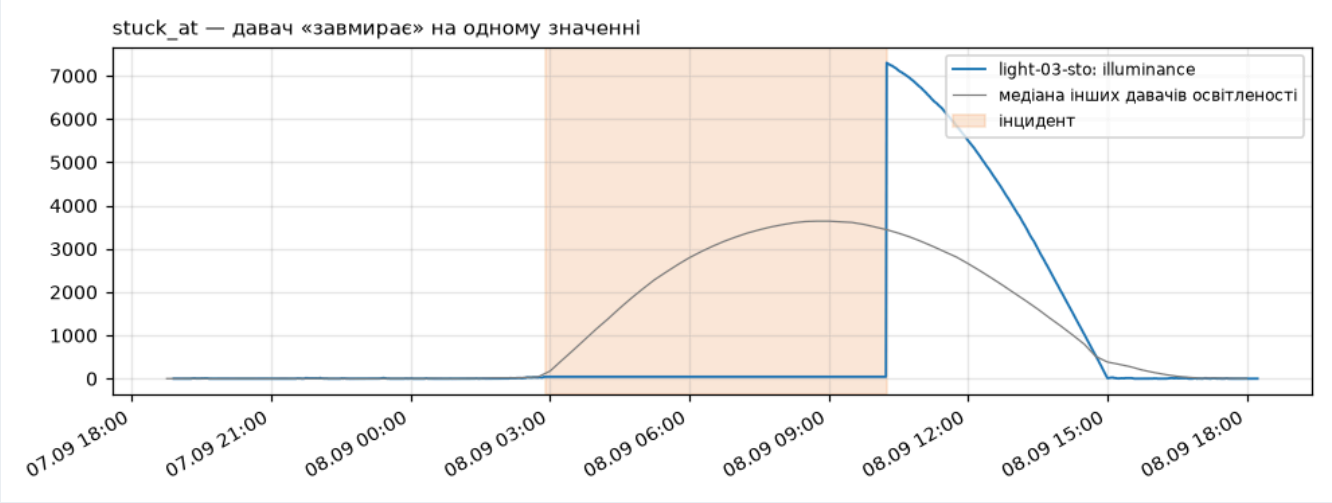
Наперед. На захисті ваш пошук запускатиметься на незнайомих даних (крок 8.2), тож уже зараз записуйте **правила**, за якими знаходите кожен клас (який показник, яке вікно, який поріг, з чим порівнюєте). Згодом ви перенесете їх у код — файл `hunt.py`.

5.4. Атлас інцидентів

Нижче — як виглядає кожен клас на навчальному наборі. Помаранчева смуга — час інциденту за відповідями. У вашому варіанті ті самі класи виглядатимуть схоже, але на інших пристроях, в інший час і з іншою силою.

Прості

`stuck_at` — давач «завмирає»



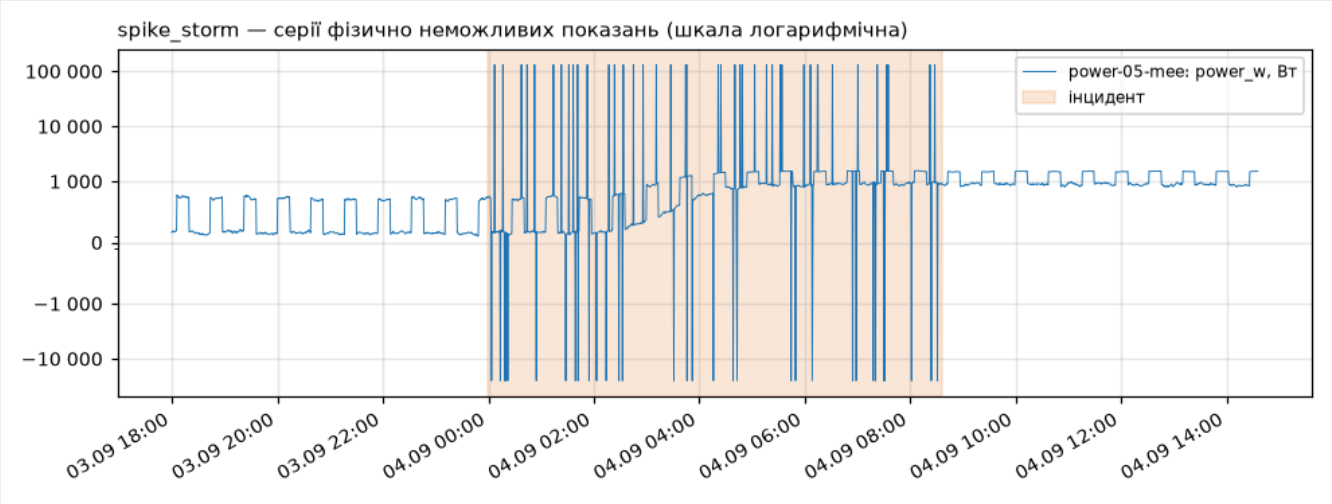
Що шукати: години, коли показання не змінюється або майже не змінюється, хоча подібні давачі в цей час змінюються. **Чим:** кількість різних значень або стандартне відхилення в рухомому вікні.

`outage` — пристрій мовчить



Що шукати: проміжок між повідомленнями пристрою, набагато довший за його номінальний інтервал (`interval_s` у маніфесті). **Чим:** різниця між сусідніми мітками часу кожного пристрою.

`spike_storm` — фізично неможливі значення



Що шукати: серії значень, яких не може бути: від'ємна потужність, сотні кіловат для офісного приладу, вологість понад 100 %. **Чим:** межі фізично можливого для кожного показника (одиниці — у маніфесті) і робастна z-оцінка.

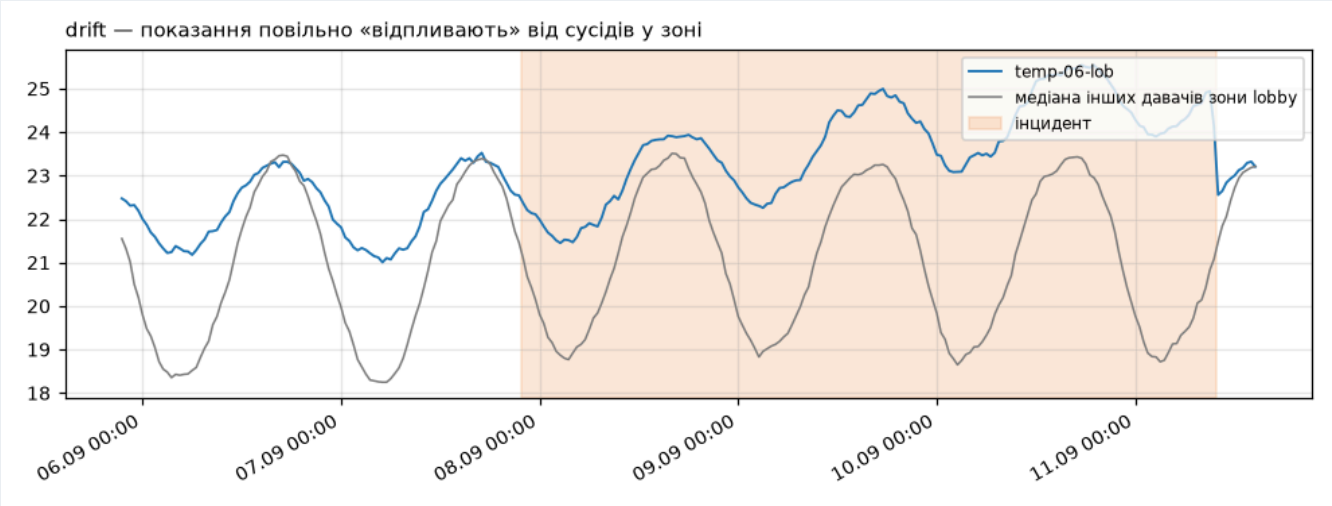
`rogue_device` — чужий пристрій



Що шукати: `device_id`, який є в даних, але відсутній у маніфесті. Час інциденту — від першого до останнього повідомлення цього пристрою.

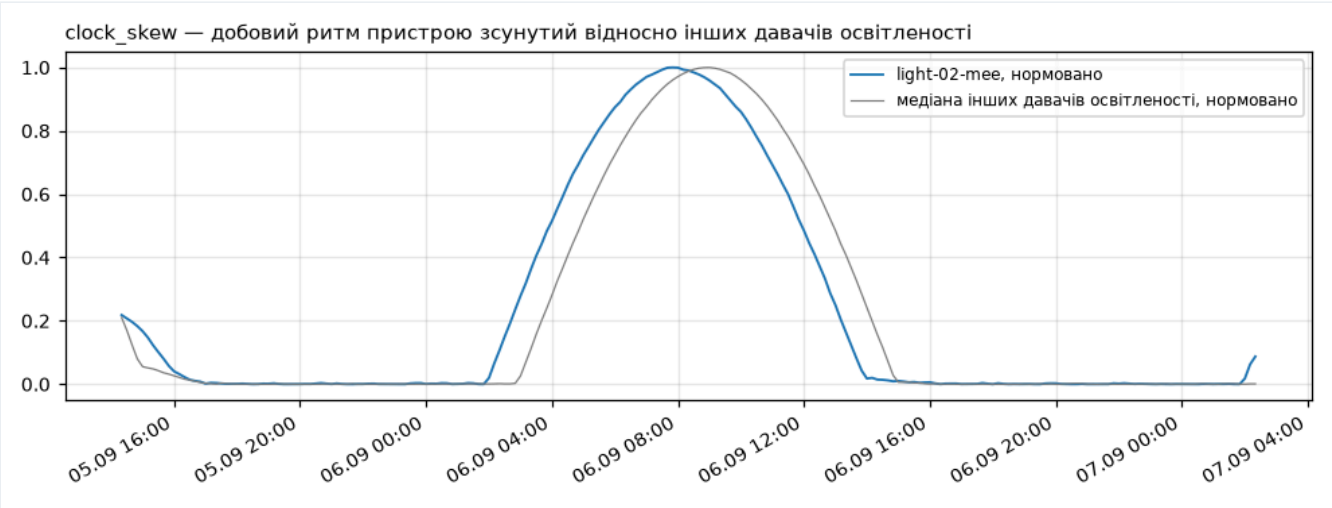
Середньої складності

`drift` — повільний дрейф



Що шукати: показання протягом днів поступово відходять від інших давачів того самого типу в тій самій зоні. **Чим:** різниця між пристроєм і медіаною зони; EWMA або кумулятивна сума цієї різниці.

`clock_skew` — зсув годинника



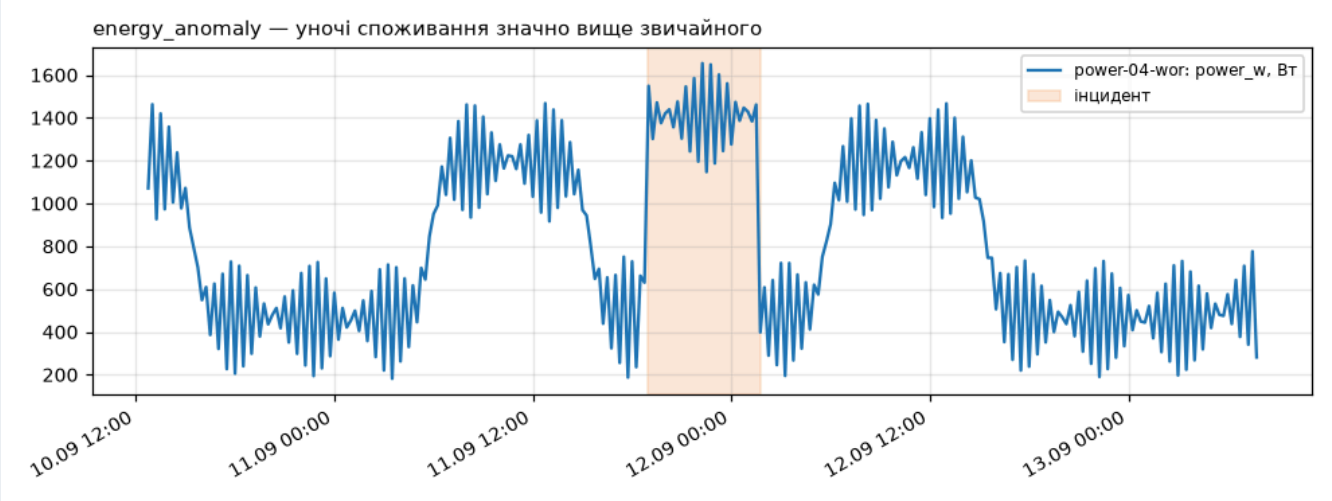
Що шукати: добовий ритм пристрою зсунутий у часі відносно подібних пристроїв. **Чим:** взаємна кореляція пристрою з медіаною подібних за кожну добу і зсув, за якого вона найбільша.

`beaconing` — підозріло рівні інтервали



Що шукати: звичайні пристрої надсилають дані з невеликим розкидом у часі. Коли інтервали між повідомленнями раптом стають ідеально однаковими, це ознака керованої активності. **Чим:** стандартне відхилення інтервалів у рухомому вікні, яке падає майже до нуля. Якщо рівні інтервали коротші за звичайні, після них пристрій може замовкнути: це продовження того самого інциденту, а не окремий `outage`. Подавайте одне вікно `beaconing` — від початку рівних інтервалів до кінця тиші.

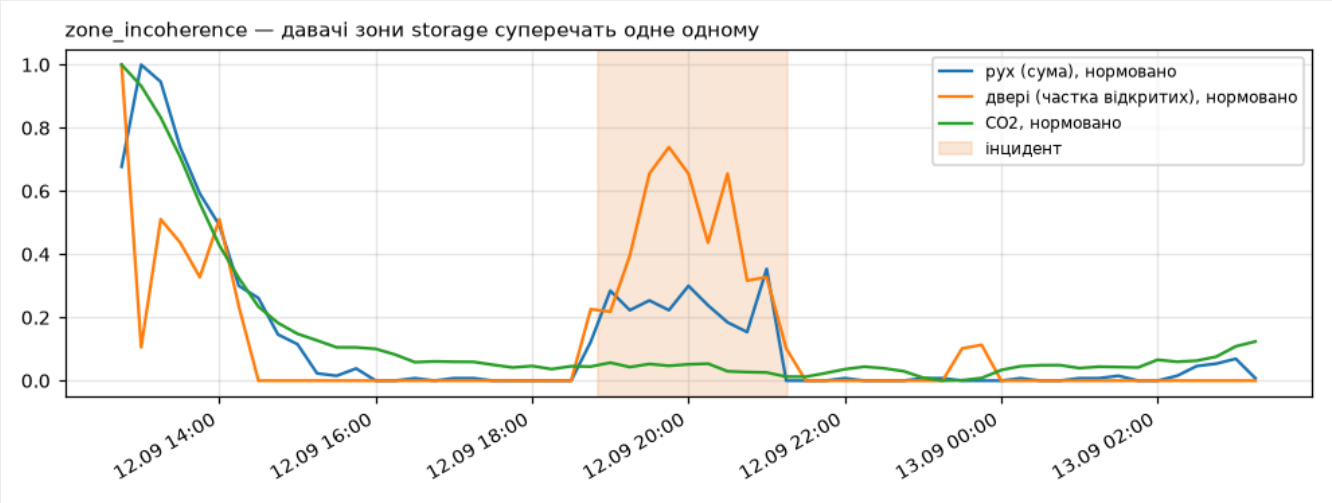
`energy_anomaly` — споживання в неробочий час



Що шукати: уночі чи у вихідні споживання значно вище, ніж у ту саму годину інших діб. **Чим:** порівняння з типовим профілем «година доби → споживання»; якщо в зоні є давач руху — перевірка, чи там справді хтось був.

Складні

`zone_incoherence` — давачі зони суперечать одне одному



Що шукати: у межах однієї зони показники присутності людей розходяться: двері відчиняються і є рух, а CO2 не росте, або навпаки. **Чим:** зведіть показники зони в одну таблицю за 15-хвилинними інтервалами і порівнюйте їх між собою.

`gateway_group_loss` — втрати за одним шлюзом



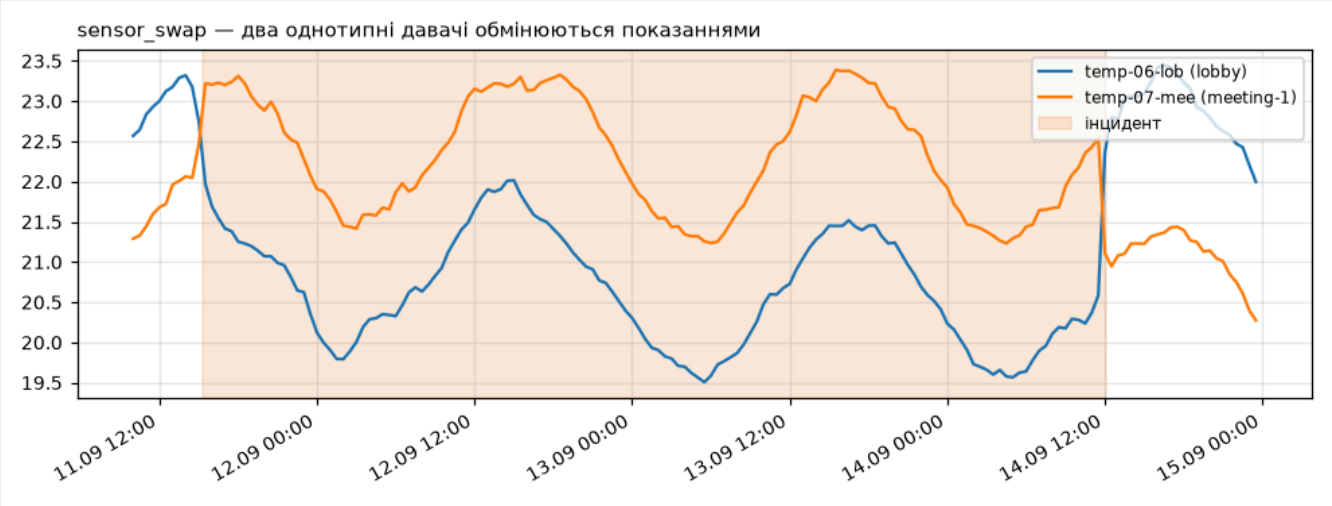
Що шукати: одночасне падіння кількості повідомлень у всіх пристроїв за одним шлюзом (поле `gateway` у маніфесті), тоді як за іншими шлюзами все гаразд. **Чим:** частота повідомлень за групами шлюзів. У знахідці перелічіть постраждалі пристрої, а не всю будівлю.

`exfil_pattern` — шлюз відправляє забагато



Що шукати: лічильник відправлених пакетів шлюзу (`pkt_sent`) росте швидше, ніж пояснює кількість повідомлень від пристроїв за ним. **Чим:** відношення приросту `pkt_sent` до кількості повідомлень за той самий час.

`sensor_swap` — давачі помінялися



Що шукати: два однотипні давачі в різних зонах раптом «обмінюються» поведінкою: кожен починає показувати те, що досі показував інший. **Чим:** порівняння кожного пристрою з його власною історією і з іншими пристроями того самого типу. У знахідці — обидва пристрої.

5.5. Оформлення `findings.json`

Вести знахідки вручну в JSON незручно і легко помилитися з дужкою. Тому ведіть їх у таблиці, а JSON зберіть скриптом.

Крок 1. Таблиця знахідок

Створіть файл `my_findings.csv` (у VS Code, Excel чи LibreOffice Calc). Перший рядок — назви колонок, далі по рядку на знахідку. Приклад для навчального набору:

```
devices,class,start,end,confidence,evidence
power-05-mee,spike_storm,2025-09-04T00:05:00Z,2025-09-04T08:30:00Z,0.95,73 показання
потужності поза фізично можливим діапазоном
```

```
"gw-01;temp-01-sto;temp-03-sto",gateway_group_loss,2025-09-08T17:15:00Z,2025-09-
09T06:10:00Z,0.8,частота повідомлень пристроїв за gw-01 впала до 13 %
power-04-wor,energy_anomaly,2025-09-11T19:00:00Z,2025-09-12T01:40:00Z,0.86,уночі споживання
вдвічі вище звичайного
```

- Кілька пристроїв — через крапку з комою, у лапках.
- Час — у UTC, у форматі `РРРР-ММ-ДДТГГ:ХХ:ССZ`.
- В Excel зберігайте через **Файл → Зберегти як → CSV UTF-8 (розділювач — кома)**. Звичайний «CSV» з крапкою з комою і десятковою комою (`0,8`) теж прочитається, але скрипт попередить, що файл не в UTF-8.

Якщо кандидатів дав детектор з ключем `--json` (стадія 3, розділ 3.5), відкрийте `windows.json` у VS Code і перенесіть у таблицю лише ті вікна, які ви перевірили на графіку: `devices`, `start` і `end` — з JSON, а `class`, `confidence` і `evidence` — ваші власні.

Крок 2. Збирання JSON

```
python make_findings.py --csv my_findings.csv --data variant_NNN.parquet --manifest
variant_NNN.json
```

На навчальному наборі:

```
Записано findings.json (знахідок: 3, варіант 0)

Тепер перевірте файл:
  python selfcheck.py --findings findings.json --data tuning.parquet --manifest tuning.json
```

Скрипт сам підставляє картку результатів з ваших даних і впорядковує знахідки за упевненістю.

Крок 3. Самоперевірка

```
python selfcheck.py --findings findings.json --data variant_NNN.parquet --manifest
variant_NNN.json
```

Добивайтеся рядка:

```
Файл придатний до перевірки.
```

Увага. `selfcheck.py` перевіряє лише **формат** і картку. Він не каже, чи правильні знахідки, — відповіді є тільки в керівника.

5.6. Що внести у звіт

Підрозділ 3.3 записки: таблиця всіх знахідок (пристрої, час, клас, упевненість) і для **кожної** — графік і пояснення, на основі чого ви дійшли висновку. Знахідка без обґрунтування на захисті викличе запитання.

Контрольний список стадії 5

- □ Перелік пристроїв у даних порівняно з маніфестом.

- □ Для кожного з 12 класів ви знаєте, чи є він у вашому варіанті і чому ви так вважаєте.
- □ Кожна знахідка має графік і обґрунтування.
- □ Знахідок не більше 15, сумнівні прибрано.
- □ `selfcheck.py` пише «Файл придатний до перевірки».

Стадія 6. Дашборд

Мета: показати стан мережі, окремий пристрій і знайдені аномалії в дашборді, зробленому **призначеним** вам інструментом, — так, щоб ним міг користуватися будь-хто, не лише ви. Складник «Дашборд» — 10 балів; його оцінюють на демонстрації.

6.1. Що вже готово

У заготовці є три готові дашборди, по одному на кожен інструмент. Усі мають однакові три сторінки — саме ті, яких вимагають вказівки (п. 4.6):

1. **Огляд мережі** — скільки пристроїв на зв'язку, карта «пристрій × година» і зведення за зонами або шлюзами;
2. **Пристрій** — показники одного пристрою в часі; вікна знахідок з `findings.json` зафарбовано;
3. **Аномалії** — знахідки на шкалі часу і таблицею, з переходом на сторінку пристрою.

Ваш інструмент записано в маніфесті (`assigned_stack` → `dashboard`):

Інструмент	Звідки бере дані	Що потрібно
Streamlit	прямо з файла Parquet (через DuckDB)	`pip install streamlit plotly`
Plotly Dash	прямо з файла Parquet (через DuckDB)	`pip install dash`
Grafana	з **вашого сховища** зі стадії 2	Docker і один підготовчий скрипт

Увесь час на всіх дашбордах — UTC.

Поки `findings.json` ще немає (до стадії 5), сторінка аномалій буде порожньою — це нормально:



Щоб побачити, як вона виглядатиме, вкажіть приклад `examples/findings.example.json` разом із навчальним набором.

6.2. Streamlit

Встановлення і запуск

```
pip install streamlit plotly
```

```
streamlit run dashboard/streamlit_app.py -- --data variant_NNN.parquet --manifest
variant_NNN.json --findings findings.json
```

Увага. Два дефіси `--` перед `--data` обов'язкові: усе, що після них, Streamlit передає дашборду. Без них буде помилка `Error: No such option '--data'.`

Під час **першого** запуску Streamlit попросить адресу пошти (`Email:`) — просто натисніть Enter. Далі (справжній вивід, адреси мережі замінено на `…`):

```
Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://…:8501
  External URL: http://…:8501
```

Браузер відкриється сам; якщо ні — відкрийте http://localhost:8501. Зупинити — Ctrl+C у терміналі. Шляхи до файлів можна змінити й просто в лівій панелі, без перезапуску. Підказку про Watchdog на macOS можна ігнорувати.

Що побачите

Deploy

Огляд мережі
Пристрій
Аномалії

Файли

Дані (Parquet)
tuning.parquet

Маніфест (JSON)
tuning.json

Знахідки (findings.json)
findings.example.json

Увесь час на графіках — UTC.

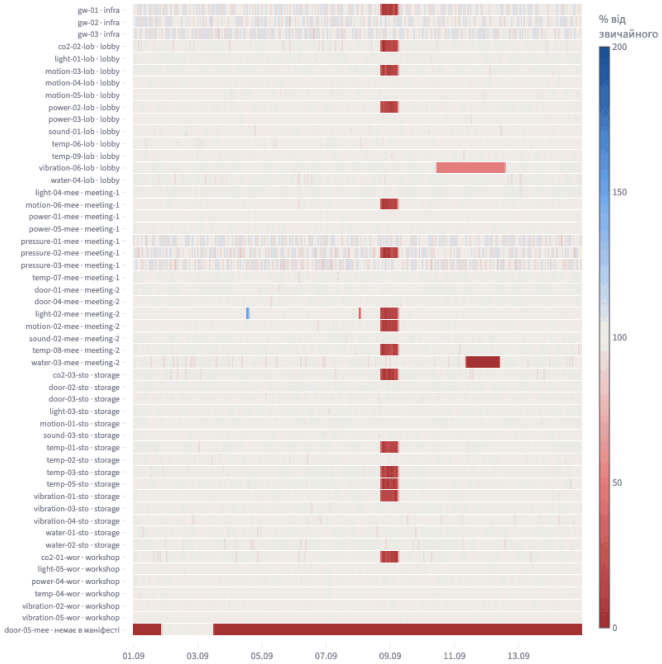
Огляд мережі

Записів	Пристроїв у даних	На зв'язку наприкінці	Період
1 157 147	52	51 з 52	14 діб

Повідомлення від кожного пристрою за кожну годину

% від звичайного для пристрою кількість повідомлень

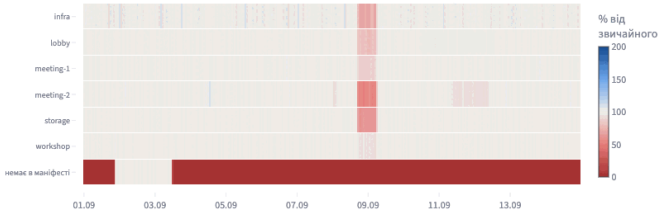
Сірий — як зазвичай, червоний — повідомлень менше або немає зовсім, синій — більше, ніж зазвичай. «Звичайне» — медіана тих годин, коли пристрій надсилав дані. Наведіть курсор на клітинку, щоб побачити числа.



Зведення за групами

Групувати за

зонами шлюзами



Зона	Пристроїв у маніфесті	Пристроїв у даних	Повідомлень	Доступність, %	Останнє (UTC)
infra	3	3	47788	99.9	14.09.2025 23:55
lobby	12	12	295876	100.0	14.09.2025 23:59
meeting-1	8	8	170768	99.9	14.09.2025 23:59
meeting-2	7	7	146192	99.0	14.09.2025 23:59
storage	15	15	344049	100.0	14.09.2025 23:59
workshop	6	6	150140	100.0	14.09.2025 23:59
немає в маніфесті	0	1	2334	11.9	03.09.2025 12:07

› Усі пристрої

Deploy

Огляд мережі
Пристрій
Аномалії

Файли

Дані (Parquet)
tuning.parquet

Маніфест (JSON)
tuning.json

Знахідки (findings.json)
findings.example.json

Увесь час на графіках — UTC.

Пристрій

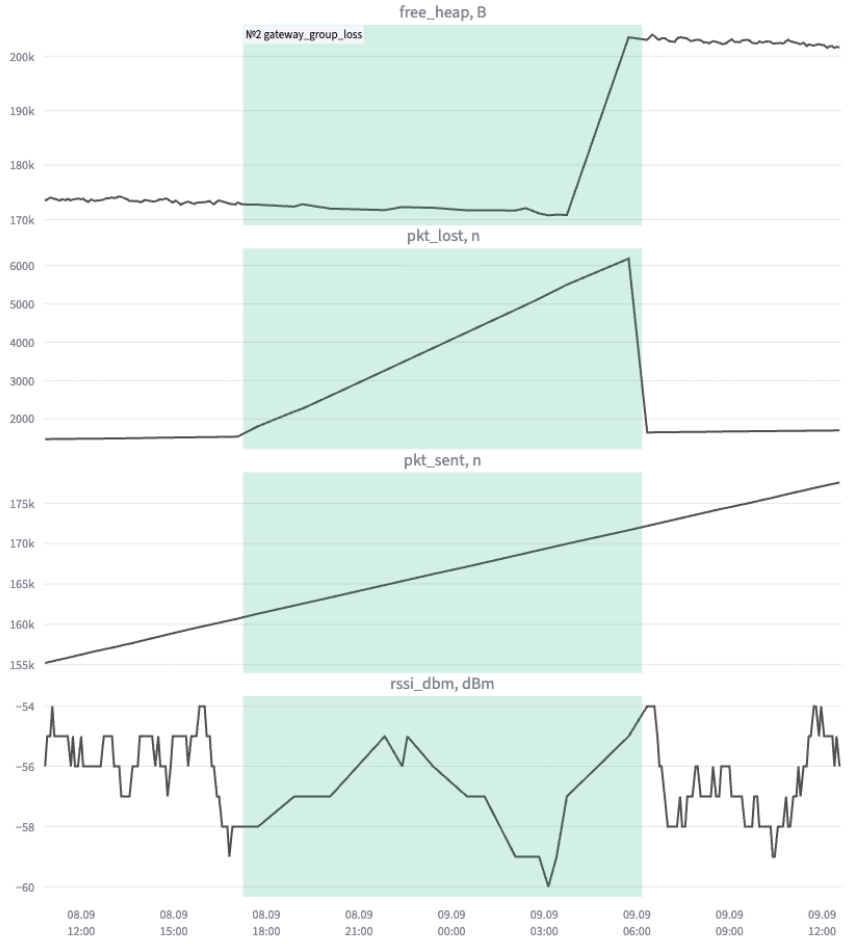
Пристрій: gw-01 · infra · gw-01

Наблизити до знахідки: №2 gateway_group_loss · 08.(

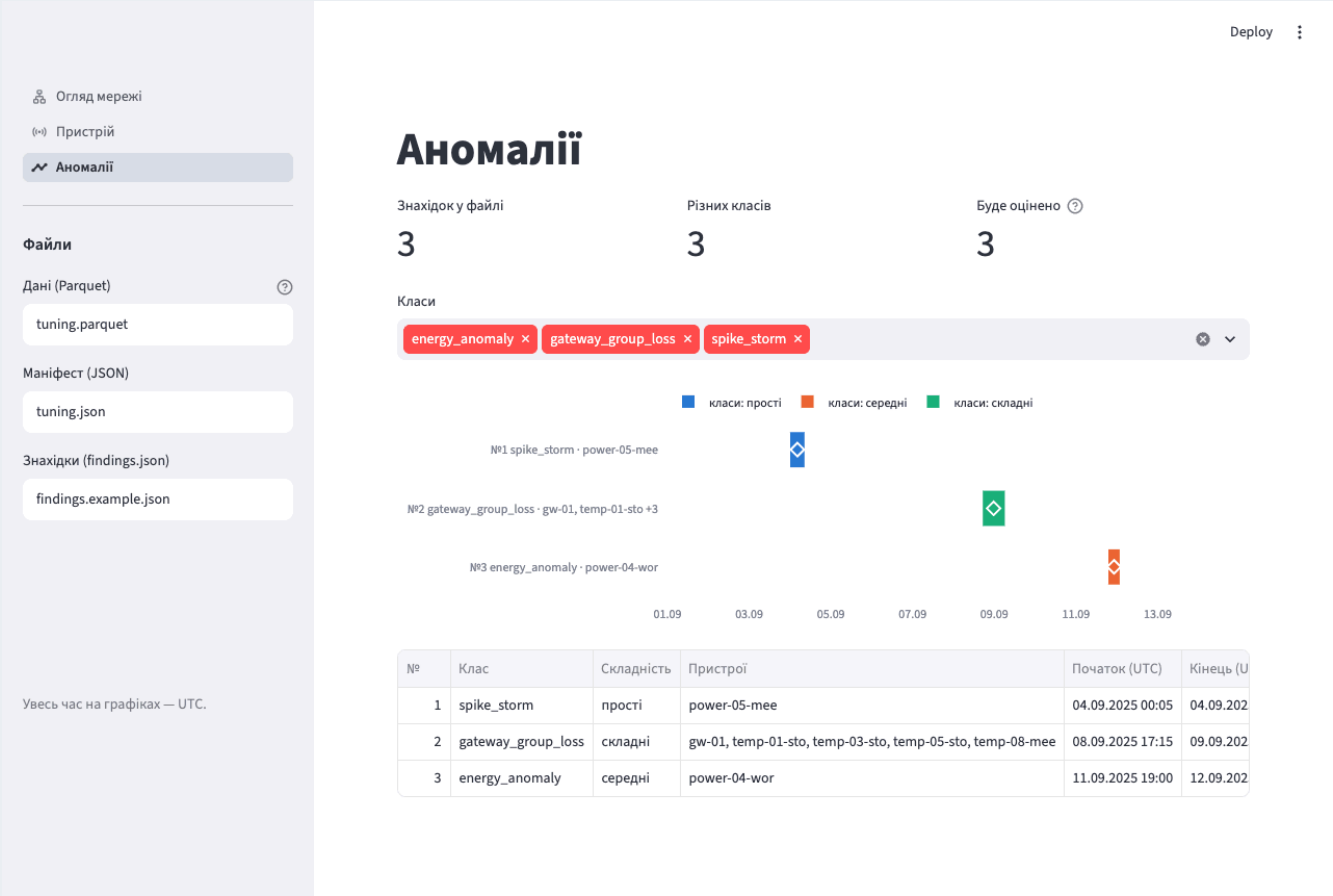
Період (UTC): 08.09.2025 10:47 — 09.09.2025 12:37

Тип gw, зона infra, шлюз gw-01, інтервал за маніфестом 300 с

Лінія — середнє за 2 хв, сіра смуга — від мінімуму до максимуму за той самий крок. Кольорові вікна — знахідки з findings.json (синій — прості класи, помаранчевий — середні, зелений — складні). Виділіть ділянку мишею, щоб збільшити; подвійний клік повертає назад.



Показник	Одиниця	Записів за весь період	Перший (UTC)	Останній (UTC)	Інтервал між записами, с (медіана)
free_heap	B	3890	01.09.2025 00:00	14.09.2025 23:54	300
pkt_lost	n	3890	01.09.2025 00:00	14.09.2025 23:54	300
pkt_sent	n	3890	01.09.2025 00:00	14.09.2025 23:54	300
rssi_dbm	dBm	3890	01.09.2025 00:00	14.09.2025 23:54	300



Посилання «відкрити» в таблиці аномалій відкриває сторінку пристрою одразу на вікні цієї знахідки.

6.3. Plotly Dash

Встановлення і запуск

```
pip install dash
```

```
python dashboard/dash_app.py --data variant_NNN.parquet --manifest variant_NNN.json --
findings findings.json
```

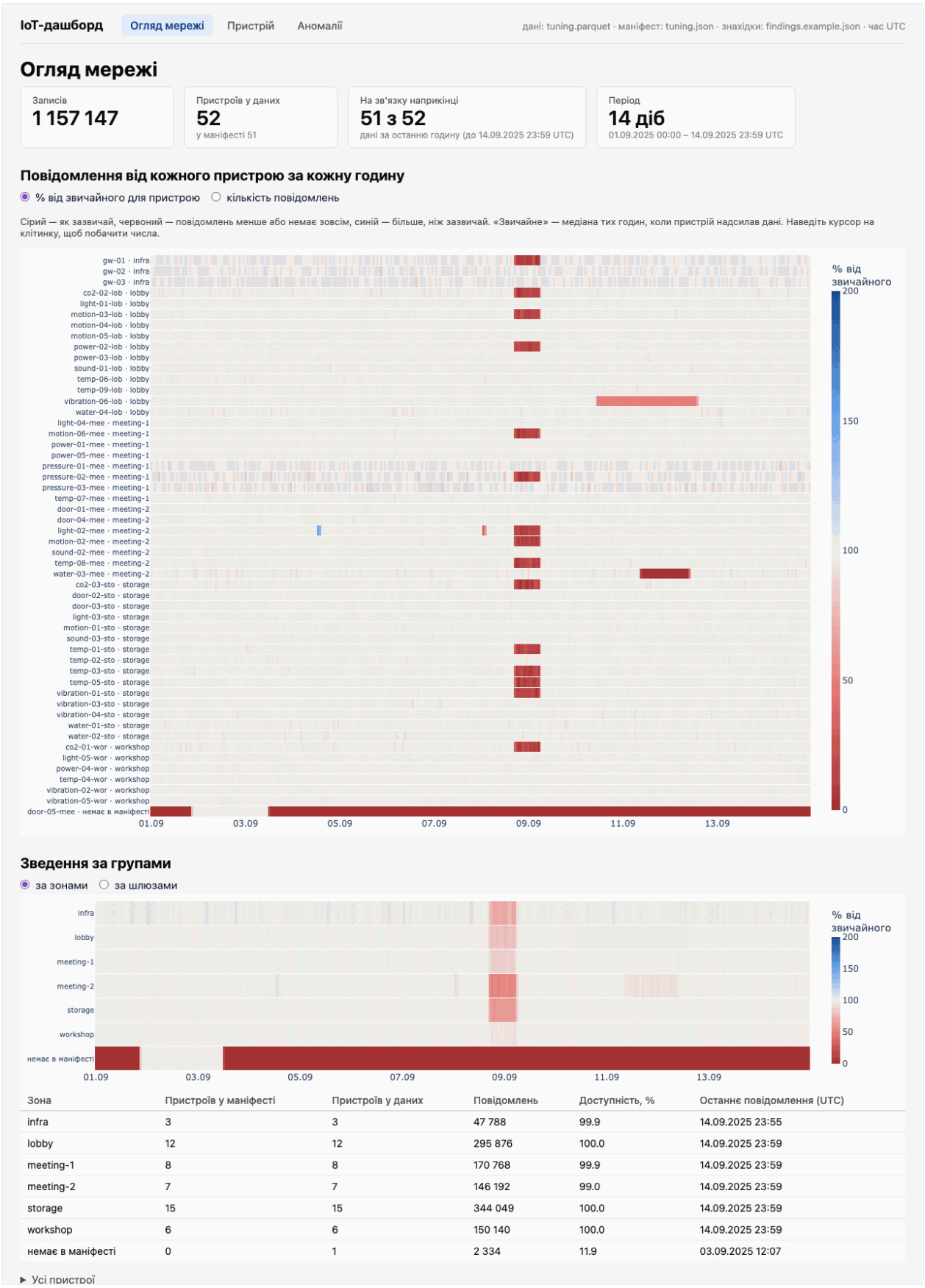
Справжній вивід:

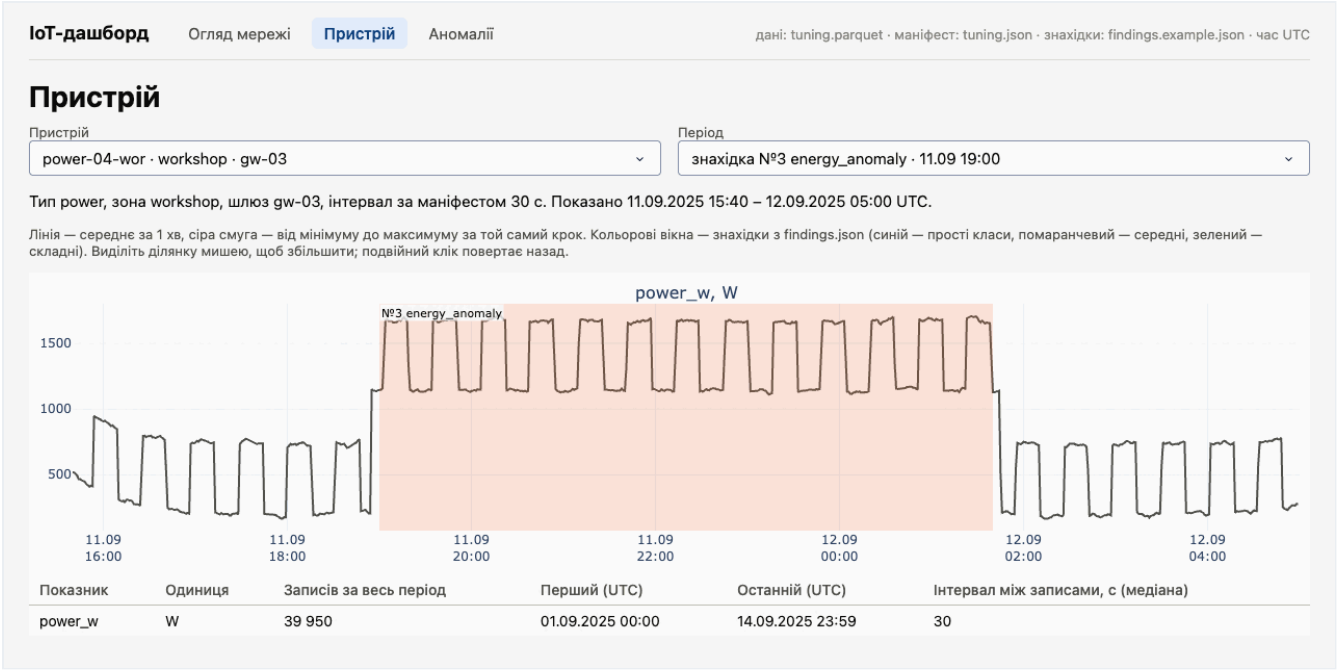
```
Дашборд: http://localhost:8050   (зупинка — Ctrl+C)
Dash is running on http://127.0.0.1:8050/

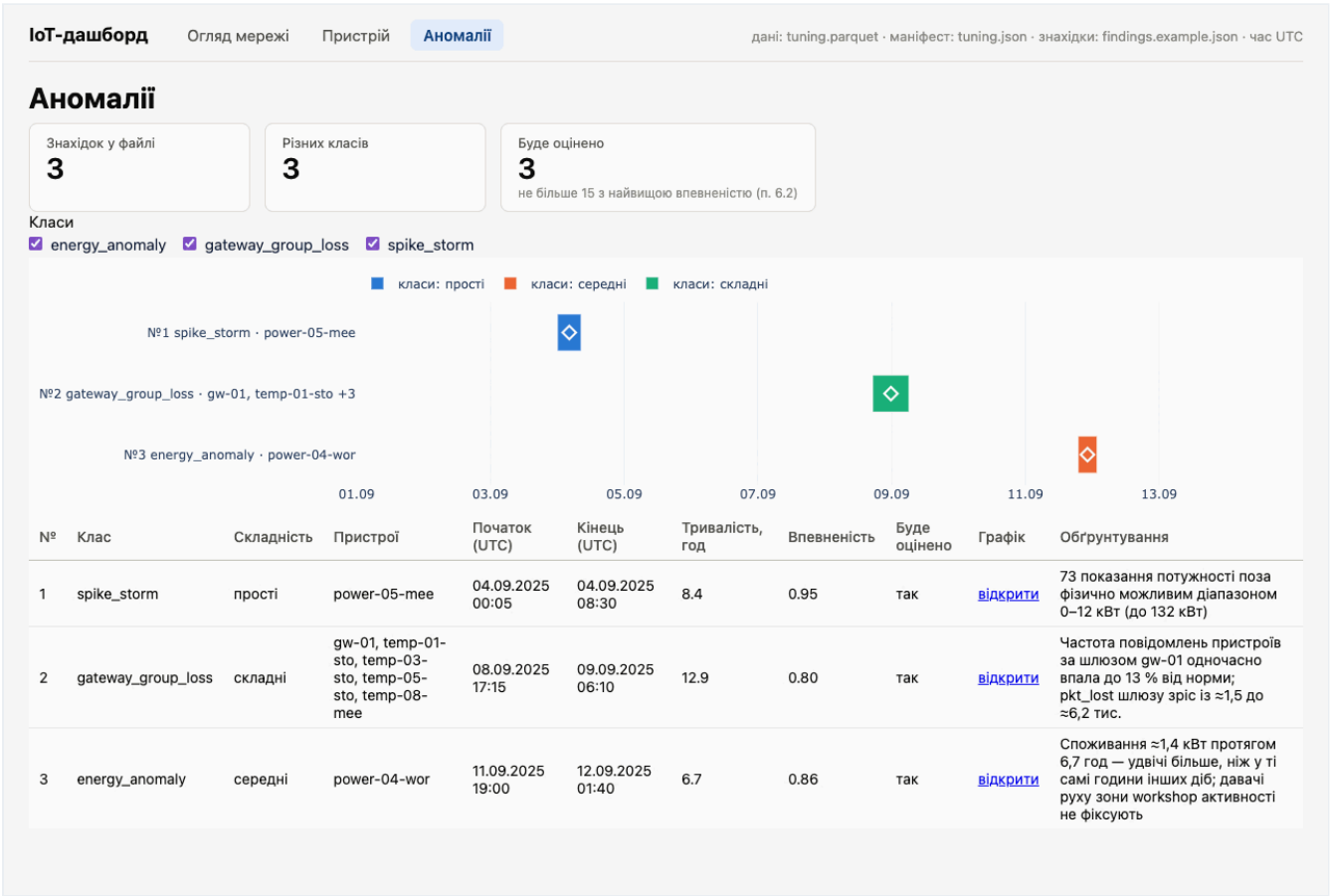
 * Serving Flask app 'dash_app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
 * Running on http://127.0.0.1:8050
Press CTRL+C to quit
```

Dash не відкриває браузер сам — відкрийте http://localhost:8050. Рядок `WARNING … development server` для дашборда на власному комп'ютері — норма. Після змін у `findings.json` просто оновіть сторінку (F5).

Що побачите


IoT-дашборд
Огляд мережі
Пристрій
Аномалії
дані: tuning.parquet · маніфест: tuning.json · знахідки: findings.example.json · час UTC
Огляд мережі
Записів
1 157 147
Пристроїв у даних
52
у маніфесті 51
На зв'язку наприкінці
51 з 52
дані за останню годину (до 14.09.2025 23:59 UTC)
Період
14 діб
01.09.2025 00:00 – 14.09.2025 23:59 UTC
Повідомлення від кожного пристрою за кожну годину
% від звичайного для пристрою
кількість повідомлень
Сірий — як зазвичай, червоний — повідомлень менше або немає зовсім, синій — більше, ніж зазвичай. «Звичайне» — медіана тих годин, коли пристрій надсилав дані. Наведіть курсор на клітинку, щоб побачити числа.
% від звичайного
200
150
100
50
0
01.09
03.09
05.09
07.09
09.09
11.09
13.09
Зведення за групами
за зонами
за шлюзами
infra
lobby
meeting-1
meeting-2
storage
workshop
немає в маніфесті
Зона
Пристроїв у маніфесті
Пристроїв у даних
Повідомлень
Доступність, %
Останнє повідомлення (UTC)
infra 3 3 47 788 99.9 14.09.2025 23:55
lobby 12 12 295 876 100.0 14.09.2025 23:59
meeting-1 8 8 170 768 99.9 14.09.2025 23:59
meeting-2 7 7 146 192 99.0 14.09.2025 23:59
storage 15 15 344 049 100.0 14.09.2025 23:59
workshop 6 6 150 140 100.0 14.09.2025 23:59
немає в маніфесті 0 1 2 334 11.9 03.09.2025 12:07
Усі пристрої





На сторінці пристрою період вибирають зі списку: увесь період, кожна знахідка цього пристрою або окрема доба. Ділянку графіка можна виділити мишею, подвійний клік повертає назад.

6.4. Grafana

Grafana працює в Docker (як сховища на стадії 2) і читає дані з **вашого** сховища. Вона не вміє читати файли JSON, тому маніфест і знахідки спершу записує у сховище підготовчий скрипт.

Перед початком:

- Docker Desktop запущено (стадія 2).
- Якщо ваше сховище InfluxDB чи TimescaleDB — його контейнер працює і в ньому ваші дані: `docker compose -f pipeline/docker-compose.yml up -d`.
- Перший запуск Grafana потребує інтернету: завантажується образ (1,3 ГБ) і модуль SQLite.

Крок 1. Підготовка даних

```
python dashboard/grafana/prepare.py --manifest variant_NNN.json --data variant_NNN.parquet
--findings findings.json
```

Сховище скрипт бере з маніфесту. Справжній вивід для SQLite на навчальному наборі (у `tuning.json` записано InfluxDB, тому для навчального набору додайте `--storage` зі своїм сховищем, наприклад `--storage sqlite`; проміжні рядки поступу пропущено):

```
Сховище: sqlite; пристроїв у маніфесті: 51; знахідок: 3
Завантажую tuning.parquet у сховище (до кількох хвилин)…
…
  [sqlite] завантажено 1 157 147 рядків за 3.4 с — файл 90 МБ
Телеметрії у сховищі: 1 157 147 рядків.
Рахую підсумки за годину (таблиця hourly)…
Таблиці devices, units, hourly і findings записано.

Тепер запустіть Grafana (якщо ще не запущена):
    docker compose -f dashboard/grafana/docker-compose.yml up -d
і відкрийте http://localhost:3000
```

Якщо ваше сховище **DuckDB**, скрипт напише `Grafana не читає DuckDB, тому телеметрія переноситься у SQLite.` і створить у папці проєкту файл `iot.sqlite` (для варіанта — близько 0,5 ГБ). Для InfluxDB і TimescaleDB дані вже у сховищі: скрипт лише дописує туди маніфест і знахідки.

Змінили `findings.json` — запустіть цю команду ще раз і оновіть сторінку в браузері.

Увага. `run_all.py` і `measure.py` замінюють дані у вашому сховищі тим набором, на якому їх запускали. Щоб Grafana знову показувала ваш варіант, повторіть команду з ключем `--reload`:

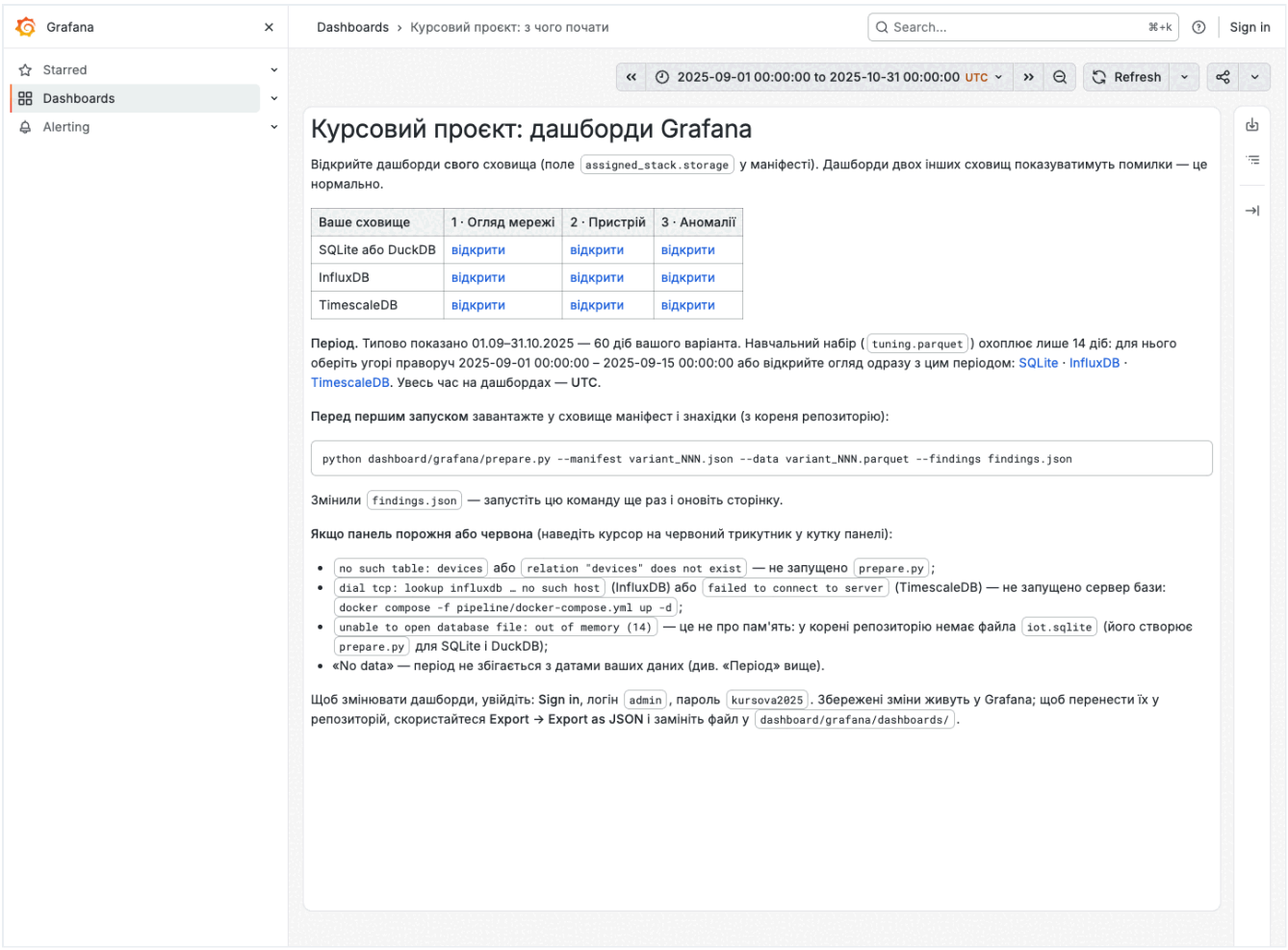
```
python dashboard/grafana/prepare.py --manifest variant_NNN.json --data variant_NNN.parquet
--findings findings.json --reload
```

Крок 2. Запуск Grafana

```
docker compose -f dashboard/grafana/docker-compose.yml up -d
```

Перший запуск триває близько хвилини (завантаження образу), наступні — кілька секунд. Може з'явитися попередження `a network with name kursova_default exists but was not created for project …` — це нормально: Grafana навмисно під'єднується до мережі ваших сховищ.

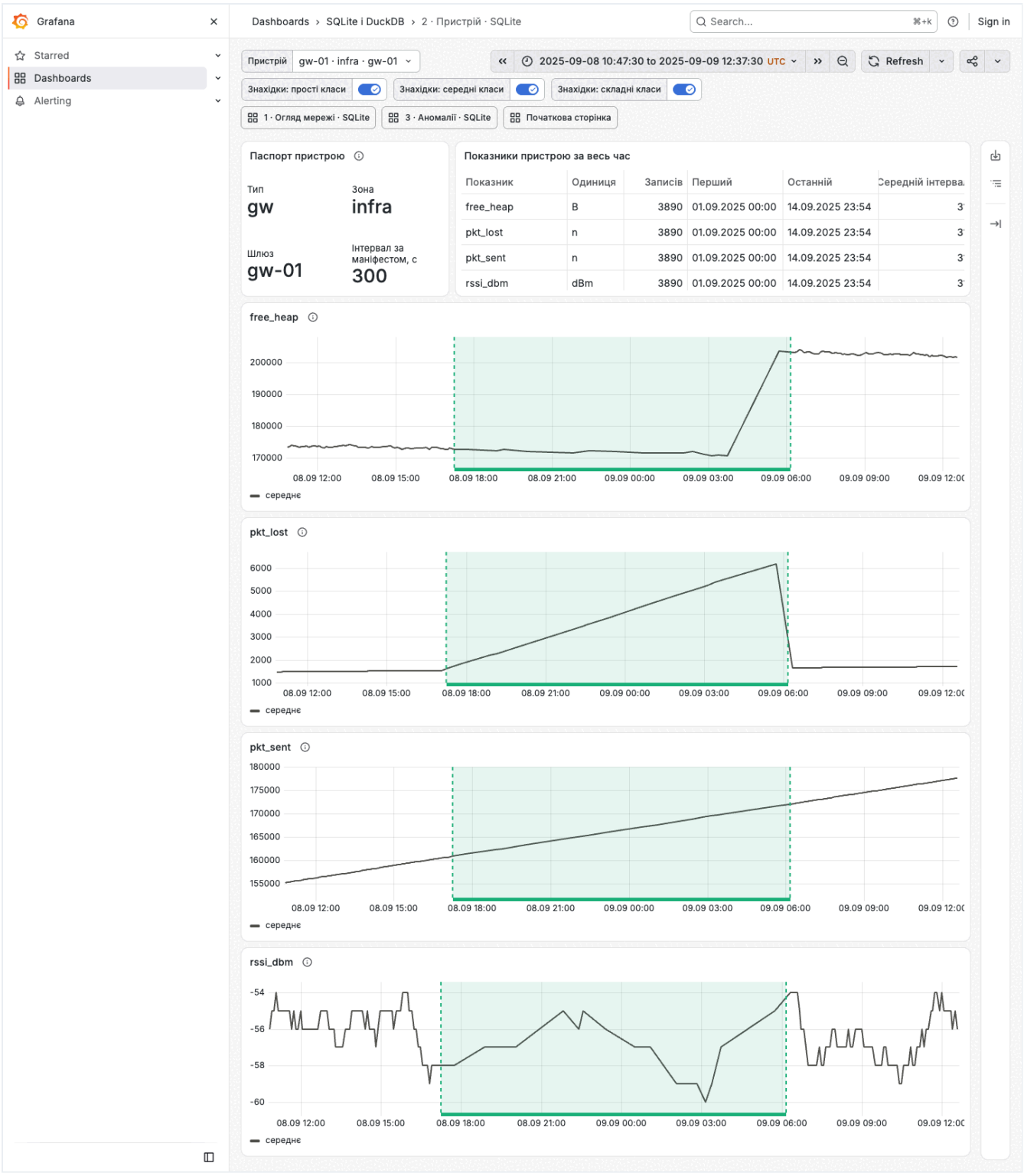
Відкрийте http://localhost:3000 — початкова сторінка відкривається без входу:

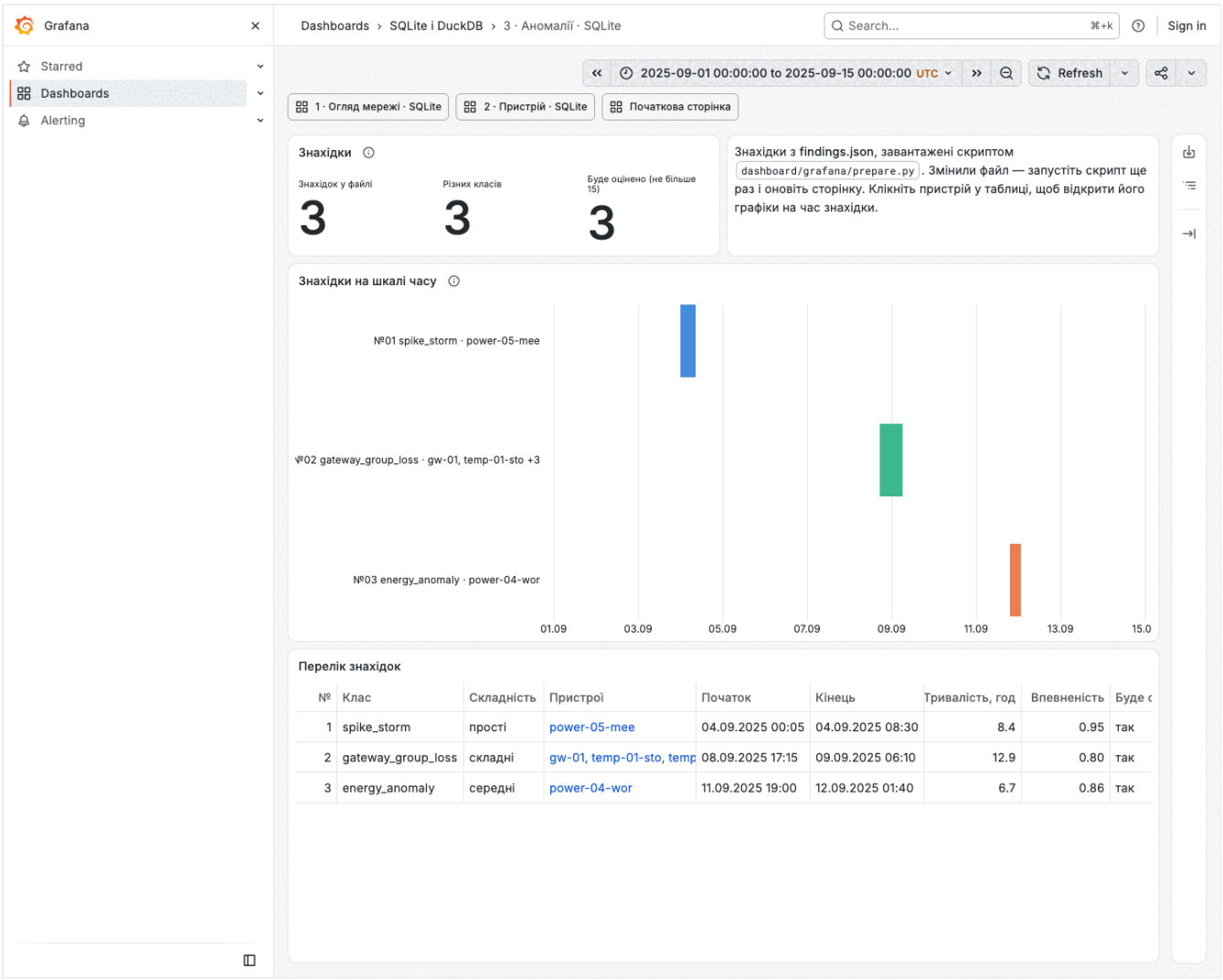


Відкривайте дашборди **свого** сховища з таблиці: дашборди двох інших показуватимуть помилки, так і має бути. Для навчального набору виберіть угорі праворуч період 2025-09-01 – 2025-09-15 (або скористайтеся посиланнями на сторінці): типово показано 60 діб вашого варіанта.



Група	Пристроїв у маніфесті	Пристроїв у даних	Повідомлень	Доступність, %	Останнє повідомл
infra	3	3	47788	99.9	14.09.2025 23:55
lobby	12	12	295876	100.0	14.09.2025 23:59
meeting-1	8	8	170768	99.9	14.09.2025 23:59
meeting-2	7	7	146192	99.0	14.09.2025 23:59
storage	15	15	344049	100.0	14.09.2025 23:59
workshop	6	6	150140	100.0	14.09.2025 23:59
немає в маніфесті	0	1	2334	11.9	03.09.2025 12:07


Grafana
Starred
Dashboards
Alerting
Dashboards › SQLite і DuckDB › 2 · Пристрій · SQLite
Search...
⌘+k
Sign in
Пристрій
gw-01 · infra · gw-01
2025-09-08 10:47:30 to 2025-09-09 12:37:30 UTC
Refresh
Знахідки: прості класи
Знахідки: середні класи
Знахідки: складні класи
1 · Огляд мережі · SQLite
3 · Аномалії · SQLite
Початкова сторінка
Паспорт пристрою
Тип
gw
Зона
infra
Шлюз
gw-01
Інтервал за маніфестом, с
300
Показники пристрою за весь час
Показник
Одиниця
Записів
Перший
Останній
Середній інтерва
free_heap
B
3890
01.09.2025 00:00
14.09.2025 23:54
pkt_lost
n
3890
01.09.2025 00:00
14.09.2025 23:54
pkt_sent
n
3890
01.09.2025 00:00
14.09.2025 23:54
rssi_dbm
dBm
3890
01.09.2025 00:00
14.09.2025 23:54
free_heap
200000
190000
180000
170000
08.09 12:00
08.09 15:00
08.09 18:00
08.09 21:00
09.09 00:00
09.09 03:00
09.09 06:00
09.09 09:00
середнє
pkt_lost
6000
5000
4000
3000
2000
1000
середнє
pkt_sent
180000
175000
170000
165000
160000
155000
середнє
rssi_dbm
-54
-56
-58
-60
середнє



Меню самої Grafana — англійською (української в ній немає), підписи дашбордів — українською.

Зупинка і зміни

```
docker compose -f dashboard/grafana/docker-compose.yml stop
```

Щоб змінювати дашборди, натисніть **Sign in** угорі праворуч (`admin` / `kursova2025`), далі **Edit** і **Save dashboard**. Щоб перенести зміни у свій репозиторій: **Export** → **Export as JSON** → **Save to file** і замініть файл у `dashboard/grafana/dashboards/<ваше сховище>/`.

Якщо панель червона або порожня

Наведіть курсор на червоний трикутник у кутку панелі, щоб прочитати помилку:

Помилка	Що зробити
`no such table: devices` (SQLite) / `relation "devices" does not exist` (TimescaleDB)	не запущено `prepare.py` — запустіть
`lookup influxdb … no such host` (InfluxDB) / `failed to connect to server` (TimescaleDB)	не запущено сервер сховища: `docker compose -f pipeline/docker-compose.yml up -d`, потім **Refresh**
`unable to open database file: out of memory (14)`	це **не** про пам'ять: у папці проєкту немає `iot.sqlite` — запустіть `prepare.py`
«No data» на всіх панелях	вибраний період не збігається з датами ваших даних
`port is already allocated` під час `docker compose up`	порт 3000 зайнятий іншою програмою: закрийте її або змініть у `dashboard/grafana/docker-compose.yml` рядок `"127.0.0.1:3000:3000"` на `"127.0.0.1:3001:3000"` і відкривайте http://localhost:3001

6.5. Як читати огляд мережі

Пристрої надсилають повідомлення з різною частотою (від кількох на хвилину до одного на 5 хвилин), тож «скільки повідомлень за годину» між пристроями не порівняти. Тому карта показує **відсоток від звичайного для цього пристрою**: сірий — як завжди, червоний — менше або нічого, синій — більше, ніж зазвичай.

Знахідки розфарбовано за **складністю класу**: синій — прості, помаранчевий — середні, зелений — складні (ті самі групи, що й у правилах оцінювання). Назва класу завжди підписана поруч.

Увага. Дашборд — це **інструмент огляду**, а не детектор: він показує, де варто придивитися, але нічого не класифікує й не записує знахідок. Кожну знахідку ви обґрунтовуєте самі (стадія 5).

6.6. Зробіть дашборд своїм

На демонстрації вас оцінюють за тим, чи працює дашборд на **ваших** даних і чи можете ви пояснити кожен його елемент. Тому:

1. Запустіть дашборд на своєму варіанті і своєму `findings.json`.
2. Пройдіть усі три сторінки й переконайтеся, що розумієте, що показує кожна панель і як вона рахується (код відкритий — подивіться запити).
3. Перевірте вимогу «придатний для того, хто не є автором»: попросіть одногрупника без вашої допомоги знайти на дашборді одну з ваших знахідок і відкрити графік відповідного пристрою. Що виявилося незрозумілим — виправте (підпис, пояснення, порядок панелей).
4. Підготуйте показ дашборда для захисту: на нього в доповіді є близько хвилини (див. крок 8.4), а запитання можуть стосуватися будь-якої панелі.

У звіт

- Знімки всіх трьох сторінок — у додатки.
- У підрозділі 3.3 для кожної знахідки — знімок сторінки пристрою з вікном цієї знахідки.
- Одним абзацом: що показує кожна сторінка і чому саме так (наприклад, чому на огляді відсоток від звичайного, а не кількість повідомлень).

Контрольний список стадії 6

- □ Дашборд призначеного інструмента запускається на вашому варіанті.
- □ На сторінці аномалій — ваші знахідки з `findings.json`.
- □ Перехід зі знахідки на сторінку пристрою працює.
- □ Хтось інший зміг знайти на дашборді вашу знахідку без підказок.
- □ Знімки трьох сторінок збережено для записки.

Крок 7. Пояснювальна записка

Мета: описати зроблене так, щоб читач без вашого коду зрозумів, що ви побудували, що виміряли і що знайшли. Записка дає 20 балів зі 100, а без неї не зараховується решта.

Зразок записки: Word · PDF. Він показує будову, оформлення й обсяг кожної частини. Жовтим у ньому позначено те, що вписуєте ви, сірим курсивом — приклади з навчального набору, які треба замінити своїми. Текст без позначок — орієнтир для обсягу й стилю: пишіть свій, своїми словами.

7.1. Обсяг і структура

Обсяг — **25–30 сторінок** без додатків. Менше 20 сторінок — підстава для оцінки «незадовільно» (п. 8.4 вказівок).

Частина	Сторінок	Що писати
Титульний аркуш	1	за зразком кафедри
Картка результатів	1	таблиця з п. 7.2 цього посібника
Зміст	1	автоматичний зміст редактора
Вступ	2–3	див. 7.3
Розділ 1. Теоретична частина	7–8	див. 7.4
Розділ 2. Проєктування та реалізація	9–10	див. 7.5
Розділ 3. Аналіз результатів	6–7	див. 7.6
Висновки	1–2	див. 7.7
Список використаних джерел	— (не менше 20 джерел)	див. 7.8
Додатки	без обмежень	код, `findings.json`, знімки дашборда

Оформлення (шрифт, поля, нумерація) — за загальними вимогами кафедри до курсових робіт. Якщо їх вам не видали, запитайте керівника.

Пишіть у науковому стилі, від третьої особи («у роботі виміряно…», а не «я виміряв…»). Кожне запозичення — з посиланням у квадратних дужках. Кожне виміряне число — з умовами вимірювання: обладнання (процесор, пам'ять), обсяг даних, версії програм (розд. 5 вказівок). Це прямо впливає на оцінку записки (п. 8.2 вказівок).

Порада. Пишіть записку **паралельно** зі стадіями, а не наприкінці: після кожної стадії одразу переносьте в записку таблиці й рисунки, позначені в посібнику як «У звіт».

7.2. Картка результатів

Розміщується одразу після титульного аркуша. Значення беруться з виводу `selfcheck.py` (крок 0.7) і з ваших вимірювань. Зразок — для навчального набору; у вас будуть ваші числа.

Поле	Значення
Варіант	0 (навчальний набір)
Кількість записів	1 157 147
Пристроїв у маніфесті	51
Пристроїв у даних	52
SHA-256 переліку пристроїв	1a33058d845b45c2ce0ddff78318cad7cab1f1b8c9a25aed2e3cdbafc84b7098
Записів за показниками	див. таблицю нижче
Призначений стек	InfluxDB / DuckDB SQL / Matrix Profile / Streamlit
Точність / повнота	базовий детектор: … / …; призначений: … / … (стадія 3)
Пропускна здатність	… записів/с (стадія 2)

Увага. Картка, що не відповідає виданому варіанту, — підстава для оцінки «незадовільно». Не набирайте числа вручну: копіюйте їх із виводу програм.

Записи за показниками рахує така команда (замініть файл на свій):

```
python -c "import pyarrow.parquet as pq; t = pq.read_table('tuning.parquet').to_pandas();
print(t.groupby('metric', observed=True).size().to_string())"
```

7.3. Вступ (2–3 с.)

Дайте відповідь на п'ять питань, по абзацу на кожне:

1. Чому моніторинг пристроїв інтернету речей важливий (масштаб, відмови, атаки)?
2. Що є об'єктом і предметом дослідження? (Готові формулювання — у розділі 2 вказівок.)
3. Яка мета роботи? (Одне речення.)
4. Які завдання ви розв'язали? (Список із 5–7 пунктів, що відповідають стадіям.)
5. Що нового у вашій роботі? (Ваш стек, ваші вимірювання, ваші висновки.)

7.4. Розділ 1. Теоретична частина (7–8 с.)

Кожен підрозділ — 1–2 сторінки, з посиланнями на джерела у квадратних дужках, наприклад [3].

- **1.1. Архітектура систем інтернету речей.** Рівні (пристрої, шлюзи, мережа, хмара/сховище, застосунки) за ISO/IEC 30141. Де у цій архітектурі ваш вузол, брокер, сховище і дашборд.
- **1.2. Протоколи передавання телеметрії та їх захищеність.** MQTT (видавець-підписник, брокер, QoS, retain, Last Will), порівняння з CoAP і HTTP. Чим небезпечний MQTT без TLS — і посилання на ваш власний досвід зі стадії 1.
- **1.3. Характеристики великих даних, архітектури обробки.** «5V» (обсяг, швидкість, різноманітність, достовірність, цінність); пакетна і потокова обробка; рядкове і стовпцеве зберігання; часові ряди і чому для них існують окремі бази (InfluxDB, TimescaleDB).
- **1.4. Методи виявлення аномалій у часових рядах.** Статистичні (z-оцінка, медіана/MAD, контрольні карти), декомпозиція (STL), методи на основі відстані й щільності (LOF), ізоляції

(Isolation Forest), Matrix Profile. Окремо — **ваш призначений метод** докладніше. Метрики якості: точність, повнота, F1.

- **1.5. Модель загроз пристроїв інтернету речей.** Які загрози стосуються розумної будівлі; класи `rogue_device`, `beaconing`, `exfil_pattern` у термінах MITRE ATT&CK (джерело 16 у списку вказівок); OWASP IoT Top 10.

7.5. Розділ 2. Проєктування та реалізація (9–10 с.)

- **2.1. Обґрунтування архітектури конвеєра.** Схема: вузол → брокер → споживач → сховище → обробка → детектори → дашборд. Намалюйте її (draw.io, безкоштовно: https://app.diagrams.net/). Чому саме такі компоненти.
- **2.2. Реалізація вузла та протоколу передавання.** Схема підключення у Wokwi, формат повідомлення (Додаток А вказівок), Last Will, вимірювання JSON проти CBOR, результати дослідження захищеності каналу зі знімками.
- **2.3. Організація зберігання даних.** Ваше сховище: модель даних (таблиця/вимірювання, поля, індекси), як завантажуєте, де межа пам'яті і як ви її обходите.
- **2.4. Реалізація детекторів.** Базовий і призначений детектор: ідея, параметри, як ви їх підібрали на навчальному наборі.

Код у тексті — лише короткими фрагментами (до 10–15 рядків). Повний код — у додатках і в репозиторії.

7.6. Розділ 3. Аналіз результатів (6–7 с.)

- **3.1. Показники продуктивності конвеєра.** Таблиця вимірювань зі стадії 2 (розмір CSV/Parquet, стиснення, швидкість завантаження, час сканування і вибірки, пам'ять) з описом комп'ютера, на якому міряли.
- **3.2. Порівняння детекторів за точністю та повнотою.** Таблиця зі стадії 3; результати на реальних даних зі стадії 4: проблеми даних, вжиті заходи, зміна кількості хибних спрацювань.
- **3.3. Виявлені інциденти та методика їх встановлення.** Таблиця ваших знахідок (пристрої, час, клас, упевненість) і для кожної — **на основі чого** ви дійшли висновку, з рисунком.
- **3.4. Недоліки рішення та напрями вдосконалення.** Чесно: що не вийшло, які класи не знаходяться і чому, що зробили б інакше.

7.7. Висновки (1–2 с.)

По одному абзацу на кожне завдання зі вступу, з **числами**: «стовпцеве зберігання зменшило обсяг у … разів», «призначений детектор досяг повноти … на навчальному наборі», «у варіанті виявлено … інцидентів, з них … — складних». Без нових фактів, яких немає в розділах.

7.8. Список використаних джерел

Не менше 20 джерел, зокрема матеріали конференцій IEEE і чинні стандарти. Перші 18 можна взяти з розділу 10 вказівок, решту доберіть самі. Оформлення — за ДСТУ 8302:2015. Зразки:

1. ISO/IEC 30141:2024. Internet of Things (IoT) — Reference architecture. Geneva : ISO, 2024.
2. Liu F. T., Ting K. M., Zhou Z.-H. Isolation Forest. 2008 Eighth IEEE International Conference on Data Mining. Pisa, 2008. P. 413–422.
3. Rosner B. Percentage Points for a Generalized ESD Many-Outlier Procedure. Technometrics. 1983. Vol. 25, no. 2. P. 165–172.
4. Intel Berkeley Research Lab sensor data. URL: https://db.csail.mit.edu/labdata/labdata.html (дата звернення: 01.11.2026).

Увага. Набір даних Intel Lab **обов'язково** має бути у списку джерел (умова його використання).

7.9. Використання штучного інтелекту

Дозволено для коду, пошуку помилок і опрацювання літератури — із зазначенням. Достатньо абзацу у вступі або окремого підрозділу, наприклад:

> Під час роботи використано систему штучного інтелекту <назва> для пояснення помилок у коді конвеєра та перекладу фрагментів документації. Усі результати вимірювань і знахідки отримано власним конвеєром і перевірено автором.

Типові помилки

- Картка результатів набрана вручну з помилкою — **«незадовільно»**.
- Таблиці без підписів і без посилань на них у тексті.
- Знімки екрана, на яких нічого не видно: обрізайте до потрібної частини.
- Теоретична частина, переписана з джерел без посилань.
- Висновки без жодного числа.
- Список джерел із самих вебсторінок.

Контрольний список кроку 7

- □ 25–30 сторінок без додатків.
- □ Картка результатів одразу після титульного аркуша, числа скопійовано з виводу програм.
- □ У кожному розділі є таблиці й рисунки з підписами.
- □ Не менше 20 джерел, серед них Intel Lab, стандарти, статті IEEE.
- □ Використання ШІ зазначено.
- □ Додатки: код, `findings.json`, знімки дашборда.

Крок 8. Захист

Мета: показати, що проєкт ваш і що він працює. Захист — 10 балів, але головне інше: якщо ваш конвеєр не запуститься на незнайомих даних, це **«незадовільно» незалежно від решти** (п. 8.3 вказівок).

8.1. Що буде на захисті

1. **Доповідь до 7 хвилин** з показом працездатного дашборда.
2. **Демонстрація вузла** (складник «Вузол IoT, MQTT, захищеність каналу» оцінюють саме так): вузол у Wokwi, що публікує повідомлення (потрібен інтернет), і ваш власний брокер з TLS разом із `node/node.py` (працює без інтернету).
3. **Запуск на незнайомих даних.** Вам дадуть набір даних, якого ви ніколи не бачили (файли `variant_XXX.parquet` і `variant_XXX.json`, де XXX — номер, який вам назвуть; формат той самий, що й у вашого варіанта), і попросять запустити ваш конвеєр при комісії: завантажити дані у **ваше** сховище, прогнати пошук і отримати перелік знахідок.
4. **Запитання** — про ваш код, ваші рішення і ваші числа.

Приносьте **свій** ноутбук з усім установленим. Інтернету на захисті може не бути — перевірте, що все працює без нього (крім Wokwi). На випадок, якщо Wokwi не відкриється, майте знімки екрана або коротке відео роботи вузла.

8.2. Один скрипт для всього

Щоб запуск при комісії зайняв хвилини, а не пів години, усі кроки зведено в один скрипт `run_all.py` — він уже є в заготовці:

```
python run_all.py --data variant_XXX.parquet --manifest variant_XXX.json --storage duckdb
```

Замість `duckdb` вкажіть **своє** сховище — те, що записане у вашому `variant_NNN.json` (`duckdb`, `sqlite`, `influxdb` або `timescaledb`). У маніфесті незнайомого набору призначено чужий стек; скрипт про це повідомить, але працюватиме з вашим сховищем, як і має бути.

Скрипт по черзі:

1. завантажує дані у ваше сховище;
2. друкує картку результатів — числа з вашого сховища;
3. шукає інциденти — **вашою** функцією `find_incidents` з файла `hunt.py`;
4. записує `findings.json`, перевіряє його через `selfcheck.py` і друкує, скільки знахідок кожного класу.

Кроки 1, 2 і 4 готові. Крок 3 — ваша робота.

Ваш `hunt.py`

На стадії 5 ви шукали інциденти, переглядаючи графіки й записуючи знахідки в таблицю. Для захисту той самий пошук має стати **кодом**: на незнайомих даних таблиці, складеної вручну, у вас

не буде.

Якщо файла `hunt.py` ще немає, `run_all.py` надрукує заготовку:

```
def find_incidents(storage, manifest):
    """storage — відкрите й заповнене сховище; manifest — маніфест (словник)."""
    findings = []
    # Приклад одного запису:
    # findings.append({
    #     "devices": ["temp-03-lob"],
    #     "class": "stuck_at",
    #     "start": "2025-09-14T08:30:00Z",
    #     "end": "2025-09-14T16:00:00Z",
    #     "confidence": 0.8,
    #     "evidence": "температура не змінювалася 7,5 год",
    # })
    return findings
```

Збережіть її у файл `hunt.py` поруч із `run_all.py` і наповніть. Усередині функції вам доступні:

Що	Як отримати
перелік пристроїв у даних	`storage.devices()`
перелік показників	`storage.metrics()`
ряд одного пристрою	`ts, val = storage.series("temp-03-lob", "temperature")` — масиви часу (секунди UTC) і значень
пристрої з маніфесту, їхні зони й шлюзи	`manifest["devices"]`

Знахідки — ті самі поля, що й у `my_findings.csv` зі стадії 5. Найпростіший шлях: для кожного класу, який ви знаходили, перенесіть у функцію ті самі правила, за якими ви вирішували вручну (пороги, вікна, порівняння з сусідами), і перевірте на навчальному наборі, що вона знаходить відомі інциденти з `tuning_truth.json`.

Порада (InfluxDB і TimescaleDB). Кожен виклик `storage.series()` — це запит до бази в Docker: на InfluxDB близько пів секунди. Якщо ваш `hunt.py` бере той самий ряд для кількох перевірок, читайте кожен ряд лише раз і зберігайте у словнику:

```
cache = {}

def series(storage, device, metric):
    if (device, metric) not in cache:
        cache[(device, metric)] = storage.series(device, metric)
    return cache[(device, metric)]
```

і далі пишіть `series(storage, "temp-03-lob", "temperature")` замість `storage.series(...)`. На перевірці посібника це скоротило пошук на InfluxDB з 9 хвилин до менш ніж двох.

Перевірка на навчальному наборі

У `tuning.json` призначено InfluxDB, тому для репетиції вкажіть **своє** сховище ключем `--storage` (тут — DuckDB):

```
python run_all.py --data tuning.parquet --manifest tuning.json --storage duckdb
```

Справжній вивід, поки `hunt.py` ще не написано (заготовку, яку ви вже бачили вище, пропущено):

```
У маніфесті цього набору записано сховище «InfluxDB». Це нормально:
конвеєр працює з вашим сховищем «duckdb», як і має бути на захисті.

[1/4] Завантаження у сховище «duckdb»
  [duckdb] завантажено 1 157 147 рядків за 2.3 с — подання (view) без копіювання даних
  завантажено 1 157 147 записів за 2.4 с

[2/4] Картка результатів
  Кількість записів          : 1 157 147
  Пристроїв у маніфесті      : 51
  Пристроїв у даних          : 52
  SHA-256 переліку пристроїв :
1a33058d845b45c2ce0ddff78318cad7cab1f1b8c9a25aed2e3cdbafc84b7098

[3/4] Пошук інцидентів (hunt.py)

Файла hunt.py ще немає. Створіть його поруч із run_all.py: скопіюйте все між
…

[4/4] findings.json і самоперевірка
Картка результатів, перерахована з ваших даних:
  Кількість записів          : 1 157 147
  Пристроїв у маніфесті      : 51
  Пристроїв у даних          : 52
  SHA-256 переліку пристроїв :
1a33058d845b45c2ce0ddff78318cad7cab1f1b8c9a25aed2e3cdbafc84b7098
  (розбіжність на 1 — це не помилка генерації)
  Призначений стек           : storage=InfluxDB, processing=DuckDB SQL, detector=Matrix
Profile, dashboard=Streamlit

Знахідок у файлі: 0
  ПОПЕРЕДЖЕННЯ: перелік знахідок порожній

Файл придатний до перевірки.
Це не означає, що знахідки правильні — лише що формат коректний.

Готово за 29 с. Знахідки за класами:
  (жодної — ваш пошук ще не записано у hunt.py)
```

Коли `hunt.py` готовий, на кроці 3 з'явиться рядок `знайдено N за … с`, а наприкінці — кількість знахідок кожного класу.

Увага. `run_all.py` перезаписує `findings.json` у папці проєкту. Якщо ви зібрали `findings.json` для здачі на стадії 5, збережіть його копію або запускайте репетицію з ключем `--out rehearsal.json`.

8.3. Репетиція

За тиждень до захисту зробіть повну репетицію, ніби ви вже при комісії:

- □ Перезавантажте комп'ютер (щоб нічого не лишилося «запущеним випадково»).
- □ Якщо ваше сховище — InfluxDB чи TimescaleDB: запустіть Docker Desktop і переконайтеся, що контейнери піднімаються командою зі стадії 2.
- □ Активуйте віртуальне середовище.
- □ Запустіть `run_all.py` на **навчальному наборі** з `--storage` вашого сховища (команда з 8.2) — він має пройти від початку до кінця без жодного втручання і знайти відомі інциденти.

- □ Засічіть час. Якщо довше 10 хвилин — подумайте, що прискорити.
- □ Запустіть дашборд і пройдіть по всіх його частинах. Для Grafana спершу перезавантажте у сховище свій варіант: `run_all.py` замінив його навчальним набором (команда — у 6.4, з ключем `--reload`).
- □ Покажіть самі собі вузол: Wokwi і власний брокер з `node.py`.
- □ Вимкніть Wi-Fi і повторіть — чи нічого не залежить від інтернету?

Увага. Найчастіша причина провалу — не код, а середовище: Docker не запущений, віртуальне середовище не активоване, файл лежить не в тій папці. Репетиція знаходить саме ці проблеми.

8.4. Доповідь на 7 хвилин

Приблизно один слайд на хвилину:

Хв	Слайд
1	Задача і ваш варіант: скільки записів, пристроїв, ваш стек
2	Архітектура конвеєра (схема з розділу 2.1 записки)
3	Вузол і захищеність каналу: що було видно до TLS і після
4	Сховище: таблиця вимірювань, межа пам'яті
5	Детектори: точність і повнота на навчальному і реальних даних
6	Знайдені інциденти: таблиця і 1–2 найцікавіші приклади з графіками
7	Показ дашборда і висновки

8.5. Запитання, які часто ставлять

- Чому для цього класу інцидентів ви обрали саме такий детектор?
- Що станеться з вашим конвеєром, якщо даних буде в 100 разів більше?
- Покажіть у коді, де ви рахуєте точність і повноту.
- Чому ця знахідка — `sensor_swap`, а не два окремі `drift`?
- Що бачив сторонній клієнт до TLS і що — після?
- Як ви перевіряли, що спрацювання на даних Intel Lab хибні?

Відповідати треба своїми словами і з вашими числами. Знати напам'ять формули не обов'язково, а розуміти, що робить ваш код, — обов'язково.

Типові проблеми і як їх виправити

Знайдіть у списку текст помилки, який бачите. Якщо нічого не підходить — скопіюйте **повний** текст помилки (а не знімок екрана) і надішліть керівнику разом із командою, яку ви виконували, і назвою вашої операційної системи.

Python і бібліотеки

«Python was not found; run without arguments to install from the Microsoft Store…» / «'python' is not recognized…» (Windows). Python не встановлено або не додано до PATH (а Windows підставляє замість нього посилання на Microsoft Store). Перевстановіть Python з позначкою **Add python.exe to PATH** (крок 0.2), закрийте й знову відкрийте PowerShell. Тимчасово можна користуватися командою `py` замість `python`.

«command not found: python» (macOS, Linux). Пишіть `python3`. Усередині активованого віртуального середовища працює і `python`.

«error: externally-managed-environment» під час `pip install`. Ви встановлюєте бібліотеки поза віртуальним середовищем. Активуйте його (крок 0.5) і повторіть.

«ModuleNotFoundError: No module named …». Або не активоване віртуальне середовище (немає `(.venv)` на початку рядка), або бібліотеку не встановлено. Активуйте середовище і виконайте `pip install` з назвою модуля, яку підказує помилка або посібник.

«running scripts is disabled on this system» (Windows). Див. крок 0.5: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

Помилка компіляції під час `pip install` (довгий червоний текст з `error: Microsoft Visual C++ … is required` або `gcc`). Для вашої версії Python немає готової збірки бібліотеки. Найпростіше — встановити Python версії, рекомендованої в кроці 0.2, і створити віртуальне середовище заново.

`pip install "dask[dataframe]>=2024.8"` на Windows з Python 3.15 зупиняється на `pyyaml` (`Failed building wheel for pyyaml` або довгий червоний текст). Для Python 3.15 на Windows ще немає готової збірки PyYAML, яку потребує Dask. Встановіть Python 3.14 (крок 0.2), створіть віртуальне середовище заново і повторіть.

Файли і папки

«FileNotFoundError: … tuning.parquet» / «Файл … не знайдено». Ви не в папці проєкту або файл лежить в іншому місці. Виконайте `ls` і переконайтеся, що бачите і `selfcheck.py`, і файл даних.

Кирилиця або пробіли в шляху (Windows). Деякі інструменти, особливо Docker, плутаються зі шляхами на кшталт `C:\Users\Олена\Мої документи`. Тримайте проєкт у простій папці, наприклад `C:\kursova-iot`.

Файл `.parquet` не відкривається в Excel. Так і має бути: Parquet читають програми (pandas, DuckDB тощо), а не табличні редактори.

Завантажений файл має назву на кшталт `variant_047 (1).parquet`**.** Браузер додав номер, бо файл уже був у «Завантаженнях». Перейменуйте його назад на `variant_047.parquet`.

Wokwi

Симуляція довго не стартує. Перша компіляція триває до хвилини. Якщо довше — оновіть сторінку і натисніть ▶ ще раз.

Помилка компіляції `DHT.h: No such file or directory` **або подібна.** Немає файла `libraries.txt` або в ньому помилка в назві бібліотеки. Порівняйте з `firmware/libraries.txt` у заготовці.

Монітор нескінченно друкує `WiFi: підключення.....`**.** У налаштуваннях мережі має бути `Wokwi-GUEST` без пароля і канал 6, як у заготовці.

`MQTT: … помилка, rc=-2`**.** Вузол не може дістатися брокера. Перевірте назву `broker.hivemq.com`, порт `1883` і зачекайте: публічний брокер іноді перевантажений.

Проєкт зник після закриття вкладки. Ви працювали без входу в акаунт. Зареєструйтеся і натискайте **Save**.

MQTT

Веб-клієнт підключився, але повідомлень немає. Найчастіше — помилка в топіку. У прошивці номер варіанта без нулів попереду (`kursova/47/…`), і в підписці має бути так само: `kursova/47/#`. Також перевірте, що симуляція у Wokwi працює.

Повідомлення з'являються і зникають, вузол постійно перепідключається. Два клієнти з однаковим ідентифікатором виштовхують один одного. Заготовка додає до `DEVICE_ID` випадковий суфікс — не прибирайте його.

`[status] …: ОФЛАЙН (збережений брокером статус)` **одразу після підписки.** Так і має бути: брокер зберігає останній статус вузла (прапорець `retain`) і віддає його кожному новому підписникові. Свіжий статус прийде, щойно вузол підключиться.

Власний брокер Mosquitto (стадія 1, крок 3)

`mosquitto` **/** `mosquitto_passwd` **— «is not recognized…» (Windows).** У цьому вікні PowerShell не виконано рядок `$env:Path += ";C:\Program Files\Mosquitto"`. Його треба повторювати в кожному новому вікні.

`mosquitto: command not found` **(macOS).** На Mac з процесором Intel брокер лежить у `/usr/local/sbin`, якої немає в PATH. Виконайте `echo 'export PATH="/usr/local/sbin:$PATH"' >> ~/.zprofile` і відкрийте Термінал заново.

`Error: Address already in use` **(або «Only one usage of each socket address…» на Windows).** Порт 1883 уже зайнятий іншим брокером:

- Windows — службою «Mosquitto Broker» (ви не зняли позначку «Service»). У PowerShell **від імені адміністратора**: `Stop-Service mosquitto` і `Set-Service mosquitto -StartupType Manual`.
- macOS — `brew services`: виконайте `brew services stop mosquitto`.
- Linux — системною службою: `sudo systemctl stop mosquitto` і `sudo systemctl disable mosquitto`.
- Або у вас уже відкрите інше вікно з вашим брокером — закрийте його.

`Error: Unable to open pwfile "broker/passwd".` Або ви запустили брокер не з папки проєкту (наприклад, перейшли в `broker`), або ще не створили файл паролів. Поверніться в папку проєкту (там, де `README.md`) і, якщо треба, створіть файл: `mosquitto_passwd -c broker/passwd node`.

`Error: Unable to load CA certificates. Check cafile "broker/certs/ca.crt".` Не створено сертифікати або брокер запущено не з тієї папки. Виконайте `python broker/make_certs.py` у папці проєкту.

`Error: Unable to open file broker/passwd for writing. File exists.` Ви вдруге використали `-c`. Для другого й наступних користувачів пишіть команду без `-c`: `mosquitto_passwd broker/passwd reader`.

Після додавання другого користувача ніхто не може підключитися (`not authorised`) — Linux. Mosquitto 2.0 з ключем `-c` створює файл заново і видаляє попередніх користувачів. Додайте їх знову, без `-c`.

`Unable to decode password salt for user …, removing entry`. Файл паролів створено іншою версією Mosquitto. Видаліть `broker/passwd` і створіть його заново командою `mosquitto_passwd` з тієї самої інсталяції, що й брокер.

Брокер не запускається після того, як ви дописали коментар у `mosquitto.conf`. Mosquitto вважає текст після значення частиною значення: коментар має стояти **окремим рядком**, що починається з `#`.

Запуск із `sudo` — брокер не може прочитати файли. Запущений від імені адміністратора, Mosquitto перемикається на службового користувача, який не бачить файлів у вашій домашній папці. Запускайте **без** `sudo`.

`Connection error: Connection Refused: not authorised`. Без імені й пароля (або з неправильним паролем) брокер нікого не пускає — у кроці 3.8 саме це і треба отримати. Законний підписник вказує `--username reader`.

`mosquitto_sub` **на порту 8883:** `Error: Protocol error` **або** `A TLS error occurred.` Не вказано `--cafile broker/certs/ca.crt` або вказано `ca.crt` з іншого запуску `make_certs.py`. Якщо ви перестворили сертифікати (`--force`), перезапустіть брокер.

`Error: Bad file descriptor` **(mosquitto_sub 2.1).** Брокер не запущений або вказано не той порт. Попри дивний текст, це просто «нема з ким з'єднатися».

Вузол чи `consumer.py` пише `!! Сертифікат брокера не пройшов перевірку: self-signed certificate…`. Не вказано `--cafile` або це `ca.crt` з іншого запуску `make_certs.py`.

`!! Брокер … закрив з'єднання, не відповівши на CONNECT.` Ви підключаєтеся без TLS до TLS-порту 8883. Додайте `--cafile broker/certs/ca.crt` або використайте порт 1883.

`!! TLS-з'єднання з 127.0.0.1:1883 не вдалося`. Навпаки: TLS на порт без шифрування. TLS — порт 8883.

Запит пароля «зависає» в Git Bash (Windows). Git Bash не вміє приховано читати пароль. Запускайте вузол у PowerShell.

Wireshark

Немає інтерфейсу loopback (Windows). Не встановлено Npcap. Перевстановіть Wireshark і погодьтеся встановити Npcap. Потрібний інтерфейс називається **Adapter for loopback traffic capture**.

«You don't have permission to capture on that device» або порожній перелік інтерфейсів (macOS). Не встановлено ChmodBPF: двічі клацніть **Install ChmodBPF.pkg** у вікні завантаженого `.dmg`, потім вийдіть із системи й увійдіть знову.

Те саме на Linux. Ви не в групі `wireshark`: `sudo usermod -a -G wireshark $USER`, потім вийдіть із системи й увійдіть знову. Якщо під час встановлення ви відповіли «No»: `sudo dpkg-reconfigure wireshark-common`.

Фільтр `mqtt` нічого не показує. Або ви записуєте не той інтерфейс (Wi-Fi чи Ethernet замість loopback), або трафік іде на порт 8883 — тоді так і має бути: це TLS, використайте фільтр `tls`.

У полі Message шістнадцяткові цифри замість JSON. Клацніть правою кнопкою **MQ Telemetry Transport Protocol → Protocol Preferences → Show Message as text**.

Docker (InfluxDB, TimescaleDB, Grafana)

`Cannot connect to the Docker daemon … Is the docker daemon running?` **(macOS, Linux) або** `error during connect: … dockerDesktopLinuxEngine` **(Windows).** Docker Desktop не запущено. Запустіть його і дочекайтеся, поки він покаже, що двигун працює (Engine running).

`no configuration file provided: not found`. Команду `docker compose` виконано не з папки проєкту. Поверніться в неї і пишіть `docker compose -f pipeline/docker-compose.yml up -d`.

`Ports are not available: … address already in use` / `port is already allocated`. Порт 5432 (TimescaleDB) чи 8086 (InfluxDB) уже зайнятий — найчастіше встановленим раніше PostgreSQL. Зупиніть ту програму (Windows: застосунок «Служби» → `postgresql-x64-…` → Зупинити, тип запуску «Вручну»). Якщо зупинити не можна, змініть ліве число порту в `pipeline/docker-compose.yml`, наприклад `"127.0.0.1:5433:5432"`, і передавайте нову адресу скриптам `measure.py`, `run_all.py` і `prepare.py`: `--dsn "host=localhost port=5433 user=student password=kursova2025 dbname=iot"` (TimescaleDB) або `--url http://localhost:8087` (InfluxDB, якщо змінили `"127.0.0.1:8087:8086"`).

`Docker Desktop - Unexpected WSL error` **або помилки з** `HCS` **у тексті (Windows).** Не встановлено або застарів WSL 2, чи вимкнено віртуалізацію. У PowerShell від імені адміністратора: `wsl --install` (потім перезавантаження) або `wsl --update`. Якщо не допомогло — увімкніть віртуалізацію (Intel VT-x або AMD-V/SVM) у налаштуваннях UEFI/BIOS.

`permission denied while trying to connect to the Docker daemon socket` **(Linux).** Додайте себе до групи `docker`: `sudo usermod -aG docker $USER`, потім вийдіть із системи й увійдіть знову.

`docker-compose: command not found` **або помилки про** `name:`**.** Старий `docker-compose` (через дефіс) не підходить. Пишіть `docker compose` — через пробіл.

Контейнер у стані `(unhealthy)` **або** `(health: starting)`**.** Зачекайте хвилину й повторіть `docker compose -f pipeline/docker-compose.yml ps`. Перший запуск довший: Docker завантажує образи.

Детектори (стадія 3)

Скрипт «завис» на першому запуску. Перший запуск після встановлення бібліотек триває до 30–40 секунд: будується кеш шрифтів і компілюються бібліотеки. Matrix Profile щоразу думає 15–20 секунд — так і має бути.

`Вкажіть пристрій: --device <назва>.` Скрипт друкує перелік пристроїв і їхніх показників — виберіть із нього. Якщо в назві помилка, скрипт підкаже схожі назви.

`--step '5': потрібна тривалість з одиницею`**.** Тривалості пишуть з одиницею: `5min`, `30min`, `2h`, `1D`.

`для --json вкажіть ще й --class`**.** Вікна, записані у JSON, мають мати клас: додайте `--class` з назвою класу з розділу 7 вказівок.

`tuning_truth.json описує інший набір даних…` **або** `Відповіді є лише до навчального набору…`**.** `--evaluate` працює тільки на `tuning.parquet` з `tuning_truth.json`. Для вашого варіанта відповідей немає — так задумано.

`Не встановлено пакет «…»`**.** Встановіть названий пакет у віртуальне середовище: `pip install statsmodels` (або `scikit-learn`, `stumpy`, `matplotlib`).

`pip install stumpy` **завершується помилкою про** `llvmlite` **(Mac з процесором Intel).** Для Python 3.14 і 3.15 на Mac з Intel немає готової збірки. Встановіть Python 3.13 (крок 0.2), створіть віртуальне середовище заново і повторіть.

Один інцидент розпався на кілька вікон. Таке вікно рахується як одна правильна знахідка і кілька хибних. Збільште `--gap` (для рідких даних, як у шлюзів) або `--contamination` (Isolation Forest, LOF).

Isolation Forest чи LOF знаходять «аномалії» на здоровому пристрої. Вони завжди позначають частку точок `--contamination`. Дивіться на величину оцінки у стовпці «пік оцінки», а не лише на те, що вікно є.

Графік перезаписується. Повторний запуск з тим самим пристроєм і показником зберігає PNG під тією самою назвою. Щоб зберегти варіанти для записки, задавайте назву: `--png ewma_L7.png`.

Кирилиця у виводі

`UnicodeEncodeError` **під час запису виводу у файл (Windows).** Команда на кшталт `python realdata/intel_lab.py > report.txt` на Windows з англійською мовою системи не може записати українські літери. Перед запуском виконайте у тому самому вікні PowerShell `$env:PYTHONIOENCODING="utf-8"` або просто скопіюйте текст із вікна терміналу.

Замість українських літер «кракозябри» у старому вікні `cmd.exe`. Користуйтеся PowerShell або терміналом VS Code.